

UNIVERSITÀ
POLITECNICA
DELLE MARCHE

Corso di «Ingegneria del Software»

A.A. 2024/2025

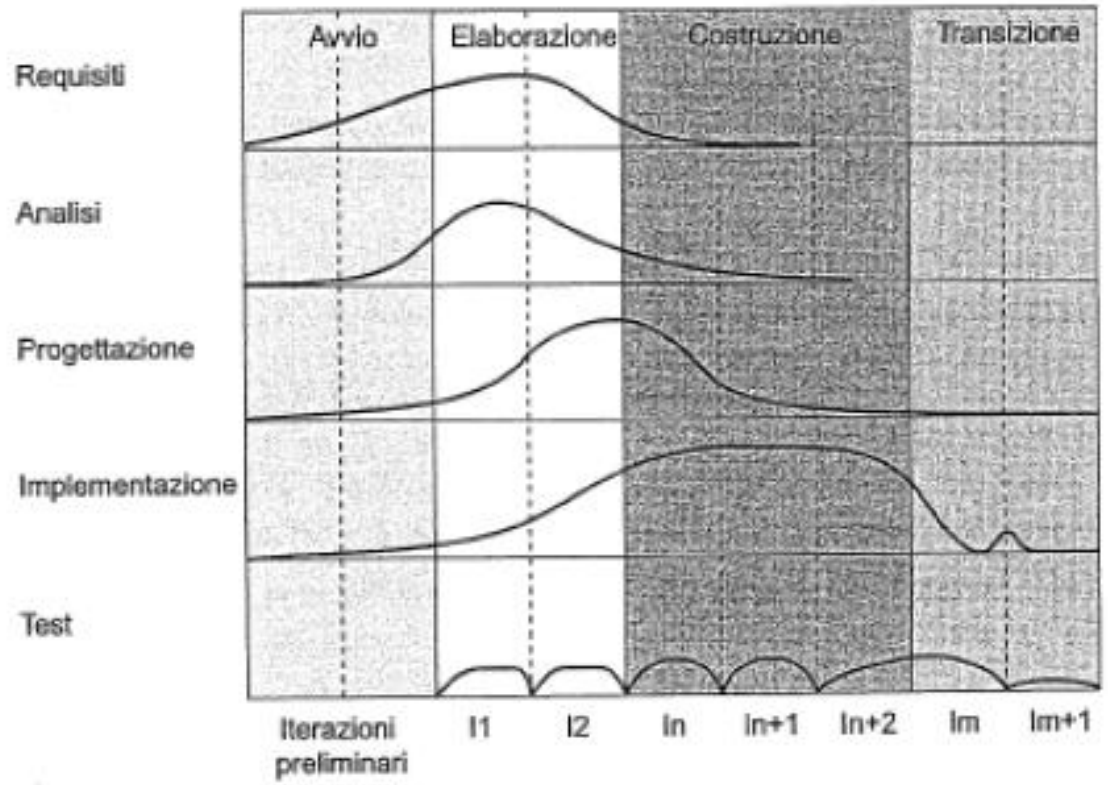
Parte IV: Progettazione

Prof. Domenico Ursino
d.ursino@univpm.it

Il flusso di lavoro della progettazione - Descrizione

- Il flusso di lavoro della progettazione costituisce la principale attività di modellazione durante l'ultima parte della fase di Elaborazione e la prima metà della fase di Costruzione.

- Come si vede dalla seguente figura, man mano che le attività di analisi si avvicinano alla conclusione, l'enfasi della modellazione si sposta sempre di più sulla progettazione:



Il flusso di lavoro della progettazione - Descrizione

- In molti casi l'analisi e la progettazione possono essere effettuate in parallelo.
- Tuttavia è molto importante distinguere chiaramente i manufatti di questi due flussi di lavoro, ovvero il modello di analisi e il modello di progettazione.
- Piuttosto che avere un gruppo di analisti e un gruppo di progettisti, UP consiglia di organizzare i gruppi di lavoro in modo verticale.
- Anziché organizzare il gruppo di lavoro per competenze professionali, UP preferisce adottare una logica incentrata sui manufatti consegnabili e sulle milestone.
- UP si concentra sugli "obiettivi" e non sulle "competenze".
- Il lavoro di analisi si concentra sulla creazione di un modello logico del sistema che fissi le funzionalità che questo deve fornire per soddisfare i requisiti dell'utente.
- Lo scopo delle attività di progettazione è quello di definire in modo completo come queste funzionalità debbano essere implementate.

Il flusso di lavoro della progettazione - Descrizione

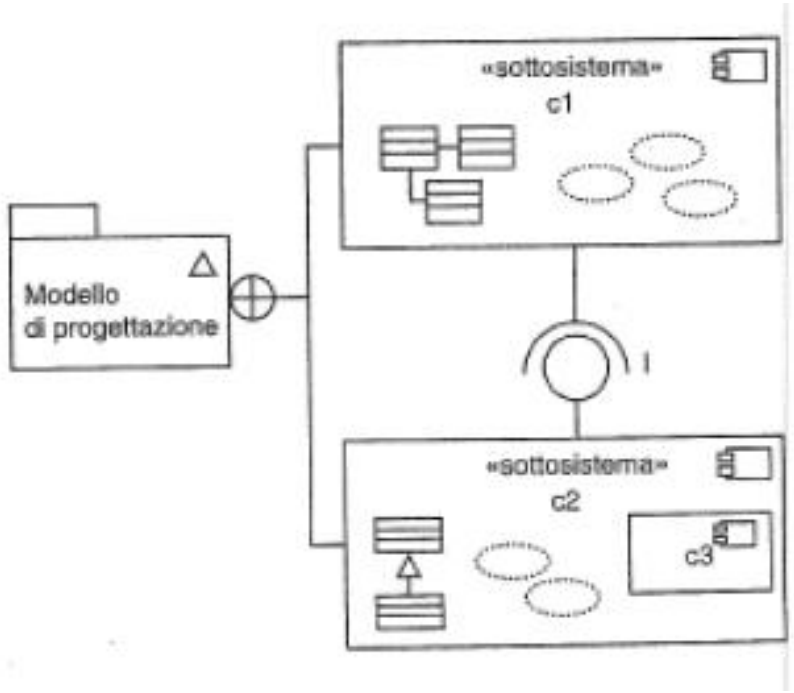
- La differenza tra i due tipi di attività può essere compresa se si pensa al dominio del problema da una parte e al dominio delle soluzioni dall'altra.

- Durante la progettazione i progettisti OO devono prendere delle decisioni in merito a questioni tecniche strategiche e creare un modello di progettazione appropriato.

- Il capo progetto e l'architetto dovrebbero, inoltre, definire degli standard per risolvere le eventuali questioni tecniche di minore importanza.

Il flusso di lavoro della progettazione – Manufatti della progettazione: meta-modello

- La figura seguente illustra il meta-modello del modello di progettazione:

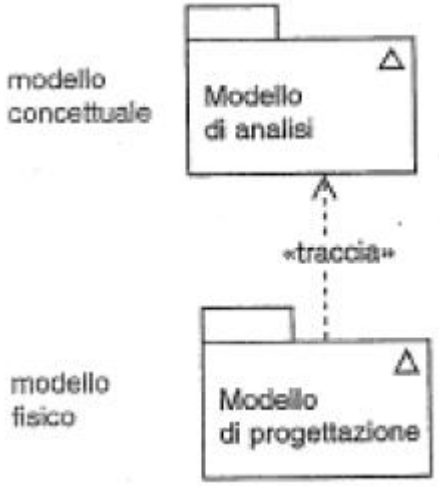


- Il modello di progettazione contiene molti sottosistemi di progettazione; questi sono componenti che possono, a loro volta, contenere molti tipi diversi di elementi di modellazione.

- È proprio durante le attività di progettazione che ci si concentra sulle interfacce.

Il flusso di lavoro della progettazione – Manufatti della progettazione: meta-modello

- Nella figura precedente è possibile vedere che il sottosistema C1 richiede l'interfaccia I che viene fornita al sottosistema C2.
- Le interfacce richieste e fornite collegano questi due sottosistemi come una spina e una presa elettriche.
- Esiste una semplice relazione di <<traccia>> tra il modello di analisi e il modello di progettazione:



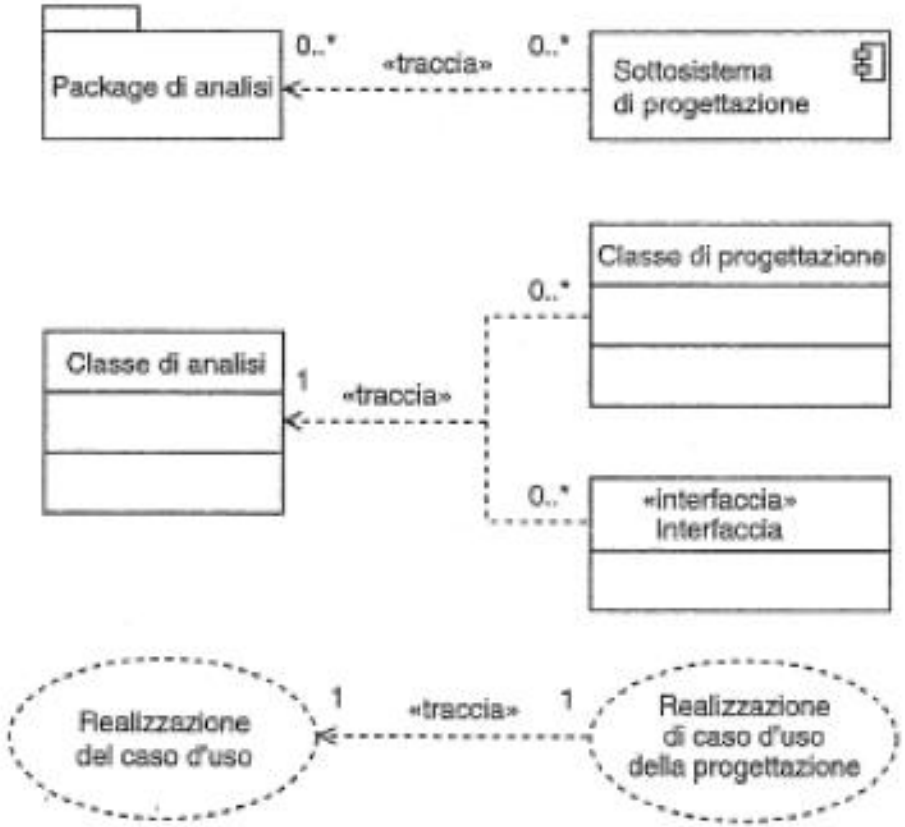
- Il modello di progettazione può essere considerato un'evoluzione del modello di analisi.
- Il modello di progettazione contiene gli stessi tipi di elementi del modello di analisi, ma tutti i manufatti sono più dettagliati e includono aspetti relativi all'implementazione.

Il flusso di lavoro della progettazione – Manufatti della progettazione: meta-modello

- Il modello di progettazione è composto da:
  - Sottosistema di progettazione.
  - Classi di progettazione.
  - Interfacce.
  - Progettazione delle realizzazioni dei casi d'uso.
  - Un diagramma di deployment.
- Uno dei manufatti più importanti prodotti durante la progettazione sono le interfacce.
- Queste consentono di eliminare le interdipendenze esistenti nel sistema.
- La progettazione produce anche una prima bozza del diagramma di deployment.

Il flusso di lavoro della progettazione – Manufatti della progettazione: meta-modello – Relazione di <<traccia>> tra i manufatti

- La seguente figura illustra le relazioni tra i principali manufatti di analisi e progettazione:



- La relazione tra i package di analisi e i sottosistemi di progettazione è leggermente più complessa.

Il flusso di lavoro della progettazione – Manufatti della progettazione: meta-modello – Relazione di <<traccia>> tra i manufatti

- Spesso un package di analisi è in relazione <<traccia>> con un sottosistema di progettazione.
- Possono esistere valide motivazioni tecniche o architettoniche per scomporre un unico package di analisi in diversi sottosistemi di progettazione.
- Una classe di analisi può trasformarsi in una o più interfacce o classi di progettazione.
- La realizzazione dei casi d'uso di analisi ha, invece, una relazione <<traccia>> biunivoca con la realizzazione dei casi d'uso di progettazione.

Il flusso di lavoro della progettazione – Manufatti della progettazione: meta-modello – E' necessario mantenere due modelli?

- In un mondo ideale esisterebbe un unico modello del sistema e lo strumento di modellazione utilizzato sarebbe in grado di generare da quello stesso modello sia la vista di analisi che la vista di progettazione.
- In realtà questo requisito è molto più complesso di quanto non possa sembrare.
- Restano, dunque, le seguenti quattro possibili strategie:

Strategia	*Conseguenze*
1. Trasformare il modello di analisi raffinandolo nel modello di progettazione	È sufficiente mantenere aggiornato il solo modello di progettazione, ma il modello di analisi è perso
2. Trasformare il modello di analisi raffinandolo nel modello di progettazione, quindi utilizzare uno strumento di modellazione per recuperare una "vista di analisi"	È sufficiente mantenere aggiornato il solo modello di progettazione, ma la "vista di analisi" recuperabile dallo strumento di modellazione potrebbe non essere soddisfacente
3. Congelare il modello di analisi a un certo punto della fase di Elaborazione; effettuare una copia del modello di analisi e trasformarla, raffinandola, nel modello di progettazione	Esistono entrambi i modelli ma non sono sincronizzati
4. Mantenere due modelli distinti: il modello di analisi e il modello di progettazione	Esistono entrambi i modelli; sono sincronizzati, ma è necessario tenerli allineati manualmente

Il flusso di lavoro della progettazione – Manufatti della progettazione: meta-modello – E' necessario mantenere due modelli?

- Non esiste una strategia migliore delle altre.
- La domanda fondamentale da porsi è: "Abbiamo bisogno di mantenere la vista di analisi del sistema?".
- La vista di analisi può contenere anche solo tra l'1% e il 10% delle classi presenti nella più dettagliata vista di progettazione ed è, quindi, tipicamente molto più comprensibile.
- Rimane, dunque, molto utile per:
  - Introdurre nuove risorse sul progetto.
  - Comprendere il funzionamento del sistema anche a distanza di mesi o di anni dal rilascio.
  - Comprendere come le funzionalità del sistema soddisfino e corrispondano ai requisiti dell'utente.
  - Pianificare interventi di manutenzione e di miglioramento.
  - Comprendere l'architettura logica del sistema.
  - Appaltare la costruzione del sistema a risorse esterne.

Il flusso di lavoro della progettazione – Manufatti della progettazione: meta-modello – E' necessario mantenere due modelli?

- Se si prevede di dover effettuare una delle suddette attività allora è fondamentale preservare la vista di analisi.
- È consigliato mantenere comunque la vista di analisi per tutti i sistemi di grandi dimensioni.
- In questi casi è necessario scegliere la strategia 3 o la strategia 4.
- La decisione se lasciare o meno che il modello di analisi e il modello di progettazione si disallineino deve essere ben ponderata.
- Se il sistema è piccolo (meno di 200 classi di progettazione) allora il modello di progettazione è di per sé sufficientemente ridotto da essere comprensibile.
- Il mantenimento dei due modelli separati potrebbe essere eccessivo anche nel caso in cui il sistema non sia strategico o si preveda che abbia una vita breve.
- In queste situazioni bisogna scegliere la strategia 1 o la strategia 2.

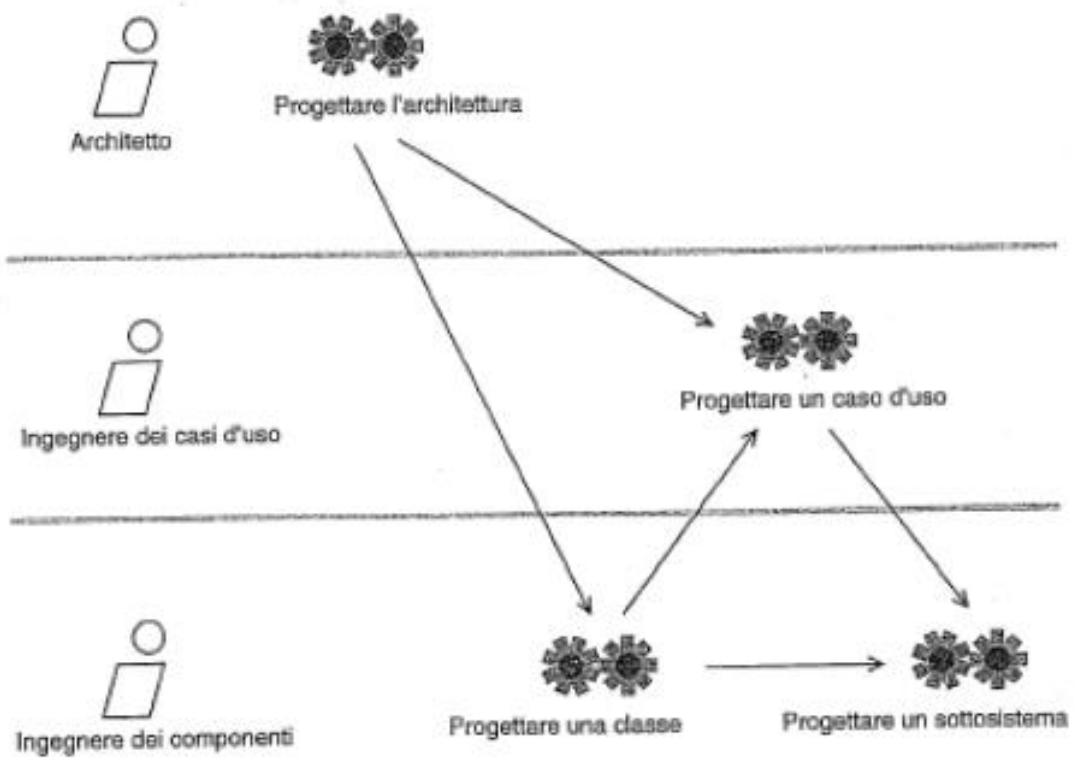
Il flusso di lavoro della progettazione – Manufatti della progettazione: meta-modello – E’ necessario mantenere due modelli?

- Alcuni strumenti di modellazione gestiscono il solo modello di progettazione ma consentono di filtrare e nascondere alcune informazioni nel tentativo di ricostruire una vista di “analisi” del modello di progettazione.

- Sarebbe saggio ricordarsi che moltissimi sistemi restano in vita molto più a lungo di quanto inizialmente previsto.

Il flusso di lavoro della progettazione – Dettagli

- La seguente figura illustra il flusso della progettazione previsto da UP:



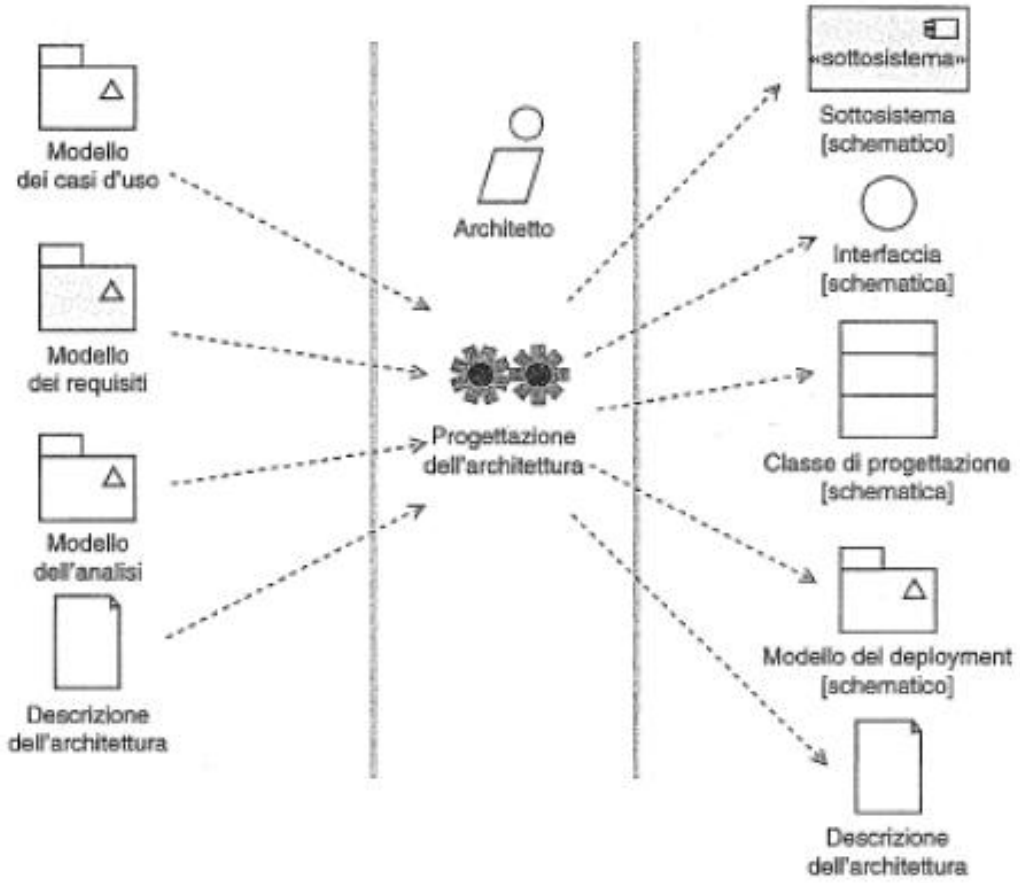
- Le principali risorse coinvolte nella progettazione sono l'architetto, l'ingegnere dei casi d'uso e l'ingegnere dei componenti.

Il flusso di lavoro della progettazione – Dettagli

- Nella maggior parte dei progetti OO il ruolo di architetto è coperto da uno o più individui dedicati.
- Un'unica persona, in tempi diversi, può ricoprire entrambi i ruoli di ingegnere dei casi d'uso e ingegnere dei componenti.
- Uno degli obiettivi di UP è che ciascun partecipante al progetto diventi esperto e responsabile di una parte del sistema su tutta la linea.

Il flusso di lavoro della progettazione – Attività di UP: progettazione dell'architettura

- In UP l'attività che dà inizio all'intero processo di progettazione è *Progettazione dell'architettura*.
- Questa attività, i cui dettagli sono illustrati nella seguente figura, viene svolta da uno o più architetti.



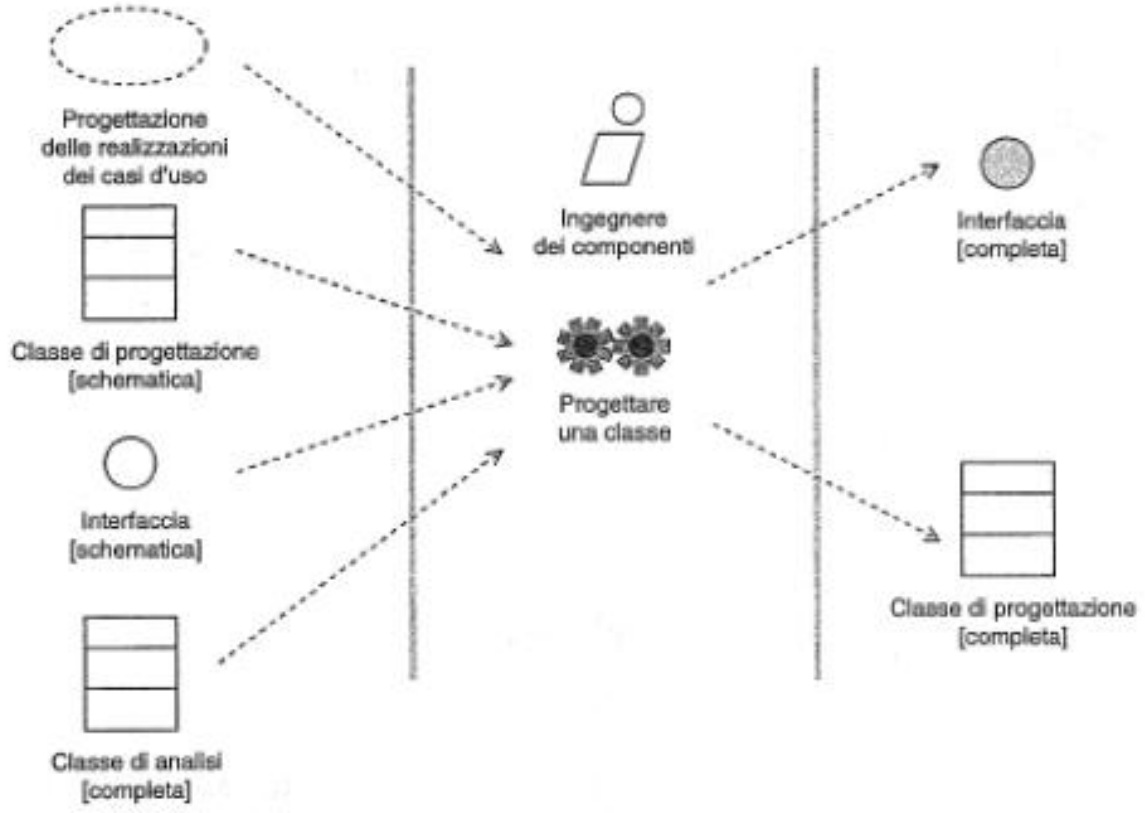
Il flusso di lavoro della progettazione – Attività di UP: progettazione dell'architettura

- Ci sono molti manufatti prodotti da Progettazione dell'architettura.
- La progettazione dell'architettura riguarda la schematicità di manufatti architetturalmente significativi per fornire il quadro di insieme dell'architettura del sistema.
- Questi manufatti schematici forniscono gli input ad attività di progettazione più dettagliata durante le quali vengono riempiti.
- Normalmente la Progettazione dell'architettura non è una fase separata.
- Questa progettazione avviene durante la parte iniziale della fase di Costruzione e alla fine della fase di Elaborazione.

Il flusso di lavoro della progettazione – Attività di UP: progettazione una classe

- Nel flusso di dettaglio della progettazione previsto da UP, dopo *Progettazione dell'architettura*, le attività successive sono *Progettare una classe* e *Progettare un caso d'uso*.

- L'attività UP *Progettare una classe* viene riportata nella seguente figura:



Il flusso di lavoro della progettazione – Attività di UP: progettazione una classe

- Si è già visto come creare il manufatto d'input *classe di analisi* [*completa*] quando abbiamo parlato dell'analisi.
- Dal diagramma dell'attività sembra che vi siano due manufatti separati e distinti: *classe di progettazione* [*schematica*] e *classe di progettazione* [*completa*].
- I due manufatti rappresentano in realtà il medesimo manufatto (una classe di progettazione) in fasi diverse della sua evoluzione.
- In ogni istante ci saranno delle classi di progettazione e ciascuna di queste sarà in una fase diversa di sviluppo.
- Una classe di progettazione è "completa" dal punto di vista di UP se è sufficientemente dettagliata da servire come base per creare il codice sorgente.
- Il livello di dettaglio necessario dipende dal progetto.
- Le considerazioni fatte per *classe di progettazione* [*schematica*] o *classe di progettazione* [*completa*] valgono anche per *interfaccia* [*schematica*] e *interfaccia* [*completa*].
- Il manufatto di input progettazione della realizzazione dei casi d'uso è solo una realizzazione dei casi d'uso alla fine del suo ciclo di vita.

Classi di progettazione – Cosa sono?

- Le classi di progettazione sono classi le cui specifiche sono talmente complete che possono essere effettivamente implementate.

- Nell'analisi le classi nascono dal dominio del problema, costituito dall'insieme dei requisiti che descrivono il problema che si vuole risolvere.

- Le classi di progettazione hanno due origini:

  - Il dominio del problema, attraverso un processo di raffinamento delle classi di analisi.

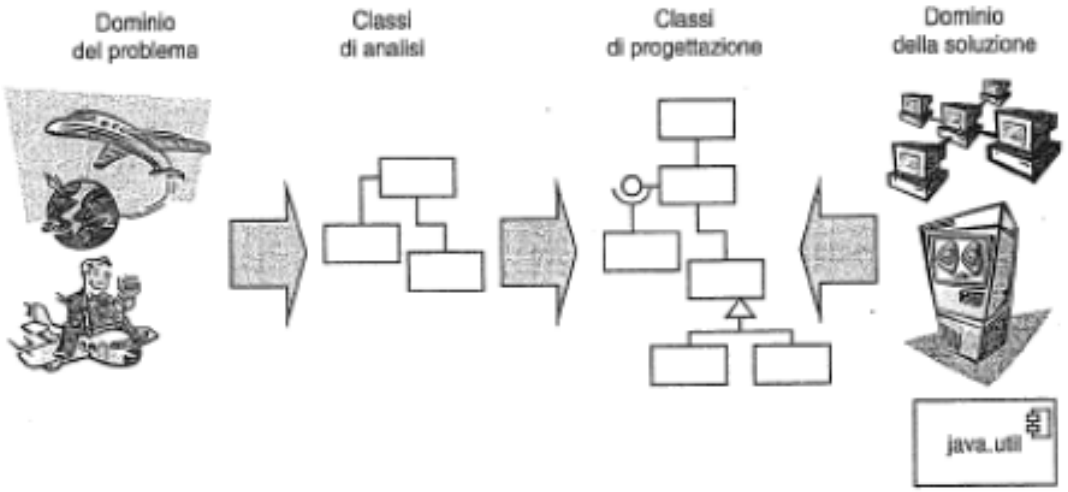
    - Durante questa attività capita spesso di dover scomporre una classe di analisi di alto livello in due o più classi di progettazione distinte.

    - Esiste una relazione `<<traccia>>` tra una classe di analisi e la classe, o le classi, di progettazione che ne descrivono l'implementazione.

  - Il dominio della soluzione; questo dominio fornisce tutti gli strumenti tecnologici che consentono di implementare il sistema.

Classi di progettazione – Cosa sono?

- Il passaggio dalle classi di analisi alle classi di progettazione viene schematizzato nella seguente figura:



- L'analisi serve a modellare cosa il sistema dovrebbe fare; la progettazione serve a modellare come tale comportamento debba essere implementato.

- La progettazione richiede che le classi definite siano piccole unità autosufficienti e coese in grado di svolgere molto bene uno o due compiti.

- Il metodo di implementazione scelto determina il grado di completezza necessario per specificare la classe di progettazione.

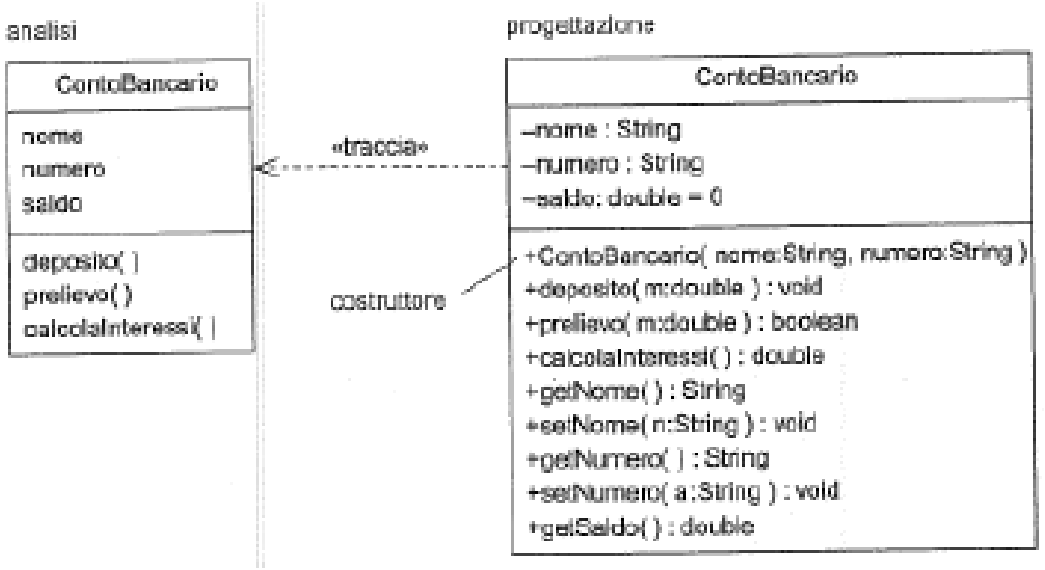
Classi di progettazione – Cosa sono?

- Se il modello della classe di progettazione viene dato ai programmatori le classi devono essere complete al punto da consentire loro di eseguire questo compito.

- Se si ha intenzione di generare il codice da classi di progettazione con uno strumento di modellazione adatto le specifiche delle classi devono essere complete.

Classi di progettazione – Anatomia di una classe di progettazione

- Con le classi di progettazione si deve specificare esattamente come queste classi assolveranno le loro responsabilità.
- Per raggiungere questo obiettivo è necessario:
  - Completare l'insieme degli attributi della classe, includendo tutte le specifiche.
  - Completare l'insieme delle operazioni, includendo il nome, l'elenco dei parametri e il tipo di valore restituito.
- Il processo di raffinamento è illustrato con un esempio molto semplice nella seguente figura:



Classi di progettazione – Anatomia di una classe di progettazione

- Una singola operazione di analisi ad alto livello può essere trasformata in una o più operazioni di progettazione implementabili; queste operazioni sono dette metodi.

- Esempio: l'implementazione di un metodo `checkIn`() nel passaggio dalle classi di analisi alle classi di progettazione.

Classi di progettazione – Classi di progettazione ben formate

- Durante la fase di progettazione diventa necessario stabilire se le classi siano ben formate o meno.
- Quando si progetta una classe è sempre importante considerarla dal punto di vista dei suoi potenziali clienti.
- Le caratteristiche minime che una classe di progettazione deve avere sono le seguenti:
  - Completezza e sufficienza.
  - Essenzialità.
  - Massima coesione.
  - Minima interdipendenza.

Classi di progettazione – Classi di progettazione ben formate – Completezza e sufficienza

- I metodi pubblici di una classe definiscono un contratto tra questa e i suoi clienti; è importante che questo contratto sia chiaro, ben definito e accettabile.
- Una classe è completa se fornisce ai suoi clienti tutto ciò che essi si aspettano da essa.
- I clienti inferiscono dal nome della classe quali debbano essere i metodi che essa mette loro a disposizione.
- La completezza è la capacità che una classe ha di soddisfare le ragionevoli richieste dei suoi clienti.
- Una classe viene detta, invece, sufficiente se tutti i metodi sono esclusivamente finalizzati alla realizzazione dell'unico scopo che la classe deve avere.
- Una classe non deve mai sorprendere i propri clienti.
- Un tipico errore commesso da progettisti poco esperti è quello di prendere una classe semplice e sufficiente, quale per esempio `ContoBancario`, e aggiungervi operazioni per processare carte di credito o polizze assicurative.
- La sufficienza di una classe misura quanto essa sia semplice e focalizzata sulle sue poche responsabilità.
- La regola d'oro della completezza e della sufficienza è che una classe deve fare esattamente ciò che i suoi utenti si aspettano: nulla di più e nulla di meno.

Classi di progettazione – Classi di progettazione ben formate – Essenzialità

- Le operazioni devono essere progettate per offrire un unico servizio primitivo e atomico.
- Una classe non deve mettere a disposizione più di un modo per effettuare la stessa operazione.
- Se per esempio la classe `ContoBancario` ha un'operazione primitiva per effettuare un deposito non dovrebbe avere anche un'operazione più complessa che consente di effettuare contestualmente una serie di depositi.
- Il punto è che le classi progettate devono sempre rendere disponibile l'insieme di operazioni più essenziale e semplice possibile.
- Anche se l'essenzialità è solitamente desiderabile esistono situazioni reali in cui può costituire un vincolo troppo rigido.
- Una ragione per cui a volte si sacrifica parzialmente l'essenzialità è l'esigenza di aumentare le prestazioni del codice.
- Tuttavia, bisogna sempre partire da una situazione di massima essenzialità, ovvero con l'insieme di operazioni più primitivo possibile.
- Molte delle cosiddette "ottimizzazioni" di progettazione non si basano su fatti concreti, quindi hanno un impatto scarso, se non nullo, sulle effettive prestazioni dell'applicazione a tempo di esecuzione.

Classi di progettazione – Classi di progettazione ben formate – Essenzialità

- L'ottimizzazione delle prestazioni può essere eseguita solo usando uno strumento di profilazione del codice, per esempio JMechanic di Java.

- Questo chiaramente è un compito dell'implementazione che può avere un impatto sul modello di progettazione.

Classi di progettazione – Classi di progettazione ben formate – Massima coesione

- Ogni classe dovrebbe modellare un unico concetto astratto e dovrebbe avere un insieme di metodi a supporto di questo suo unico compito.
- Se una classe ha molte responsabilità diverse, allora alcune di queste devono essere implementate da classi ausiliarie.
- Una classe coesa ha un insieme ridotto di responsabilità molto correlate tra loro.
- Un esempio di classi non coese viene riportato nella seguente figura:



Classi di progettazione – Classi di progettazione ben formate – Massima coesione

- Questo modello contiene diversi errori:
  - I nomi delle classi sono discutibili.
  - Il prefisso "`Bean`" non è necessario in quanto fa riferimento a un particolare dettaglio relativo all'implementazione.
  - La classe `BeanAlbergoAuto` è poco coesa: ha due responsabilità principali le quali, tra l'altro, risiedono in altre classi.
  - Questo non è un modello di analisi (in quanto contiene informazioni di progettazione nei suffissi "`Bean`") né un modello di progettazione (in quanto non è sufficientemente completo).
  - Dal punto di vista della coesione, `BeanAlbergo` e `BeanAuto` sono accettabili, ma `BeanAlbergoAuto` è semplicemente assurda.

Classi di progettazione – Classi di progettazione ben formate – Minima interdipendenza

- Una data classe dovrebbe essere associata all'insieme minimo di altre classi che consente ad essa di realizzare le proprie responsabilità.
- Due classi dovrebbero essere associate soltanto se tra loro esiste una correlazione semantica: questa viene detta minima interdipendenza.
- Uno degli errori comuni dei progettisti OO alle prime armi è quello di connettere ogni elemento del modello ad una moltitudine di altri elementi, a seconda delle esigenze che vengono progressivamente individuate.
- Un modello a oggetti molto interconnesso è l'equivalente del "codice spaghetti" del mondo non OO.
- I sistemi OO con un alto grado di interdipendenza sono spesso il risultato di progetti in cui non viene effettuata alcuna attività formale di progettazione.
- I progettisti con poca esperienza devono stare attenti a non mettere in relazione due classi solo perché una delle due contiene un frammento di codice che potrebbe essere utile anche all'altra.
- Molte associazioni tra classi vengono introdotte per rispettare i vincoli di implementazione o per riusare il codice.
- Ovviamente, all'interno di un sottosistema l'interdipendenza può anche essere molto alta; è l'interdipendenza tra sottosistemi diversi che compromette l'architettura del sistema.

Classi di progettazione – Ereditarietà

- Durante le attività di progettazione l'ereditarietà tra classi assume un maggior rilievo.
- Nel modello di analisi si utilizza l'ereditarietà soltanto se tra due classi di analisi esiste un'ovvia e non ambigua relazione "è un/una".
- In progettazione, invece, si può anche decidere di utilizzare l'ereditarietà in modo tattico per riusare il codice.
- L'ereditarietà in questo caso serve per facilitare l'implementazione di una classe discendente e non per rappresentare una relazione di business tra le classi.

Classi di progettazione – Ereditarietà – Aggregazione o ereditarietà?

- L'ereditarietà è una tecnica molto potente; tuttavia, i progettisti OO poco esperti spesso ne abusano.
- È necessario capire che l'ereditarietà ha anche alcune caratteristiche non desiderabili:
  - È in assoluto la forma di interdipendenza più forte tra due o più classi.
  - All'interno di una gerarchia di classi l'incapsulamento risulta debole; modifiche alla classe base si propagano alle sottoclassi.
  - È un tipo di relazione poco flessibile; in tutti i linguaggi OO comunemente utilizzati la relazione di ereditarietà non è modificabile a tempo di esecuzione.
- Un esempio di modellazione errata dell'ereditarietà viene riportata nella seguente figura:

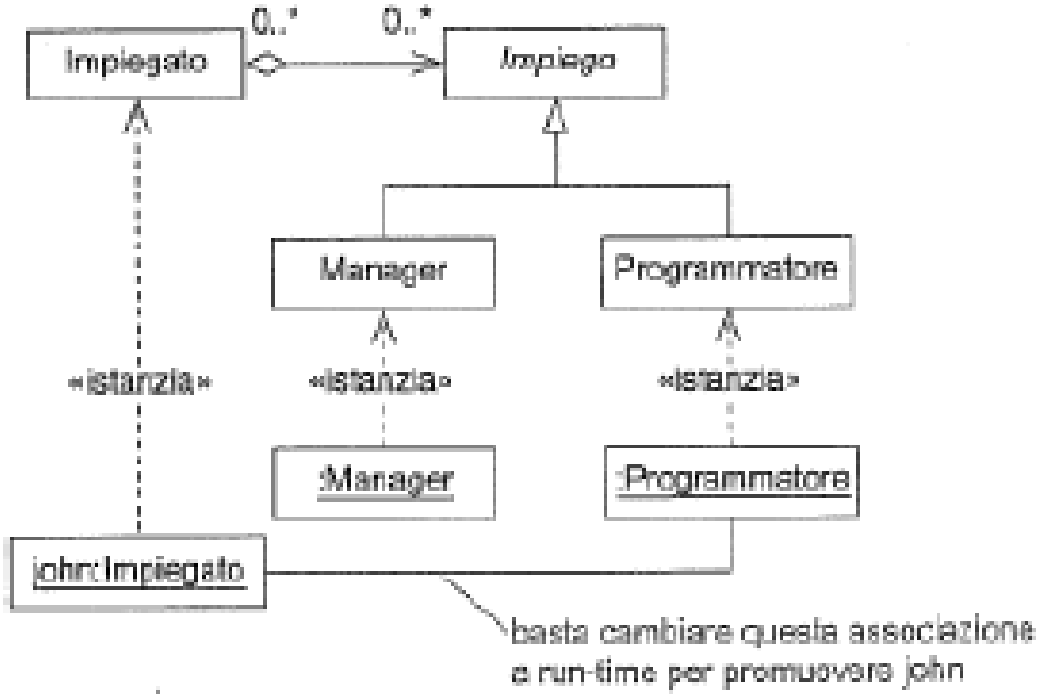


Classi di progettazione – Ereditarietà – Aggregazione o ereditarietà?

- Il modello precedente nasconde un errore semantico fondamentale: è più corretto dire che `Impiegato` è un `Impiego` o che `Impiegato` ha un `Impiego`?

- Utilizzando l'aggregazione si riesce a correggere l'errore semantico: un `Impiegato` ha un `Impiego`.

- La modellazione corretta dello scenario mostrato nella figura precedente viene mostrata nella seguente figura:



Classi di progettazione – Ereditarietà – Aggregazione o ereditarietà?

- Questo esempio illustra un principio generale molto importante: le sottoclassi dovrebbero sempre rappresentare una relazione “è un tipo di” e non una relazione “è un ruolo interpretato da”.

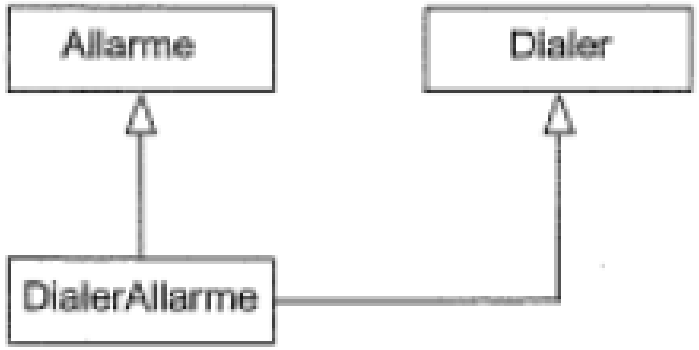
Classi di progettazione – Ereditarietà – Ereditarietà multipla

- A volte c'è l'esigenza che una classe erediti da più di una classe genitore.
- Questo meccanismo di ereditarietà multipla non è supportata da tutti i linguaggi di programmazione OO.
- Questa mancanza di supporto per l'ereditarietà multipla non costituisce un vero problema, dato che si può ottenere lo stesso effetto con l'uso combinato di ereditarietà singola e delegazione.
- L'ereditarietà multipla deve essere usata solo se è supportata dal linguaggio che verrà utilizzato per l'implementazione.
- È necessario tenere sempre presenti le seguenti considerazioni riguardanti l'ereditarietà multipla:
  - Le classi genitore devono risultare tutte semanticamente disgiunte tra loro.
  - Il principio di sostituibilità e la relazione "è un tipo di" devono essere applicabili tra la sottoclasse e ciascuna delle superclassi interessate.
  - Le superclassi non dovrebbero avere antenati in comune.

Classi di progettazione – Ereditarietà – Ereditarietà multipla

- Un caso in cui l'ereditarietà multipla si rileva semplice ed efficace è quello delle classi "mixin"; si tratta di classi che non sono autosufficienti ma sono state progettate per essere incorporate in altre classi utilizzando l'ereditarietà.

- Un esempio di classe mixin viene riportato nella seguente figura:



Classi di progettazione – Ereditarietà – Ereditarietà o realizzazione di interfaccia?

- Con l'ereditarietà si definiscono:
  - L'interfaccia, ovvero le operazioni pubbliche della classe base.
  - L'implementazione, ovvero gli attributi, le relazioni e i metodi privati della classe base.
- Con la realizzazione di interfaccia, che vedremo in seguito, si definisce:
  - L'interfaccia, ovvero un insieme di operazioni pubbliche, attributi e relazioni non implementati.
- L'ereditarietà e la realizzazione di interfaccia hanno qualcosa in comune, in quanto entrambe consentono di definire un contratto.
- Tuttavia queste due tecniche hanno una semantica e un utilizzo molto diversi tra loro.
- L'ereditarietà è necessaria solo se lo scopo è quello di ereditare anche alcuni dettagli implementativi dalla superclasse.

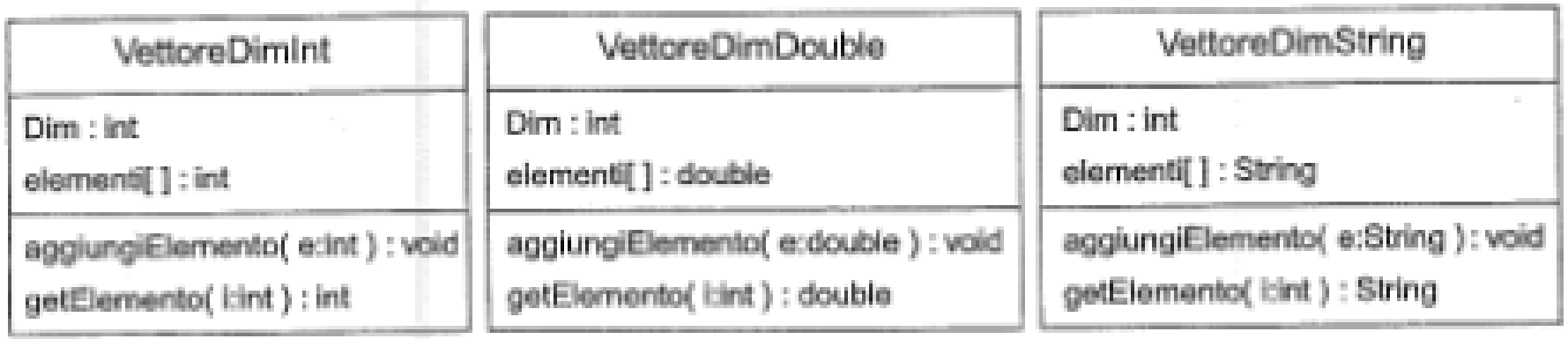
Classi di progettazione – Ereditarietà – Ereditarietà o realizzazione di interfaccia?

- La realizzazione di interfaccia è utile quando si vuole definire un contratto senza, però, essere disposti ad accettare i dettagli di implementazione ereditati.

- Se da un lato è vero che la realizzazione di interfaccia non offre alcuna possibilità di riuso del codice, dall'altro, però, offre un meccanismo molto pulito per definire un contratto e garantire che certe classi lo rispettino.

Classi di progettazione – Template di classe

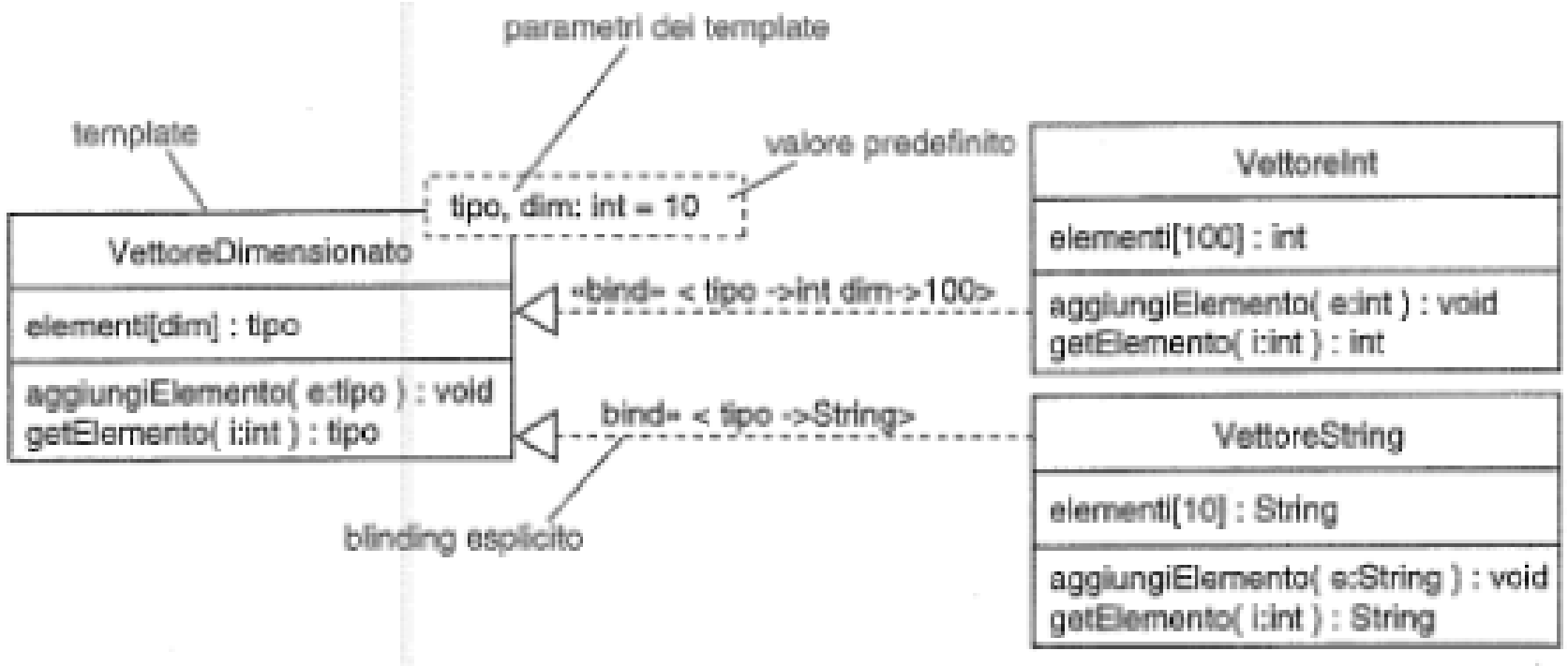
- Finora abbiamo detto che, data una classe, è necessario specificare esplicitamente i tipi dei suoi attributi, dei parametri e del valore restituito da tutte le sue operazioni.
- Questo può a volte limitare la possibilità di riutilizzare il codice.
- La seguente figura riporta un esempio in cui sono definite tre classi vettore:



- Se non fosse per il tipo di elemento contenuto nel vettore, queste tre classi sarebbero del tutto identiche.
- I template di classe consentono la parametrizzazione dei tipi; al posto di specificare il tipo effettivo degli attributi e dei parametri e valori restituiti dalle operazioni è possibile definire una classe facendo riferimento a dei segnaposto o dei parametri.

Classi di progettazione – Template di classe

- La seguente figura illustra la definizione della classe `VettoreDimensionato` facendo riferimento ai parametri `tipo` (che è un classificatore, dal momento che non è specificato il tipo) e `dim`, che è un `int`.



- Si noti che il template specifica un valore predefinito di 10 per `dim`.

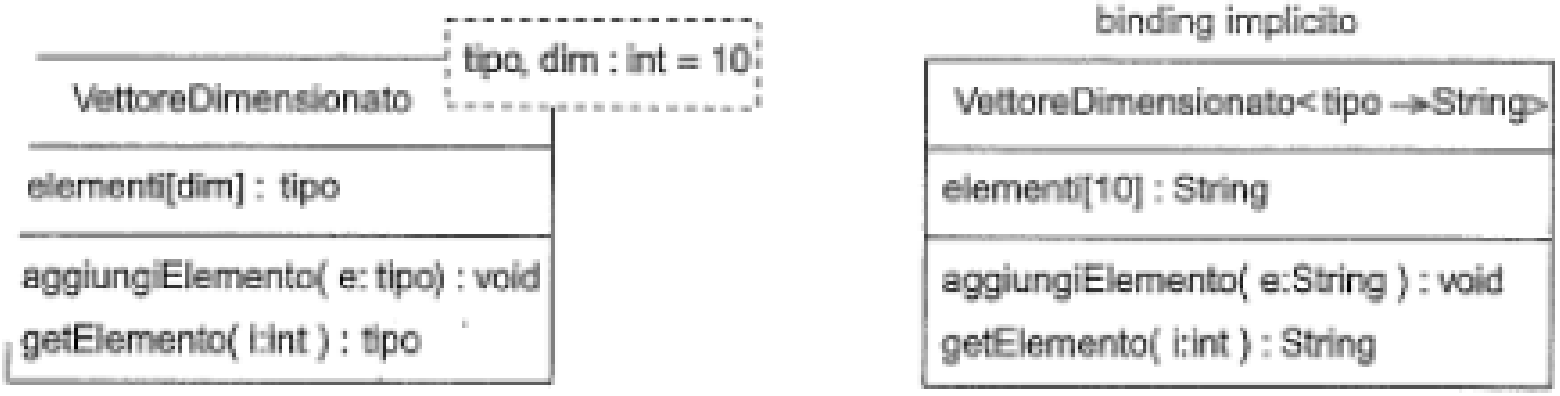
- Associando a questi parametri formali dei valori specifici (operazione di “binding”) è possibile creare nuove classi; questa procedura viene detta istanziazione di un template.

Classi di progettazione – Template di classe

- È possibile istanziare un template utilizzando una relazione di dipendenza con lo stereotipo <<`bind`>>; in questo caso si parla di binding esplicito.
- Per istanziare un template è anche necessario specificare i valori effettivi che devono essere sostituiti ai parametri del template.
- L'espressione `tipo -> int` significa che il parametro `tipo` dei template viene sostituito da `int` all'istanziazione.
- I template sono ovviamente un meccanismo di riuso molto potente: è possibile definire una classe molto generica creando poi molte versioni specializzate.
- Nella figura precedente il binding viene in realtà utilizzato in due diversi modi: il parametro `tipo` richiede come valore un classificatore; il parametro `dim` richiede, invece, un valore effettivo intero.
- I nomi dei parametri di un template sono locali al template stesso.
- Esiste una sintassi alternativa per specificare l'istanziazione di un template, ovvero il binding implicito.

Classi di progettazione – Template di classe

- Per istanziare un template implicitamente è sufficiente utilizzare, al posto del nome della classe, il nome del template di classe, seguito dall'elenco dei valori effettivi dei parametri racchiusi tra parentesi angolari, come illustrato nella seguente figura:



- Lo svantaggio del binding implicito è che la classe istanziata non può veramente avere un suo nome.

- Per tale ragione è consigliabile l'utilizzo del binding esplicito.

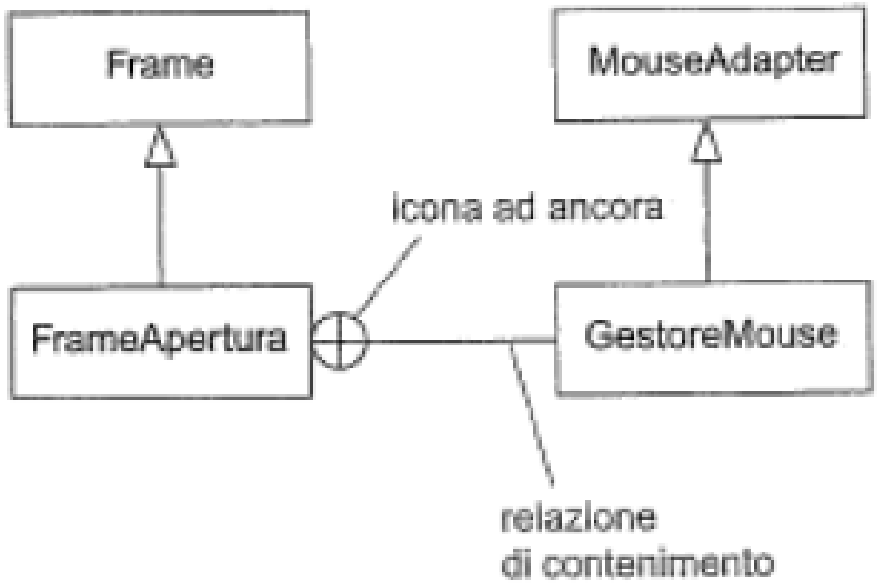
- Anche se i template di classe sono un meccanismo molto potente, attualmente gli unici linguaggi OO comunemente usati che li supportano sono C++ e Java.

Classi di progettazione – Classi annidate

- Alcuni linguaggi consentono di definire una classe all'interno della definizione di un'altra classe.
- Questa prima classe viene chiamata classe annidata; in Java è anche nota come classe inner.
- Una classe annidata è dichiarata dentro alla dichiarazione di una seconda classe, detta classe esterna, ed è accessibile solo alla classe esterna e agli oggetti di quella classe.
- Le classi annidate tendono ad essere una questione di progettazione.
- Le classi annidate sono usate principalmente in Java per la gestione degli eventi.

Classi di progettazione – Classi annidate

- La seguente figura illustra una semplice classe di finestra interfaccia utente chiamata `FrameApertura`:



- Questo approccio apporta notevoli benefici.

- Il codice di gestione del mouse ottiene la sua classe `GestoreMouse` separandola dal resto del codice `FrameApertura` e `GestoreMouse` viene completamente incapsulato all'interno di `FrameApertura`.

Raffinare le relazioni di analisi – Relazioni di progettazione

- Quando si giunge alla progettazione è necessario raffinare le relazioni tra le classi di analisi e trasformarle in relazioni tra classi di progettazione.
- Molte classi individuate e fissate durante l'analisi non sono direttamente implementabili così come sono state definite.
- Per ottenere le associazioni di progettazione a partire dalle associazioni di analisi è necessario operare un raffinamento che prevede diverse procedure:
  - Trasformare le associazioni in relazioni di aggregazione o composizione, dove applicabile.
  - Implementare le classi associazione.
  - Implementare le associazioni uno-a-uno.
  - Implementare le associazioni molti-a-uno.
  - Implementare le associazioni molti-a-molti.
  - Implementare le associazioni bidirezionali.

Raffinare le relazioni di analisi – Relazioni di progettazione

- Tutte le associazioni di progettazione devono specificare la navigabilità e la molteplicità di entrambi gli estremi.

- Tutte le associazioni di progettazione dovrebbero, inoltre, specificare il nome dell'associazione o il nome di un ruolo, almeno sull'estremo destinazione.

Raffinare le relazioni di analisi – Aggregazione e composizione

- Nella progettazione una relazione di associazione può essere raffinata in una relazione di aggregazione o in una forma di aggregazione nota come relazione di aggregazione compositiva.
  - L'aggregazione è una relazione tra oggetti poco forte; un calcolatore e le sue periferiche sono un esempio di aggregazione.
  - La composizione è una relazione tra oggetti molto forte; un albero e le sue foglie sono un esempio di composizione.

Raffinare le relazioni di analisi – Semantica dell'aggregazione

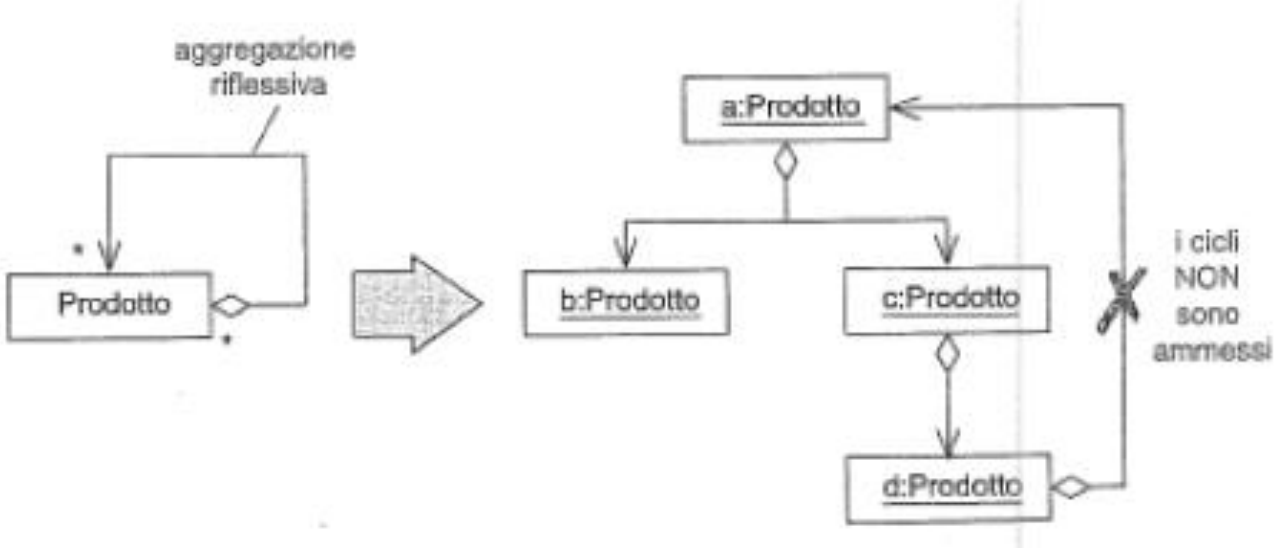
- L'aggregazione è una relazione del tipo tutto-parte per cui un aggregato è costituito da molte parti.
- In una relazione tutto-parte, un oggetto (il Tutto) utilizza i servizi di un altro oggetto (la Parte).
- Un esempio di aggregazione viene riportato nella seguente figura:



- La semantica dell'aggregazione può essere così sintetizzata:
  - L'aggregato può in alcuni casi esistere indipendentemente dalle parti, ma in altri casi no.
  - Le parti possono esistere indipendentemente dall'aggregato.
  - L'aggregato è in qualche modo incompleto se mancano alcune delle sue parti.
  - È possibile che più aggregati condividano una stessa parte.

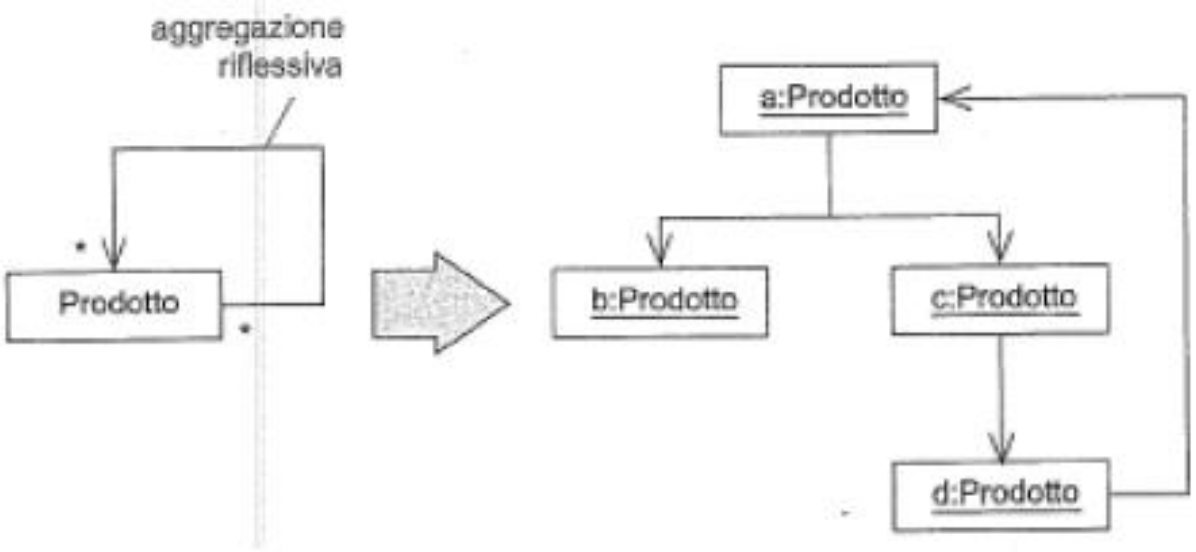
Raffinare le relazioni di analisi – Semantica dell'aggregazione

- L'aggregazione è transitiva.
- L'aggregazione è asimmetrica.
- Questo limita il possibile utilizzo dell'aggregazione nei modelli.
- Esempi di modellazioni ammesse e vietate dalla proprietà di asimmetria della relazione sono riportati nella seguente figura:



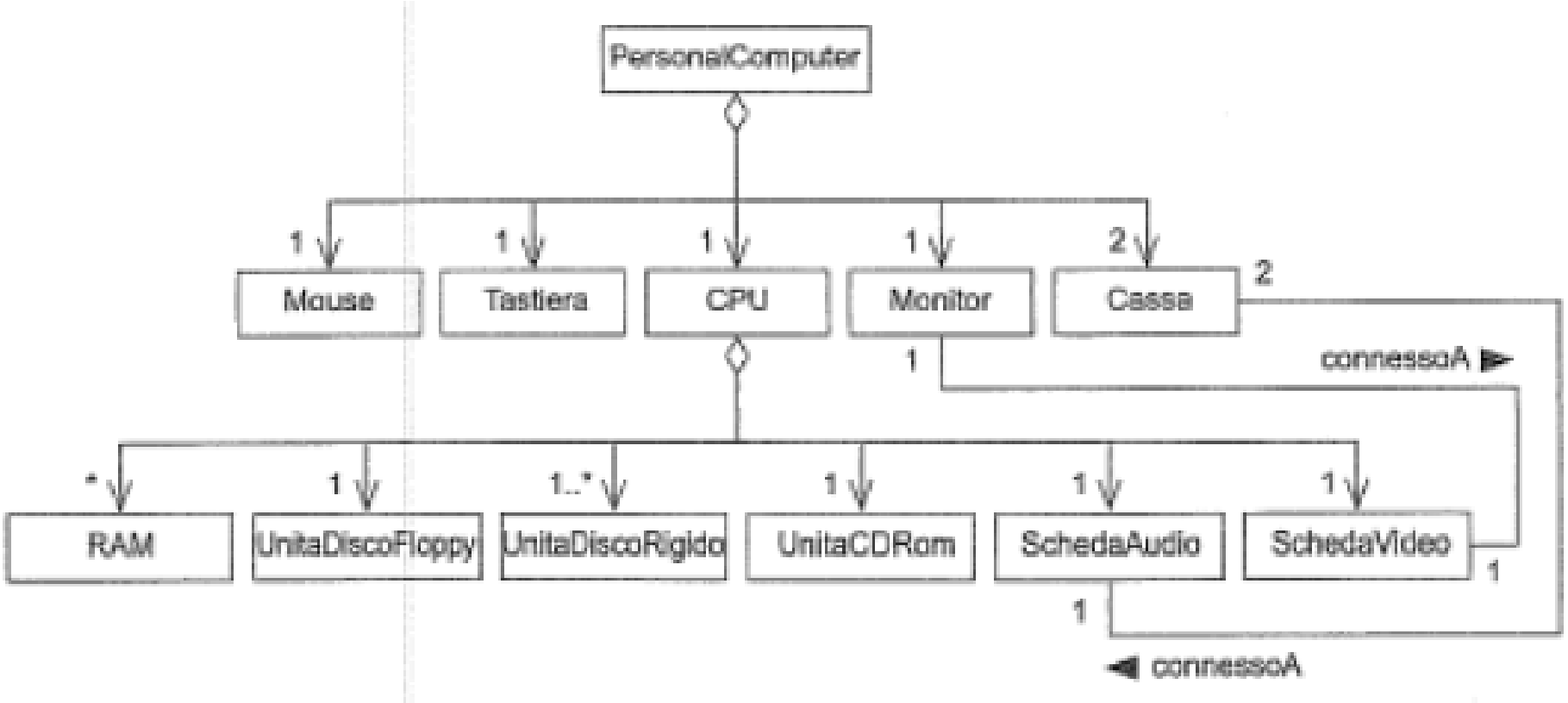
Raffinare le relazioni di analisi – Semantica dell'aggregazione

- I problemi della figura precedente possono essere superati come mostrato nella seguente figura:



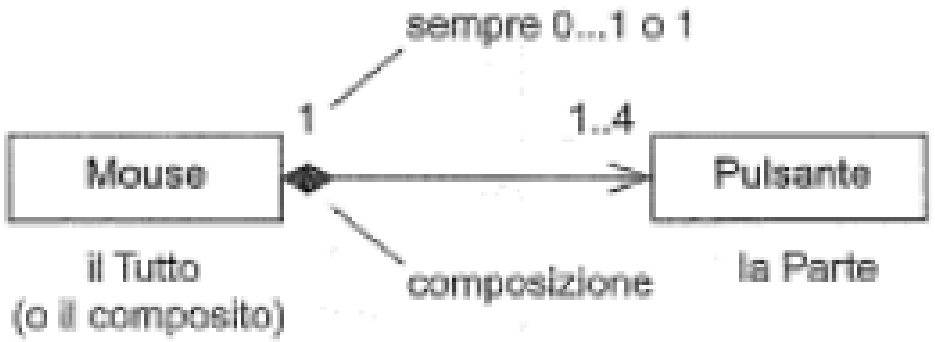
Raffinare le relazioni di analisi – Semantica dell'aggregazione

- La seguente figura mostra un altro tipico esempio di aggregazione:



Raffinare le relazioni di analisi – Semantica della composizione

- La composizione è una forma più forte di aggregazione e ha una semantica del tutto simile.
- Nella composizione la Parte non ha una propria esistenza indipendente al di fuori del Tutto.
- Nella composizione ogni Parte appartiene esclusivamente a un solo Tutto, mentre nell'aggregazione una Parte può essere condivisa da più Tutti.
- Un esempio di composizione viene riportato nella seguente figura:



Raffinare le relazioni di analisi – Semantica della composizione – Composizione e attributi

- Ragionando sulla semantica della composizione ci si rende conto di quanto sia simile alla semantica degli attributi.
- Perché abbiamo bisogno di due modi per esprimere lo stesso concetto?
- Le ragioni sono due:
  - Gli attributi possono essere dei tipi di dati primitivi; questi tipi primitivi dovrebbero sempre essere modellati come attributi.
  - Esistono alcune classi di utilità, quali `Time`, `Date` e `String`, che sono utilizzate in modo molto pervasivo; se ogni uso di queste classi venisse modellato con una relazione di composizione alla classe stessa, il modello risulterebbe illeggibile.
- Dovendo modellare un tipo primitivo o una classe di utilità o anche solo una classe che non risulta interessante ai fini del modello o del diagramma in questione, è meglio utilizzare un attributo e non una relazione di composizione.

Raffinare le relazioni di analisi – Come raffinare le relazioni di analisi

- In analisi si utilizzano spesso associazioni semplici, senza entrare nel merito della semantica della relazione.
- Durante la progettazione è, invece, necessario specificare completamente le relazioni raffinando ciascuna associazione, laddove possibile, in una forma di aggregazione.
- L'unico caso in cui è lecito utilizzare un'associazione in progettazione è quando sia necessario per evitare un ciclo nel grafo delle aggregazioni.
- La maggior parte delle associazioni individuate durante l'analisi vengono poi trasformate in aggregazioni o composizioni.
- Per affinare un'associazione di analisi in un'aggregazione o in un'associazione è necessario procedere nel seguente modo:
  - Qualora non siano presenti, aggiungere la molteplicità e i nomi dei ruoli dell'associazione.
  - Decidere quale estremo dell'associazione è il Tutto e quale è la Parte.

Raffinare le relazioni di analisi – Come raffinare le relazioni di analisi

  - Controllare la molteplicità all'estremo del Tutto: se è 0..1 o esattamente 1 allora si può utilizzare una composizione, altrimenti è necessario utilizzare un'aggregazione.

  - Aggiungere la navigabilità dal Tutto alla Parte: in progettazione le associazioni devono essere unidirezionali.

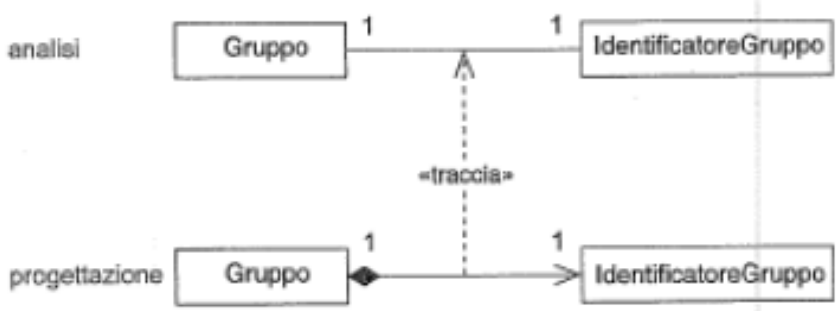
- A questo punto l'associazione è stata raffinata in un'aggregazione o in una composizione.

- Se la molteplicità a uno degli estremi è maggiore di 1, allora è necessario stabilire come verrà implementata.

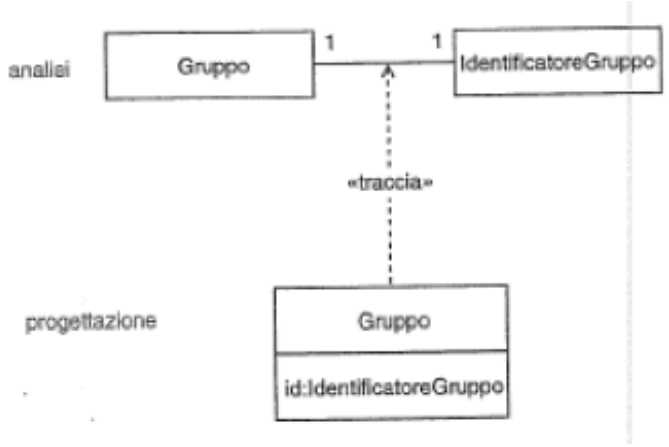
- Questo è il passo successivo dell'operazione ai raffinamento.

Raffinare le relazioni di analisi – Come raffinare le relazioni di analisi – Associazioni uno-a-uno

- Quasi tutte le associazioni uno-a-uno vengono raffinate in relazioni di composizione.
- Un possibile raffinamento per le associazioni uno-a-uno viene riportato nella seguente figura:

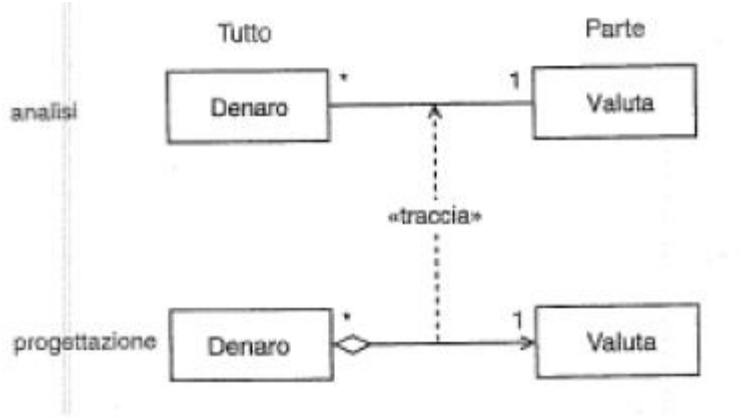


- Si può prendere in considerazione anche la possibilità di rendere `IdentificatoreGruppo` un attributo di `Gruppo` se `IdentificatoreGruppo` non è una classe particolarmente importante.



Raffinare le relazioni di analisi – Come raffinare le relazioni di analisi – Associazioni molti-a-uno

- Un'associazione molti-a-uno ha una molteplicità di molti all'estremo del Tutto e di esattamente 1 all'estremo della Parte.
- La composizione non è applicabile: la Parte è condivisa tra molti Tutti.
- Può essere applicabile l'aggregazione; bisogna solo controllare che non nascano cicli nel grafo delle aggregazioni.
- Tutto ciò viene mostrato nella seguente figura:



- Un singolo oggetto `Valuta` è condiviso tra molti oggetti `Denaro`; `Denaro` è una quantità di una singola `Valuta`.

Raffinare le relazioni di analisi – Come raffinare le relazioni di analisi – Associazioni uno-a-molti

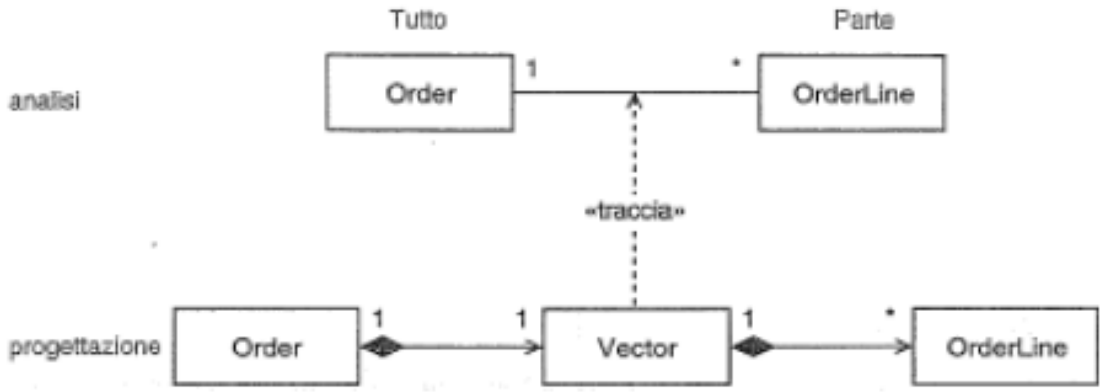
- In un'associazione uno-a-molti un Tutto ha una collezione omogenea di oggetti Parte.
- Per implementare questo tipo di relazione è necessario ricorrere ad una classe collezione.
- La maggior parte dei linguaggi OO offre poco supporto nativo per gestire collezioni di oggetti.
- Molti linguaggi mettono a disposizione dei programmatori soltanto gli array o vettori.
- Questi sono veloci ma decisamente poco flessibili.
- Le classi collezione sono tipicamente molto più potenti e flessibili dei vettori e degli altri tipi di collezioni di oggetti nativi.

Raffinare le relazioni di analisi – Come raffinare le relazioni di analisi – Classi collezione

- Una classe collezione è una classe le cui istanze sono specializzate nella gestione di una collezione di altri oggetti.
- Molti linguaggi mettono a disposizione una libreria standard che contiene classi collezione.
- Uno dei modi per eccellere nella progettazione e nell'implementazione OO è la padronanza delle classi collezione.
- Tutte le classi collezione hanno metodi per:
  - Aggiungere oggetti alla collezione.
  - Rimuovere oggetti dalla collezione.
  - Recuperare il riferimento a un oggetto della collezione.
  - Iterare sulla collezione.
- Esistono tante collezioni diverse e ciascuna è specializzata nel gestire una collezione di oggetti in modo particolare.
- Bisogna saper scegliere la classe collezione giusta per il compito giusto.

Raffinare le relazioni di analisi – Come raffinare le relazioni di analisi – Classi collezione

- Un esempio di traduzione delle associazioni uno-a-molti tramite classi collezione è riportato nella seguente figura:

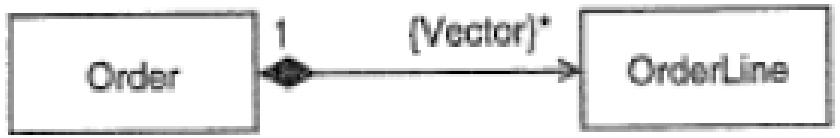


- Per quanto riguarda la modellazione con le classi collezione ci sono quattro strategie di base:
  - Modellare esplicitamente anche la classe collezione:
    - Questa è la situazione riportata nella figura precedente.
    - Questa soluzione ha il vantaggio che è molto esplicita ma ha anche lo svantaggio che le classi collezione appesantiscono il modello di progettazione che risulta spesso poco leggibile.

Raffinare le relazioni di analisi – Come raffinare le relazioni di analisi – Classi collezione

- Sostituendo a ogni associazione uno-a-molti una classe collezione, il modello si riempie molto in fretta.
- La scelta della classe collezione è, inoltre, una decisione tattica.

- Specificare mediante lo strumento di modellazione come debba essere implementata ciascuna associazione uno-a-molti.
  - Molti strumenti di modellazione che generano il codice consentono di assegnare a ciascuna associazione uno-a-molti una specifica collezione.
  - Questa tecnica è illustrata nella figura seguente:



- Specificare la semantica della collezione aggiungendo una proprietà alla relazione senza però specificare nessuna classe collezione per l'implementazione.
  - È importante non esagerare quando si devono modellare collezioni di oggetti.

Raffinare le relazioni di analisi – Come raffinare le relazioni di analisi – Classi collezione

- In molti casi la scelta è più tattica che strategica.
- Questa spesso è una scelta valida perché è piuttosto concisa e consente di dare ai programmatori un suggerimento su quale classe collezione dovrebbero usare.
- Tuttavia, questa tecnica di solito non consente di avere il codice generato automaticamente.
- UML mette a disposizione delle proprietà standard che possono essere applicate alle molteplicità per indicare la semantica richiesta della collezione; esse sono riportate nella seguente tabella:

Proprietà Standard	*Descrizione*
{ordinata}	Gli elementi nella collezione vengono tenuti in un ordine rigoroso
{non-ordinata}	Non esiste alcun ordine degli elementi nella collezione
{univoca}	Gli elementi dell'insieme sono tutti unici: un oggetto compare nella collezione al massimo una sola volta
{non-univoca}	La collezione ammette duplicati

- L'ordine stabilisce se gli elementi nella collezione vengono tenuti in un ordine rigoroso rispetto a un altro.
- L'univocità determina se ogni oggetto nella collezione ha un'identità univoca.

Raffinare le relazioni di analisi – Come raffinare le relazioni di analisi – Classi collezione

- La semantica predefinita per una relazione uno-a-molti è {non ordinata, univoca}.

- Per esempio, se serve una classe collezione ordinata di `OrderLine`, si può esprimere la figura precedente come nella figura seguente:



- Le variazioni della proprietà relativa all'ordine e della proprietà relativa all'univocità comportano un insieme di classi collezione elencate nella tabella seguente:

Proprietà	*Classe collezione dell'OCL*
{non-ordinata, non univoca}	Bag
{non-ordinata, univoca}	Set
{ordinata, univoca}	OrderedSet
{ordinata, non-univoca}	Sequence

- Non ci si preoccupa di raffinare le relazioni uno-a-molti nelle classi collezione; questo compito è lasciato ai programmatori.

Raffinare le relazioni di analisi – Come raffinare le relazioni di analisi – La mappa

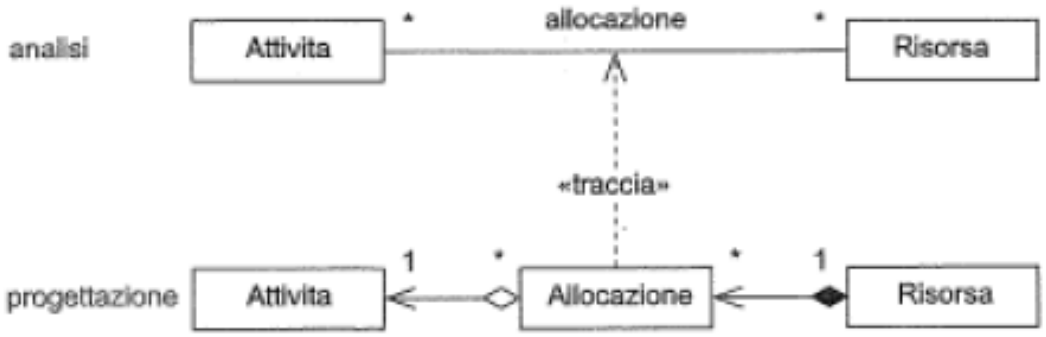
- Un altro tipo di classe collezione molto utile è la mappa, nota anche come dizionario.
- Queste classi si comportano in modo simile e una tabella di database con due sole colonne: la chiave e il valore.
- La mappa è un'ottima classe collezione quando c'è l'esigenza di avere una collezione di oggetti a cui si deve accedere utilizzando un valore chiave univoco, oppure quando si vuole costruire un indice per l'accesso veloce ad altre collezioni.
- Le mappe di solito si basano internamente su un insieme di nodi, dove a ogni nodo corrisponde una coppia di oggetti, l'oggetto chiave e l'oggetto valore.
- In Java una mappa può essere, ad esempio, gestita dalla classe `HashMap`.
- UML non ha alcuna proprietà standard per indicare una mappa e le mappe non fanno parte dell'OCL.
- Se si vuole segnalare che è richiesta una mappa nel modello di progettazione si indica il tipo specifico della classe collezione (per esempio, {`HashMap`}), oppure si usa il valore etichettato {`map keyName`}.

Raffinare le relazioni di analisi – Come raffinare le relazioni di analisi – Relazioni reificate

- Alcuni tipi di relazioni sono solo manufatti dell'analisi e non sono supportati dai linguaggi OO comunemente utilizzati.
- La reificazione è il processo con cui si implementano queste relazioni di analisi.
- I seguenti tipi di relazioni di analisi richiedono una reificazione:
  - Associazioni molti-a-molti.
  - Associazioni bidirezionali.
  - Classi associazione.

Raffinare le relazioni di analisi – Come raffinare le relazioni di analisi – Relazioni reificate – Associazioni molti-a-molti

- Le associazioni molti-a-molti non sono direttamente supportate dai linguaggi OO comunemente utilizzati.
- Esse, quindi, devono essere reificate.
- Per fare questo è necessario innanzitutto decidere quale delle due classi coinvolte sia il Tutto e quale sia la Parte.
- Nella seguente figura la relazione `allocazione` è stata reificata in una classe `Allocazione`; questo traduce l'associazione molti-a-molti in due aggregazioni:



- Basandosi sui requisiti del sistema si è deciso che la classe `Risorsa` è il Tutto; questo perché il sistema è incentrato sulle risorse.
- Se il sistema fosse stato incentrato sulle attività, allora avremmo utilizzato la classe `Attivita` come Tutto.

Raffinare le relazioni di analisi – Come raffinare le relazioni di analisi – Relazioni reificate – Associazioni molti-a-molti

- Si è anche deciso che `Risorsa` sarà responsabile del ciclo di vita dei propri oggetti `Allocazione` e si è, quindi, utilizzata una relazione di composizione.

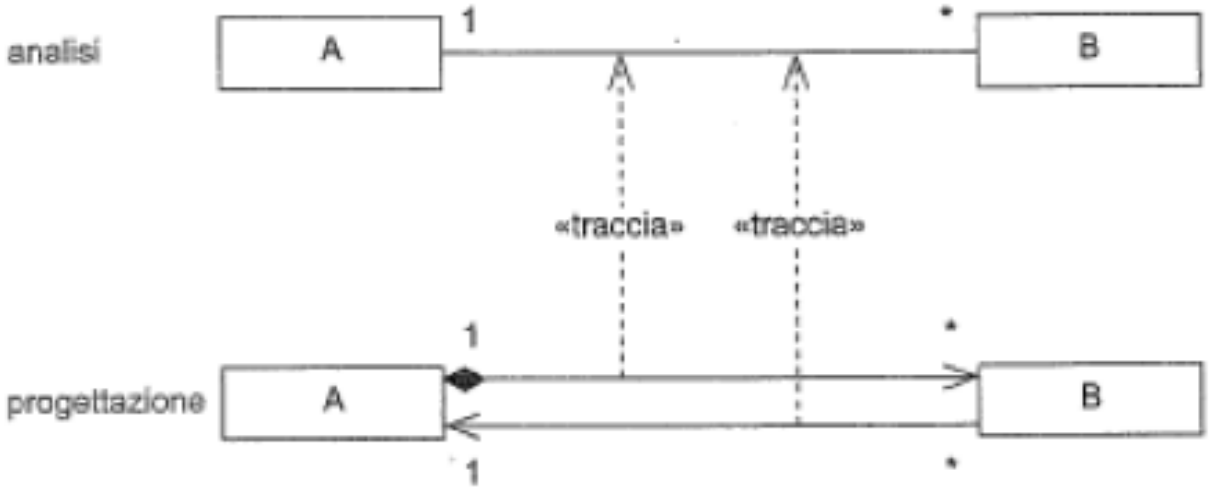
- Se il sistema avesse un punto di vista più equidistante allora si potrebbe dire che è incentrato sulle allocazioni (dalle risorse alle attività).

Raffinare le relazioni di analisi – Come raffinare le relazioni di analisi – Relazioni reificate – Associazioni bidirezionali

- Capita spesso la situazione in cui un oggetto `a` della classe `A` utilizza i servizi di un oggetto `b` della classe `B` e che l'oggetto `b` abbia a sua volta esigenza di effettuare un callback sull'oggetto `a` per usarne qualche servizio.
- Un tipico esempio è quello di un controllo GUI di una classe `Finestra` che è composto da uno o più oggetti `Pulsante`.
- In analisi per risolvere questa situazione è sufficiente modellare un'unica associazione bidirezionale.
- In progettazione è necessario trovare una soluzione migliore in quanto nessuno dei linguaggi OO comunemente utilizzati supporta vere associazioni bidirezionali.
- Bisogna quindi reificare l'associazione bidirezionale in due distinte relazioni o dipendenze unidirezionali.

Raffinare le relazioni di analisi – Come raffinare le relazioni di analisi – Relazioni reificate – Associazioni bidirezionali

- Tale modo di procedere è illustrato nella seguente figura:



- Quando si modella un callback è necessario tenere presente il vincolo di asimmetria a cui sono soggette le aggregazioni e le composizioni: un oggetto non può mai essere, direttamente o indirettamente, parte di se stesso.

- Esistono anche associazioni bidirezionali in cui il Tutto passa un riferimento a se stesso come parametro di uno dei metodi della Parte o dove la Parte istanzia direttamente il Tutto.

- In questi casi si dovrebbe usare una relazione di dipendenza dalla Parte del Tutto.

Raffinare le relazioni di analisi – Come raffinare le relazioni di analisi – Relazioni reificate – Classi associazione

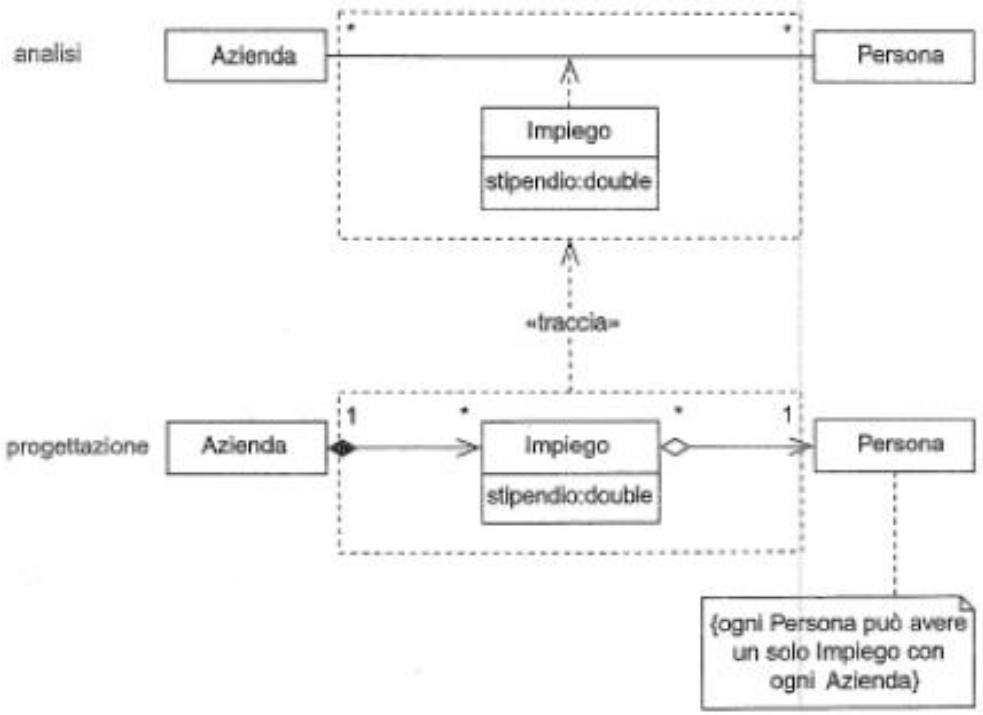
- Le classi associazione sono dei manufatti propri dell'analisi che non trovano riscontro nei linguaggi di programmazione OO comunemente utilizzati.

- Le classi associazione vengono reificate tramite l'uso di classi normali e una combinazione di associazioni, aggregazioni, composizioni o persino dipendenze.

- Si comincia decidendo quale estremo dell'associazione costituisca il Tutto, quindi si specificano opportunamente composizione, aggregazione e navigabilità.

Raffinare le relazioni di analisi – Come raffinare le relazioni di analisi – Relazioni reificate – Classi associazione

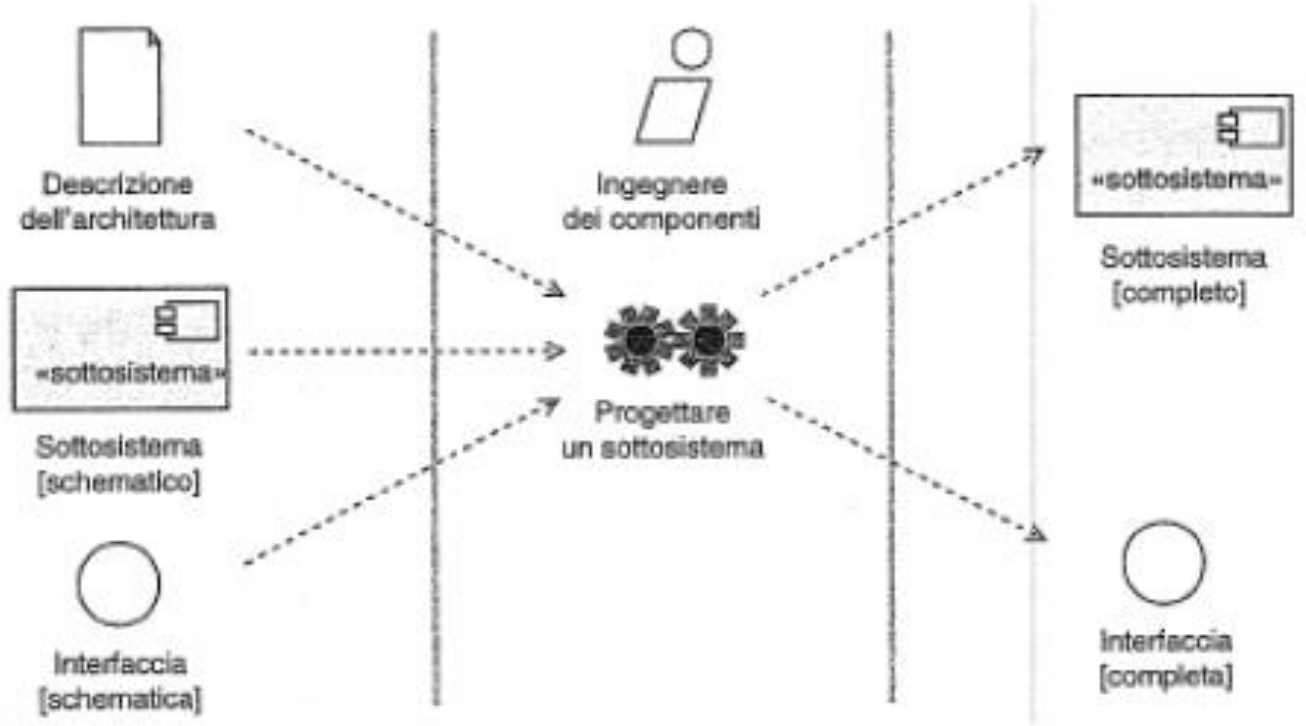
- Un esempio di traduzione delle classi associazione viene riportato nella seguente figura:



- Quando si reifica una classe associazione si perde parte della semantica.

Interfacce e componenti – Attività UP: progettare un sottosistema

- L'attività UP `Progettare un sottosistema` viene riportata nella seguente figura:



- Essa prevede la suddivisione del sistema in parti (sottosistemi) per quanto possibili indipendenti.

- Le interazioni tra i sottosistemi sono mediate da interfacce.

Interfacce e componenti – Attività UP: progettare un sottosistema

- Gli obiettivi della progettazione di sottosistemi sono:
  - Ridurre l'interdipendenza all'interno del sistema creando interfacce appropriate.
  - Assicurare che ogni sottosistema realizzi correttamente il comportamento specificato dalle sue interfacce.

Interfacce e componenti – Che cos'è un'interfaccia

- Un'interfaccia è un insieme di funzionalità pubbliche identificate da un nome.
- L'idea sottostante è quella di separare le specifiche di una funzionalità (l'interfaccia) dall'implementazione della stessa.
- Un'interfaccia non può essere istanziata; essa dichiara semplicemente un contratto che può essere realizzato da zero o più classificatori.
- Le possibili funzionalità specificate da un'interfaccia sono riportate nella seguente tabella:

L'interfaccia specifica	*Classificatore di realizzazione*
Operazione	Deve avere un'operazione con la stessa segnatura e semantica
Attributo	Deve avere operazioni pubbliche per impostare e prendere il valore dell'attributo; non è richiesto che il classificatore di realizzazione abbia tutti gli attributi specificati nell'interfaccia, ma deve comportarsi come se li avesse
Associazione	Deve avere un'associazione con il classificatore destinazione: se un'interfaccia specifica un'associazione con un'altra interfaccia, i classificatori di implementazione di queste interfacce devono avere un'associazione tra di loro
Vincolo	Deve supportare il vincolo
Stereotipo	Deve avere lo stereotipo
Valore etichettato	Deve avere il valore etichettato
Protocollo	Deve realizzare il protocollo

Interfacce e componenti – Cos'è un'interfaccia

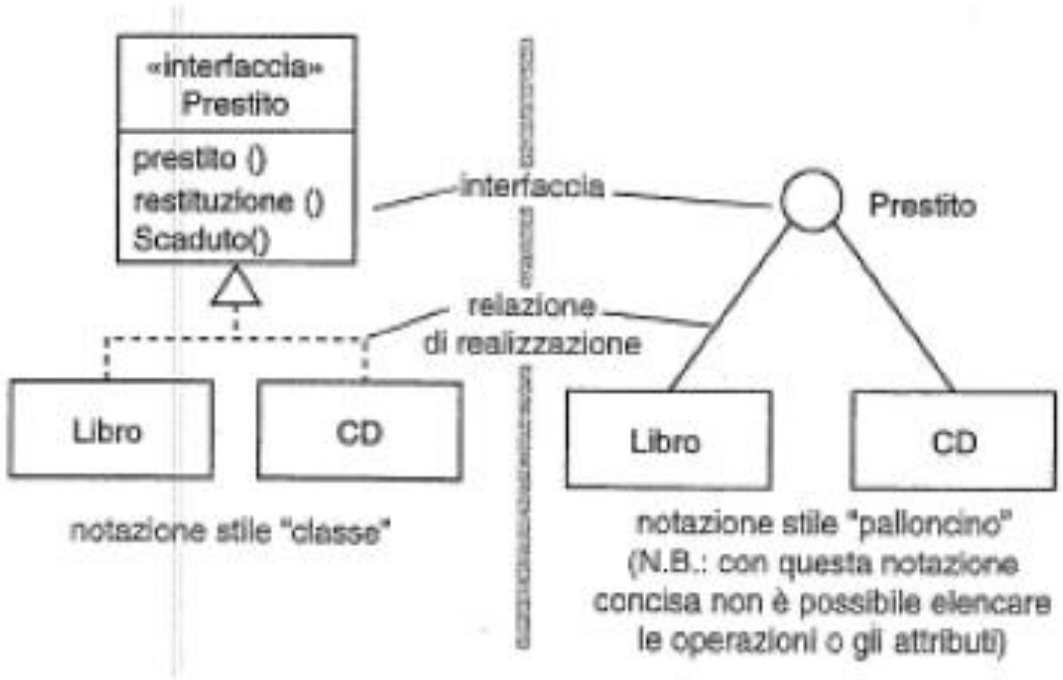
- Le interfacce richiedono anche la specifica della semantica delle loro funzionalità.
- Gli attributi e le operazioni di un'interfaccia dovrebbero essere ben specificati.
- Essi dovrebbero comprendere:
  - La segnatura completa.
  - La semantica dell'operazione.
  - Il nome e il tipo degli attributi.
  - Gli stereotipi di operazioni o attributi, vincoli e valori etichettati.
- Un'interfaccia definisce solo una specifica per le sue funzionalità e non prescrive mai una particolare implementazione.
- Le interfacce possono avere un impatto notevole sulle attività di progettazione: è più agevole "progettare per un contratto".

Interfacce e componenti – Cos'è un'interfaccia

- Un'interfaccia definisce un servizio offerto da una classe, un sottosistema o un componente.
- Java è il primo linguaggio comunemente utilizzato a introdurre un costrutto nativo per le interfacce.
- Per implementare le interfacce in C++ si può definire una classe astratta pura, ovvero una classe astratta le cui operazioni sono tutte astratte.

Interfacce e componenti – Interfacce richieste e fornite

- Le interfacce realizzate da un classificatore sono note come interfacce fornite.
- Quando un classificatore richiede una o più interfacce per il suo funzionamento, queste interfacce vengono definite interfacce richieste.
- La sintassi UML per le interfacce fornite viene riportata nella seguente figura:

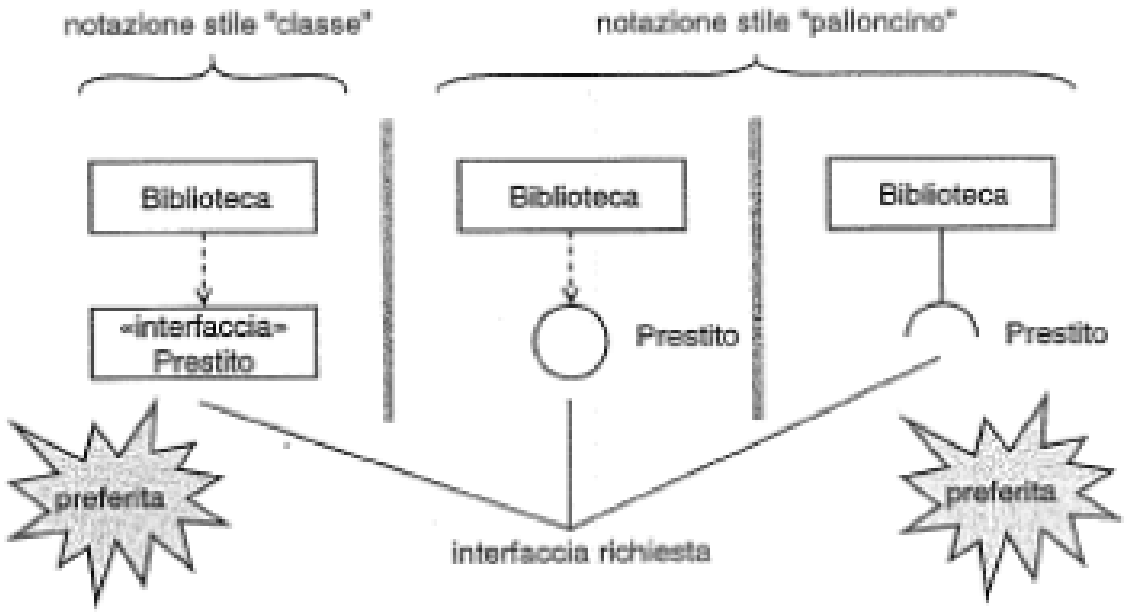


Interfacce e componenti – Interfacce richieste e fornite

- Esistono due possibili notazioni:
  - La notazione stile “classe”.
  - La notazione concisa stile “palloncino”.
- La figura precedente illustra l’esempio di un sistema di gestione di una biblioteca.
- Le interfacce tipicamente vengono scritte in UpperCamelCase come le classi.
- In Visual Basic e in C# il nome delle interfacce ha come prefisso una I maiuscola, per esempio `Iprestito`.
- La relazione di realizzazione è quella che intercorre tra una specifica (in questo caso un’interfaccia) e ciò che la specifica realizza (in questo caso le classi Libro e CD).
- Nella notazione stile “classe” la relazione di realizzazione viene disegnata come una linea tratteggiata che termina con una punta a triangolo, mentre nella notazione stile “palloncino” si utilizza una linea continua senza punta.

Interfacce e componenti – Interfacce richieste e fornite

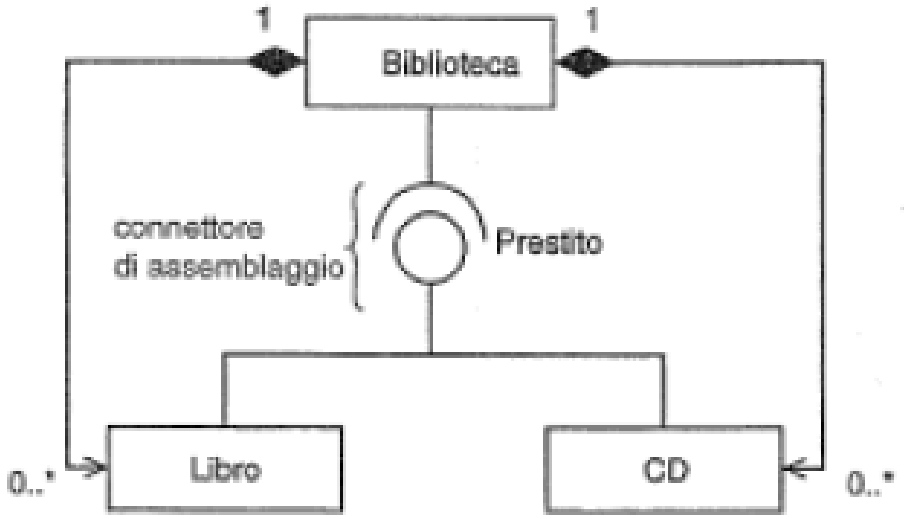
- La sintassi UML per le interfacce richieste viene riportata nella seguente figura:



- In questa figura la classe `Biblioteca` richiede l'interfaccia `Prestito`.

Interfacce e componenti – Interfacce richieste e fornite

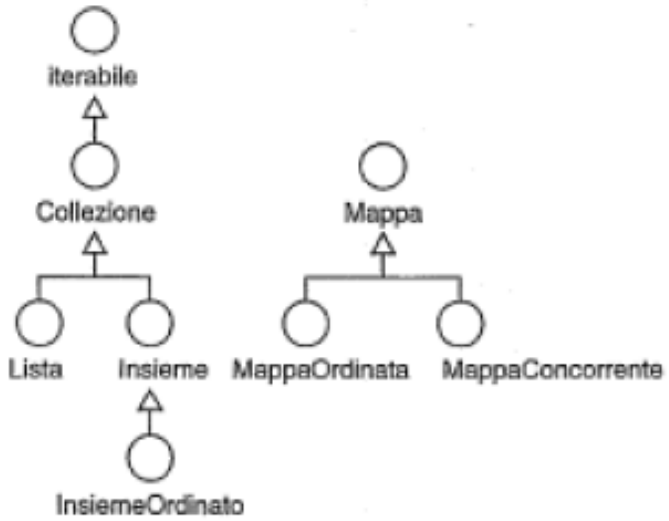
- Nella seguente figura viene riportato il sistema `Biblioteca` assemblato:



- Un altro valido esempio di interfacce con più implementazioni si trova nelle classi `Collezione` delle librerie standard Java.

Interfacce e componenti – Interfacce richieste e fornite

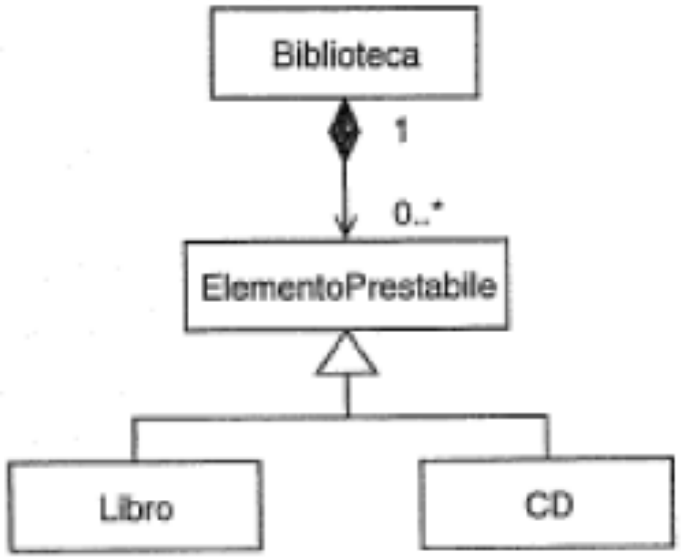
- Nella seguente figura vengono riportate le classi `Collezione` delle librerie standard di Java:



- Nella progettazione di un'interfaccia il progettista Java può rimandare effettivamente la realizzazione dell'interfaccia a quando deve essere implementata.

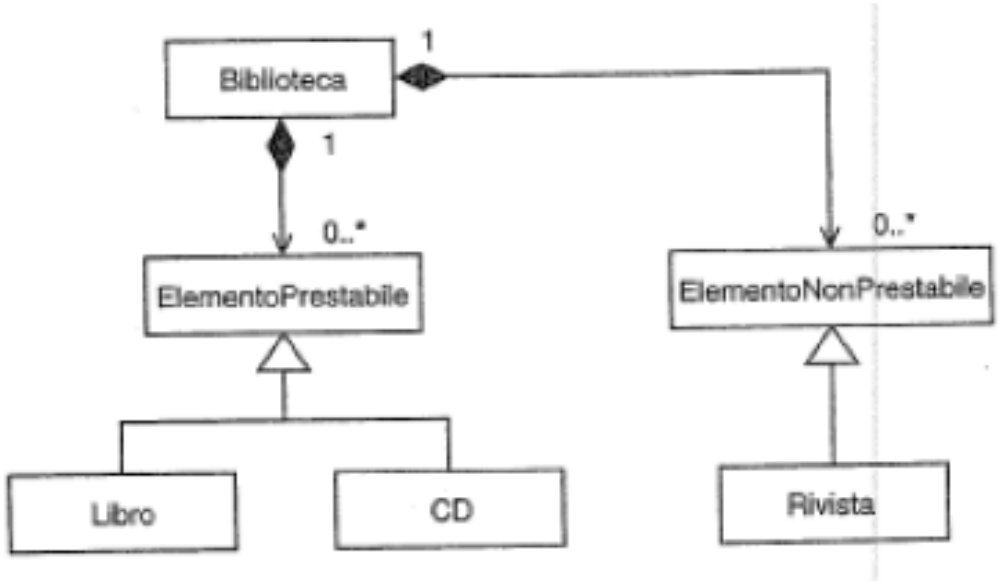
Interfacce e componenti – Realizzazione di interfaccia o ereditarietà

- Esistono delle differenze tra realizzazione di interfaccia ed ereditarietà.
- Il principio di sostituibilità viene applicato sia all'ereditarietà sia alla realizzazione di interfaccia.
- Entrambi i tipi di relazione possono generare polimorfismo.
- Consideriamo la seguente soluzione alternativa basata sull'ereditarietà per il problema della biblioteca:



Interfacce e componenti – Realizzazione di interfaccia o ereditarietà

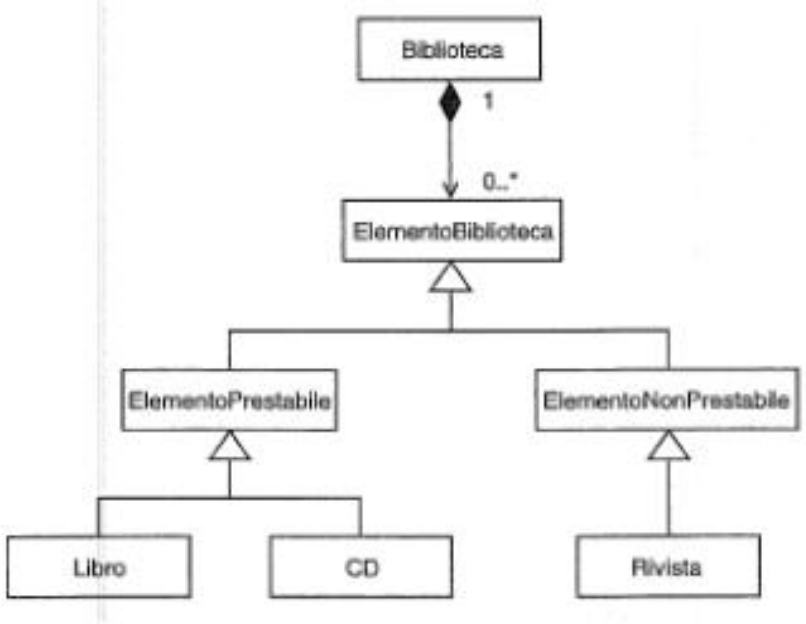
- Questa soluzione presenta alcuni problemi.
- In questo modello si dichiara che `Libri` e `CD` sono di tipo `ElementoPrestabile` ma questa caratteristica non appare sufficiente a specificare il loro tipo.
- È semanticamente più corretto mostrare `ElementoPrestabile` come un particolare ruolo che sia `Libri` che `CD` ricoprono rispetto alla `Biblioteca`.
- Per meglio comprendere ciò consideriamo l'aggiunta di `Riviste` alla `Biblioteca`; le riviste non sono prestabili:



Interfacce e componenti – Realizzazione di interfaccia o ereditarietà

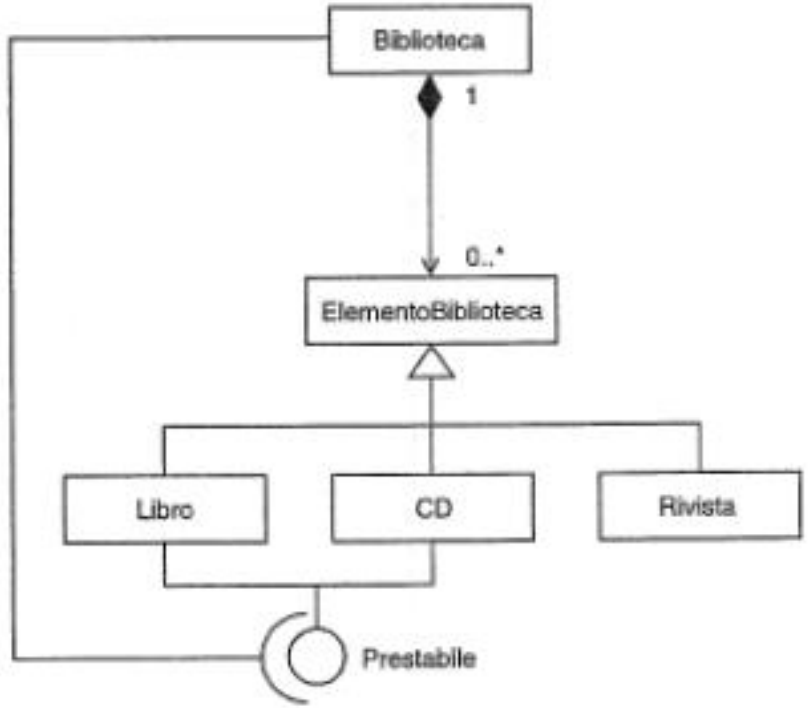
- Tale soluzione potrebbe sembrare accettabile ma non è molto elegante perché mette insieme due compiti della `Biblioteca` molto diversi, ovvero archiviare oggetti e prestare oggetti.

- È possibile migliorare questo modello aggiungendo un livello in più alla gerarchia di ereditarietà:



Interfacce e componenti – Realizzazione di interfaccia o ereditarietà

- Un modello che usa sia l'interfaccia che l'ereditarietà offre una soluzione più elegante:



Interfacce e componenti – Realizzazione di interfaccia o ereditarietà

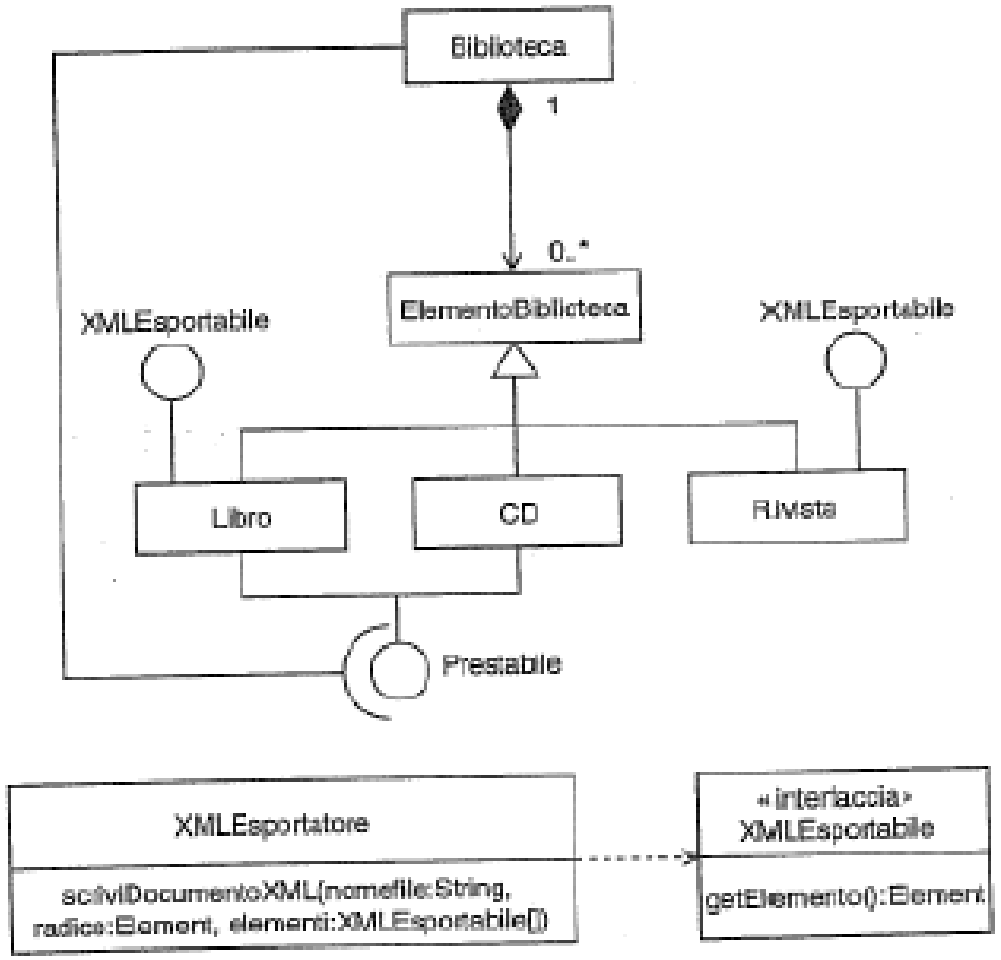
- I vantaggi di questa soluzione basata sull'interfaccia sono i seguenti:
  - Ogni elemento della `Biblioteca` è un `ElementoBiblioteca`.
  - La nozione di "prestabilità" è stata scorporata in un'interfaccia separata, `Prestabile`, che è applicabile a `ElementiBiblioteca` quando necessario.
  - Si hanno meno classi.
  - Si hanno meno relazioni di composizione.
  - Si ha una gerarchia di ereditarietà più semplice con solo due livelli.
  - Si hanno meno relazioni di ereditarietà.
- La soluzione basata sull'interfaccia è più semplice e ha una semantica più precisa.

Interfacce e componenti – Realizzazione di interfaccia o ereditarietà

- Si supponga di dover esportare dettagli di `Libri` e `Riviste` (ma non di `CD`) in un file XML.
- La soluzione viene così progettata:
  - Si introdurrà una classe `XMLEsportatore` per eseguire l'esportazione XML.
  - Si introdurrà un'interfaccia `XMLEsportabile` che definisce il protocollo che ogni elemento esportabile deve avere per funzionare con `XMLEsportatore`.
- Sono dati i seguenti requisiti non funzionali:
  - Il linguaggio di implementazione sarà Java.
  - Per l'elaborazione XML verrà utilizzata la libreria JDOM.
- Il protocollo `XMLEsportabile` consiste in una singola operazione, `getElemento()`, che restituisce una rappresentazione dell'elemento esportabile sotto forma di `Element`.

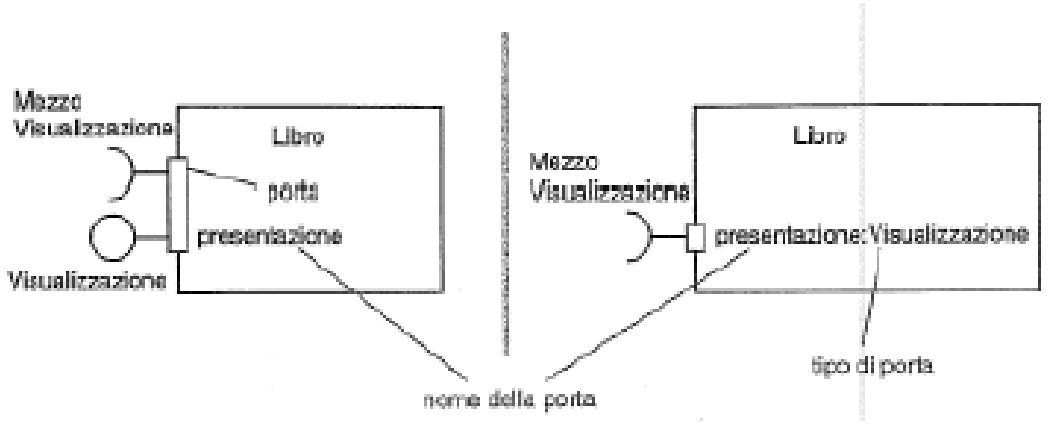
Interfacce e componenti – Realizzazione di interfaccia o ereditarietà

- La figura seguente mostra la soluzione completa:



Interfacce e componenti – Porte

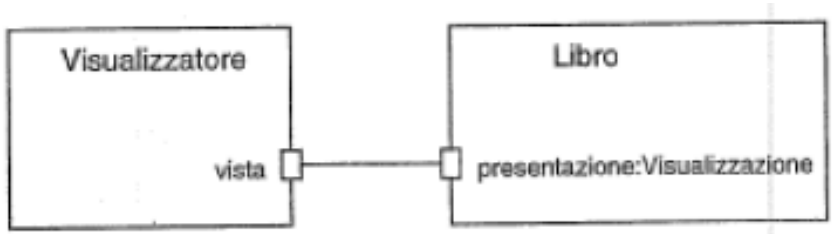
- Una porta raggruppa un insieme semanticamente coeso di interfacce fornite e richieste.
- Essa indica un punto specifico di interazione tra un classificatore e il suo ambiente.
- La figura seguente illustra la notazione per le porte e mostra una classe `Libro` che ha una porta chiamata `presentazione`:



- La figura mostra due varianti della notazione di porta.
- Le porte sono un metodo molto utile per strutturare interfacce fornite e richieste di un classificatore.

Interfacce e componenti – Porte

- La figura seguente mostra una classe `Visualizzatore` che si collega alla porta `presentazione` della classe `Libro`:



- Perché le porte siano connesse le loro interfacce fornite e richieste devono corrispondere.

- L'utilizzo delle porte è più conciso rispetto alla visualizzazione di tutte le interfacce fornite e richieste ma può necessitare di una lettura più attenta.

- Le porte possono avere una visibilità:

  - Quando una porta è disegnata sovrapposta al limite del classificatore è pubblica.

  - Se il rettangolo della porta viene mostrato all'interno del limite del classificatore la porta ha visibilità protetta (di default) o privata; la visibilità effettiva non viene mostrata graficamente ma è registrata nella specifica della porta.

Interfacce e componenti – Porte

- Un esempio di porta protetta viene riportata nella seguente figura:



- Le porte possono avere una molteplicità che può essere specificata tra parentesi quadre dopo il nome della porta e il nome del tipo (per esempio, `presentazione:visualizzazione[1]`).

- Essa precisa il numero di istanze della porta che avrà ciascuna istanza del classificatore.

Interfacce e componenti – Interfacce e sviluppo basato sui componenti

- Le interfacce sono l'elemento chiave dello sviluppo basato sui componenti (Component-Based Development, CBD).
- L'unico modo per creare del software flessibile basato su componenti è quello di utilizzare le interfacce sin dalla progettazione.
- Un'interfaccia consente l'uso di un qualunque numero di diverse implementazioni, a patto che ciascuna rispetti il contratto.

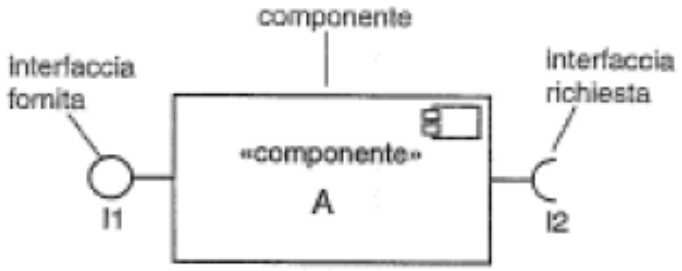
Interfacce e componenti – Cos'è un componente?

- Un componente rappresenta una parte modulare del sistema che incapsula i suoi contenuti e la cui manifestazione è sostituibile all'interno del suo ambiente.
- Un componente funge quindi da "scatola nera" il cui comportamento esterno è completamente definito dalle sue interfacce fornite e richieste.
- I componenti possono avere attributi e operazioni e possono partecipare a relazioni di associazione e generalizzazione.
- I componenti sono classificatori strutturati e possono avere una struttura interna che comprende parti e connettori.
- I componenti possono rappresentare qualcosa che può essere istanziato a tempo di esecuzione, per esempio un Enterprise Java Bean.
- Essi possono anche rappresentare un costrutto puramente logico, per esempio un sottosistema, istanziato solo indirettamente in virtù del fatto che le sue parti vengono istanziate.
- Un componente può essere reso manifesto da uno o più manufatti; per esempio, un componente EJB potrebbe essere mostrato tramite un file JAR (Java Archive).

Interfacce e componenti – Cos'è un componente?

- Il diagramma dei componenti può mostrare dei componenti, le dipendenze tra di essi e il modo in cui i classificatori vengono assegnati ad altri componenti.

- Un esempio di diagramma dei componenti viene riportato nella seguente figura:



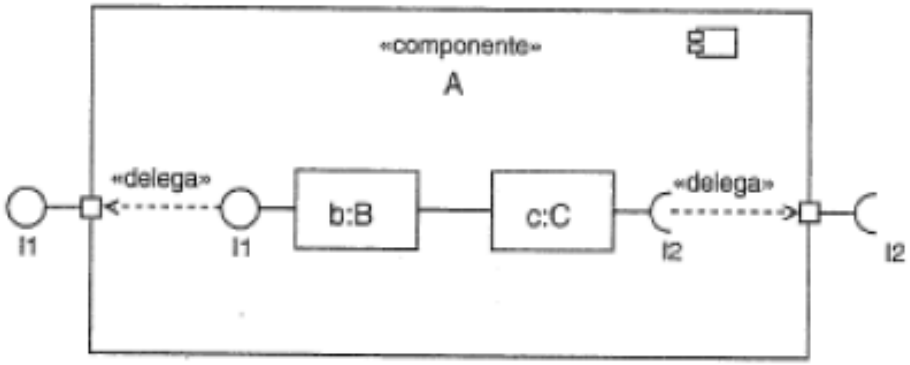
- Un componente viene disegnato come un riquadro con stereotipo <<componente>> e/o un'icona del componente nell'angolo superiore destro.

- Un componente può avere una struttura interna.

- È possibile mostrare le parti annidate all'interno del componente oppure esternamente, connettendole al componente stesso mediante relazioni di dipendenza.

Interfacce e componenti – Cos'è un componente?

- Nella seguente figura viene mostrata una rappresentazione di un componente con la sua struttura interna:



- Quando un componente ha struttura interna di solito delega le responsabilità definite dalle sue interfacce a una o più parti interne.

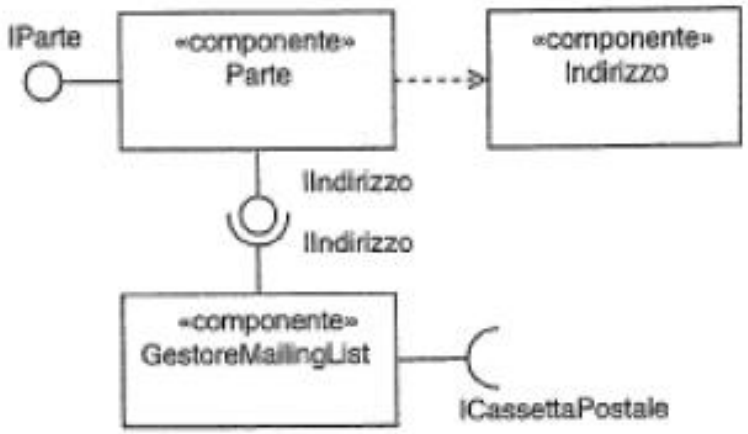
- I componenti possono dipendere da altri componenti.

- Perché i componenti non abbiano interdipendenze si media sempre la dipendenza con un'interfaccia.

- Quando un componente richiede un'interfaccia lo si può rappresentare come una dipendenza tra il componente e l'interfaccia oppure si può usare il connettore di assemblaggio.

Interfacce e componenti – Cos'è un componente?

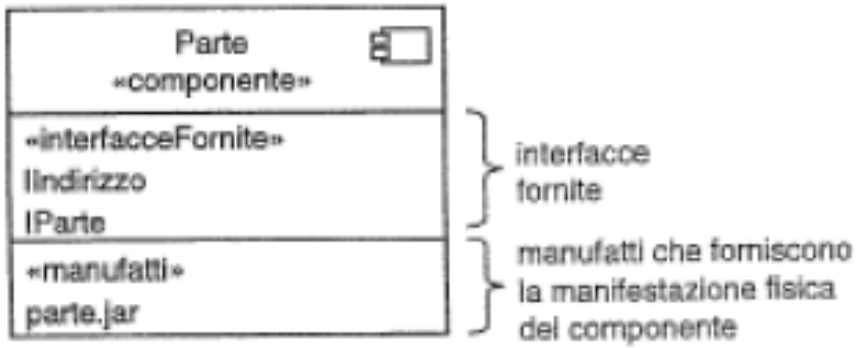
- Nella seguente figura viene mostrato l'utilizzo delle interfacce per modellare le dipendenze tra i componenti:



- Da questa figura si può notare che:
  - Il componente `Parte` fornisce due interfacce di tipo `IParte` e `Iindirizzo`.
  - Il componente `GestoreMailingList` richiede due interfacce di tipo `IIndirizzo` e `ICassettaPostale`.
  - Esiste un connettore di assemblaggio tra il componente `Parte` e il componente `GestoreMailingList`.
  - In questo modello il componente `Parte` si comporta come una façade.

Interfacce e componenti – Cos'è un componente?

- I componenti spesso vengono presentati semplicemente come "scatole nere" a cui sono associate le loro interfacce fornite e richieste.
- Sarebbe possibile mostrare un componente anche come una "scatola bianca" evidenziando i suoi dettagli interni.
- Un componente può essere rappresentato come una scatola bianca come mostrato nella seguente figura:



Interfacce e componenti – Stereotipi dei componenti

- I componenti sono l'elemento UML che ha più stereotipi perché possono essere usati in svariate circostanze.
- UML fornisce un piccolo insieme degli stereotipi standard dei componenti che vengono elencati nella seguente tabella:

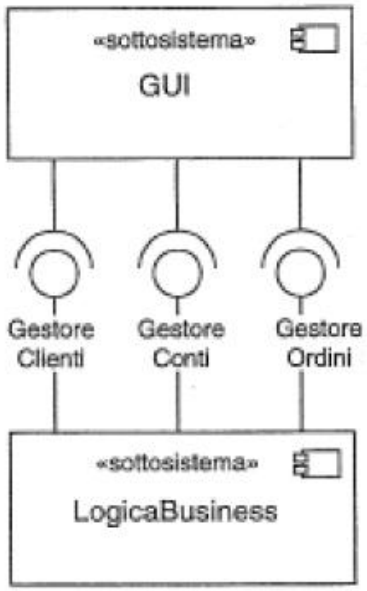
Stereotipo	*Significato*
`<<componenteBase>>`	Un componente che definisce un insieme di cose per scopi organizzativi o di sviluppo a livello di sistema
`<<entità>>`	Un componente informativo persistente che rappresenta un concetto di business
`<<implementazione>>`	Una definizione di componente che non ha specifica; è un'implementazione per una `<<specifica>>` separata da cui dipende
`<<specifica>>`	Un classificatore che specifica un dominio di oggetti senza definire l'implementazione fisica degli stessi; per esempio, un componente con stereotipo specifico ha solo interfacce fornite e richieste e nessun classificatore di realizzazione
`<<processo>>`	Un componente basato su transizioni
`<<servizio>>`	Un componente funzionale senza stato che calcola un valore
`<<sottosistema>>`	Un'unità di scomposizione gerarchica per grandi sistemi

Interfacce e componenti – Sottosistemi

- Un sottosistema è un componente che agisce come unità di scomposizione per un sistema più grande.
- I sottosistemi vengono disegnati come componenti con stereotipo <<`sottosistema`>>.
- I sottosistemi non possono essere istanziati a tempo di esecuzione, a differenza dei loro contenuti.
- Dal punto di vista dell'UP i sottosistemi sono un concetto fondamentale della strutturazione; essi consentono di suddividere un problema di sviluppo grande e difficile in molti problemi più piccoli e facilmente gestibili.
- Le interfacce vanno di pari passo con i sottosistemi.
- Esse connettono i sottosistemi per creare un'architettura di sistema.

Interfacce e componenti – Sottosistemi

- Un esempio di utilizzo di sottosistemi e interfacce viene riportato nella seguente figura:



Interfacce e componenti – Individuare le interfacce

- Quando si è a buon punto nella progettazione di un sistema vale la pena di esaminare il modello alla ricerca di potenziali interfacce.

- Valgono i seguenti suggerimenti:

  - Mettere in discussione ogni associazione; se si giunge alla conclusione che un'associazione deve essere più flessibile allora si può prendere in considerazione l'uso dell'interfaccia.

  - Mettere in discussione ogni messaggio inviato; se esistono situazioni in cui lo stesso messaggio potrebbe essere inviato a oggetti di classi diverse allora si può prendere in considerazione l'uso di un'interfaccia.

  - Individuare gruppi di operazioni che possono essere riusate in altre classi; ciascuno di questi gruppi può dare luogo ad un'interfaccia.

  - Individuare gruppi di operazioni ripetute in classi diverse.

  - Individuare gruppi di attributi ripetuti in classi diverse.

Interfacce e componenti – Individuare le interfacce

  - Ricercare classi del sistema che sembrino avere un ruolo simile o identico; quel ruolo potrebbe diventare un'interfaccia.
  - Individuare possibilità per future espansioni del sistema.
  - Osservare le dipendenze tra componenti: dove possibile mediarli tramite connettori di assemblaggio.
- Come si può vedere le interfacce possono essere usate in molti modi.

Interfacce e componenti – Progettare usando le interfacce

- È molto più facile progettare un sistema se le diverse parti si comportano in modo omogeneo e uniforme.
- L'utilizzo delle interfacce consente di definire protocolli comuni che molte classi o componenti diversi possono realizzare.
- Un esempio di scenario che può beneficiare dell'utilizzo delle interfacce è un sistema che deve fornire un'interfaccia comune per diversi sistemi aziendali esistenti:
  - `apri()`, `leggi()`, `scrivi()` e `chiudi()`
- Un secondo esempio riguarda un sistema che modella un'organizzazione:
  - il ruolo di `unitaindirizzabile` e l'interfaccia `NomeIndirizzo`.
- Le classi devono anche avere delle associazioni riflessive con se stesse.

Interfacce e componenti – Progettare usando le interfacce

- Le interfacce diventano uno strumento molto potente nel momento in cui vengono utilizzate per consentire l'inserimento di plug-in nel sistema.
- In un sistema progettato a interfacce, le associazioni e i messaggi inviati non sono più vincolati agli oggetti di una specifica classe ma diventano vincolati a una specifica interfaccia.
- Questo rende più facile l'aggiunta di nuove classi al sistema.
- Un tipo di modulo software che è molto utile poter inserire e cambiare a piacere sono i plug-in contenenti algoritmi.

Interfacce e componenti – Vantaggi e svantaggi delle interfacce

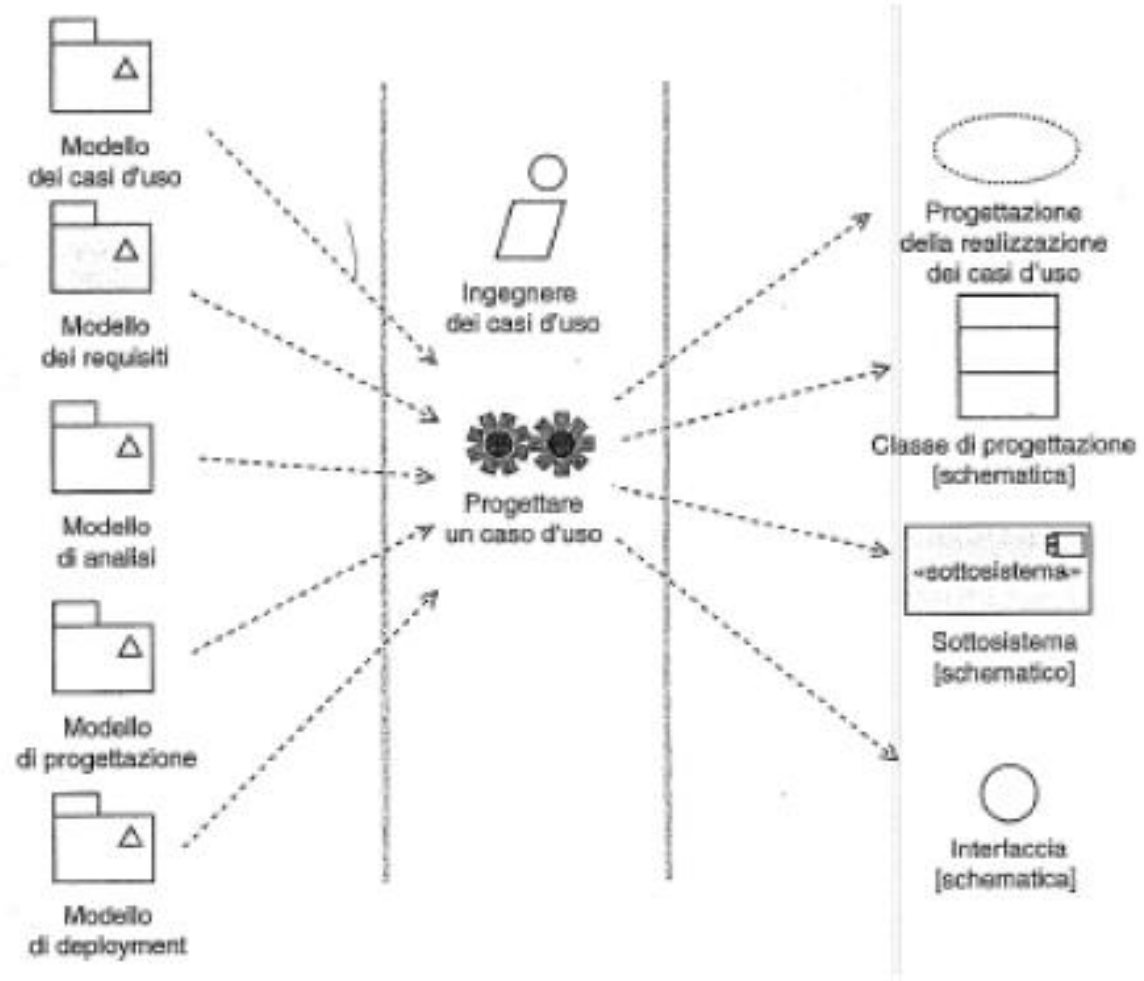
- La progettazione per classi vincola il progetto a certe specifiche implementazioni.
- La progettazione per interfacce, invece, si basa su contratti che possono essere realizzati da molte diverse implementazioni.
- La progettazione per interfacce svincola il modello da dipendenze dell'implementazione e ne aumenta la flessibilità e l'estendibilità.
- La progettazione per interfacce consente di ridurre il numero di dipendenze esistenti tra le classi, i sottosistemi e i componenti.
- L'interdipendenza è il peggior nemico dello sviluppatore a oggetti in quanto i sistemi fortemente interdipendenti sono difficili da comprendere, mantenere e far evolvere.
- Esistono anche degli svantaggi nell'uso delle interfacce.
- In linea generale rendere un sistema più flessibile comporta spesso anche un suo aumento di complessità.
- Chi usa la progettazione per interfacce deve comunque cercare sempre un buon compromesso tra flessibilità e complessità.

Interfacce e componenti – Vantaggi e svantaggi delle interfacce

- La flessibilità comporta, spesso, anche un costo in prestazioni ma si tratta di una considerazione di poco conto rispetto all'aumento della complessità.
- Alcuni dei requisiti di un sistema sono fluidi e cambiano molto in fretta mentre altri risultano relativamente stabili.
- Una semplificazione del sistema può essere ottenuta sacrificando in parte la flessibilità laddove i requisiti sono più stabili.
- Tuttavia la corretta modellazione di un sistema resta comunque più importante della sua modellazione flessibile.
- Bisogna ricordarsi sempre di utilizzare interfacce semplici e utili.

Progettazione della realizzazione dei casi d'uso – Attività UP: progettare un caso d'uso

- L'attività UP *Progettare un caso d'uso* prevede l'individuazione delle classi di progettazione, delle interfacce e dei componenti che agiscono per fornire il comportamento specificato da un caso d'uso, come mostrato nella seguente figura:



Progettazione della realizzazione dei casi d'uso – Attività UP: progettare un caso d'uso

- Questo è il processo della realizzazione dei casi d'uso ma ora ci si focalizza sulla progettazione.
- Questo cambiamento di focus ha conseguenze molto importanti:
  - Le realizzazioni dei casi d'uso nella progettazione conterranno classi di progettazione, interfacce e componenti invece di classi di analisi.
  - Il processo di creazione delle realizzazioni dei casi d'uso nella progettazione può servire anche per scoprire nuovi requisiti non funzionali e nuove classi di progettazione.
  - La progettazione della realizzazione dei casi d'uso aiuta ad individuare i cosiddetti "meccanismi principali".

Progettazione della realizzazione dei casi d'uso – Attività UP: progettare un caso d'uso

- Gli input di *Progettare un caso d'uso* sono:
  - Il modello dei casi d'uso.
  - Il modello dei requisiti.
  - Il modello di analisi.
  - Il modello di progettazione, che vedremo ora; UP mostra questo manufatto come un input in relazione a *Progettare un caso d'uso* per indicare la natura iterativa del processo.
  - Il modello di deployment.
- La progettazione è un processo iterativo e non una sequenza di passi.

Progettazione della realizzazione dei casi d'uso – Descrizione generale

- Una progettazione della realizzazione di un caso d'uso è una collaborazione tra oggetti e classi di progettazione che realizzano un caso d'uso.
- Esiste una relazione <<traccia>> tra la realizzazione di un caso d'uso di analisi e la progettazione della realizzazione dei casi d'uso.
- La progettazione della realizzazione dei casi d'uso specifica le decisioni relative all'implementazione e implementa i requisiti non funzionali.
- La progettazione della realizzazione di un caso d'uso è costituita da:
  - I diagrammi di interazione di progettazione.
  - I diagrammi di classe contenenti le classi di progettazione partecipanti.
- Nelle realizzazioni dei casi d'uso di analisi ci si concentra su cosa il sistema deve fare; nella progettazione ci si preoccupa di come il sistema lo farà.

Progettazione della realizzazione dei casi d'uso – Descrizione generale

- È importante ricordarsi che la modellazione deve servire solo per comprendere il sistema che si vuole costruire.
- Durante la progettazione ci si deve limitare a un livello di dettaglio che sia utile; questa si chiama progettazione strategica.
- Esiste anche la progettazione tattica che però può essere tranquillamente rimandata alla fase di implementazione.

Progettazione della realizzazione dei casi d'uso – Diagrammi di interazione di progettazione

- I diagrammi di interazione sono una parte fondamentale della progettazione della realizzazione dei casi d'uso.
- In genere si preferisce utilizzare i diagrammi di sequenza piuttosto che i diagrammi di comunicazione.
- I diagrammi di interazione possono essere:
  - Una rifinitura dei principali diagrammi di interazione prodotti dall'analisi.
  - Diagrammi completamente nuovi costruiti per illustrare questioni tecniche che sono state individuate durante le attività di progettazione.
- Durante la progettazione viene introdotto un certo numero di meccanismi, quali la persistenza degli oggetti, la distribuzione degli oggetti, le transazioni, etc.
- Spesso si realizzano nuovi diagrammi di interazione specifici per illustrare questi meccanismi.
- I diagrammi di interazione che illustrano i meccanismi sono solitamente trasversali a diversi casi d'uso.

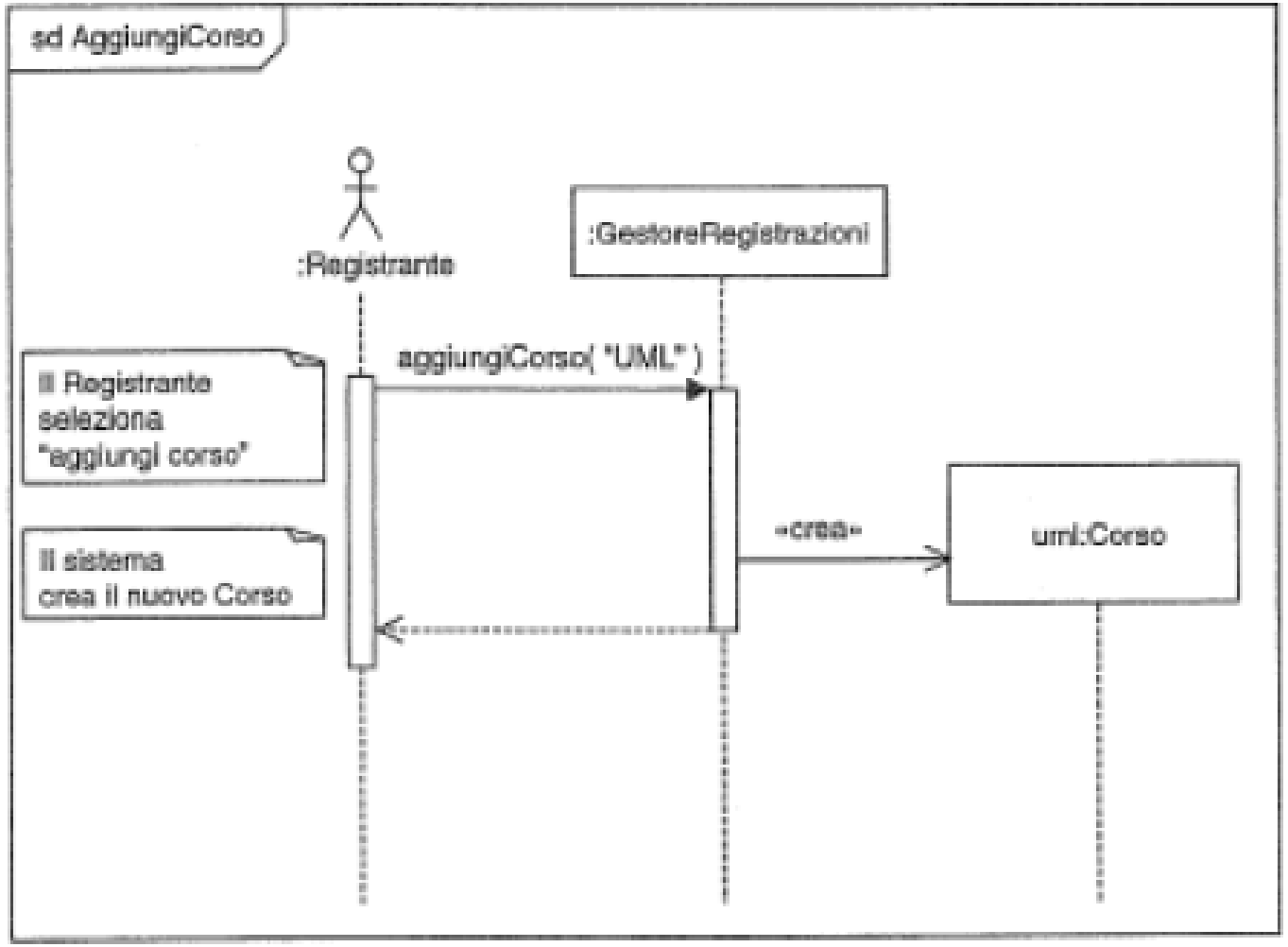
Progettazione della realizzazione dei casi d'uso – Diagrammi di interazione di progettazione

- Per capire meglio il ruolo dei diagrammi di sequenza nella progettazione consideriamo il seguente caso d'uso:

Caso d'uso:AggiungiCorso
ID: 8
Breve descrizione Aggiungi dettagli di un nuovo corso al sistema
Attori Registrante
Attori secondari Nessuno
Precondizioni 1. Il Registrante è stato autenticato dal sistema
Sequenza degli eventi principale 1. Il Registrante seleziona "aggiungi corso" 2. Il Registrante inserisce il nome del nuovo corso 3. Il sistema crea il nuovo corso
Postcondizioni 1. Un nuovo corso è stato aggiunto al sistema
Sequenza degli eventi alternativa CorsoEsistente

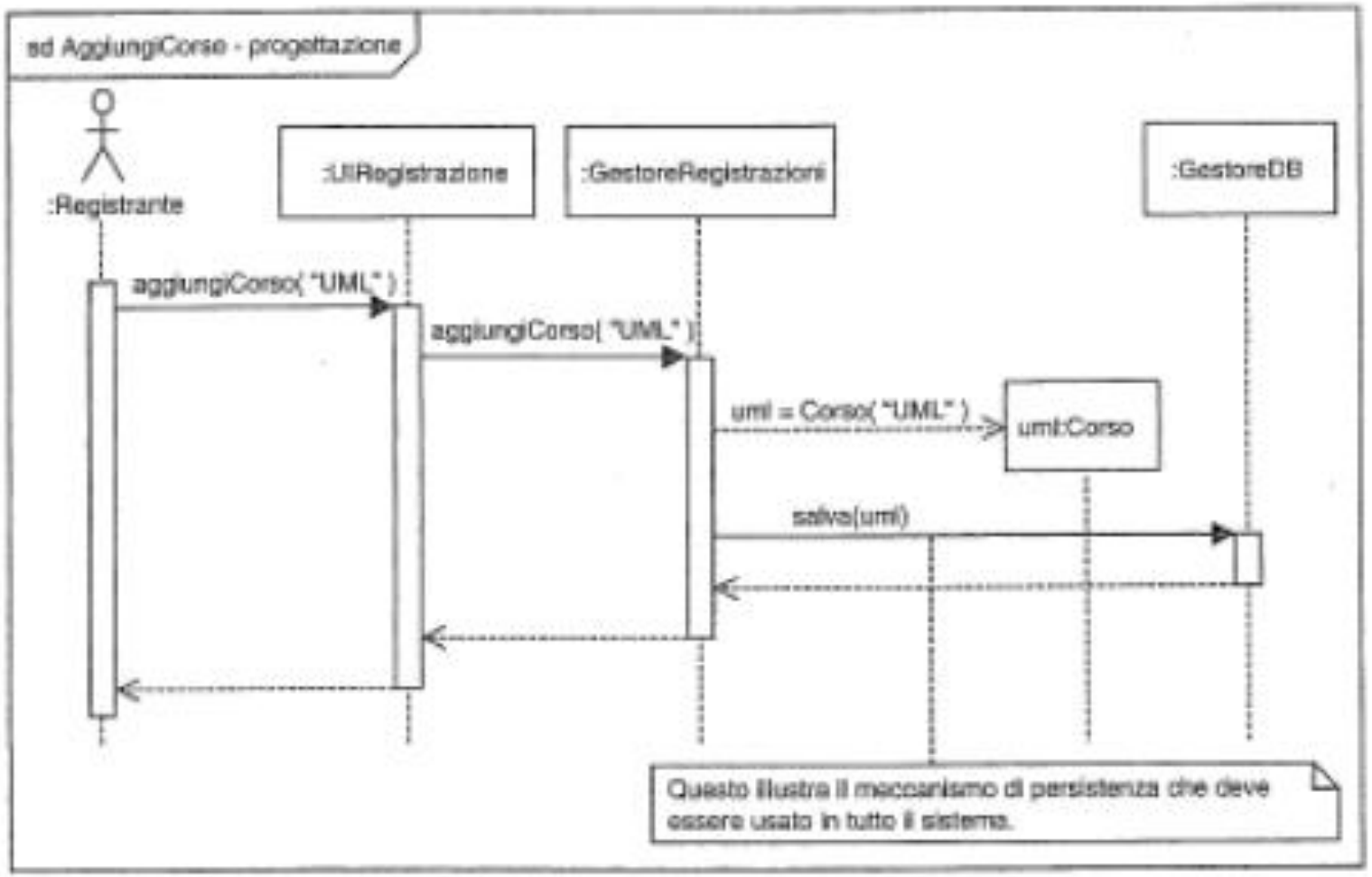
Progettazione della realizzazione dei casi d'uso – Diagrammi di interazione di progettazione

- La seguente figura illustra il diagramma di interazione dell'analisi realizzato in precedenza:



Progettazione della realizzazione dei casi d'uso – Diagrammi di interazione di progettazione

- La seguente figura mostra quello che potrebbe essere il diagramma di sequenza per il caso d'uso `AggiungiCorso` tipico delle fasi iniziali della progettazione:



Progettazione della realizzazione dei casi d'uso – Diagrammi di interazione di progettazione

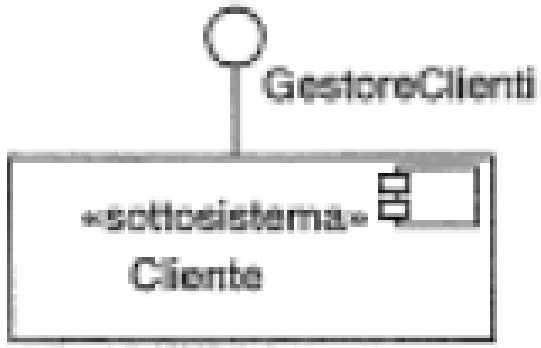
- Si può vedere che è stato aggiunto lo strato GUI, anche se non è stato modellato con troppo dettaglio.
- Viene incluso anche un meccanismo di gestione della persistenza mediante l'uso di un database.
- È essenziale che, una volta definito, questo stesso meccanismo venga utilizzato con consistenza in tutto il modello di progettazione.

Progettazione della realizzazione dei casi d'uso – Interazioni a livello di sottosistema

- Una volta che è stata definita l'architettura fisica dei sottosistemi e delle interfacce può risultare utile modellare le interazioni tra i sottosistemi.

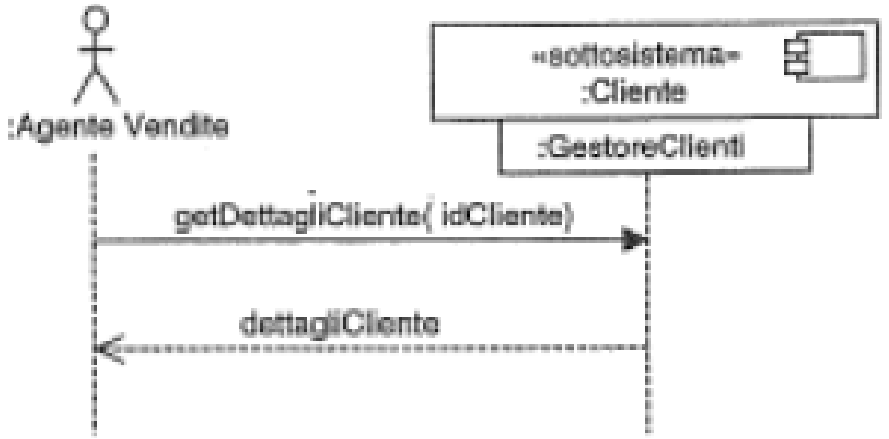
- In questo tipo di diagramma ogni sottosistema deve essere trattato come una scatola nera che fornisce e richiede servizi specificati dalle sue interfacce richieste e fornite pubbliche.

- La seguente figura mostra il sottosistema `Cliente` che possiede un'unica interfaccia chiamata `GestoreCliente`:



Progettazione della realizzazione dei casi d'uso – Interazioni a livello di sottosistema

- La seguente figura riporta parte di un diagramma di sequenza relativo a un attore che interagisce con questo sottosistema:



- L'interfaccia è stata visualizzata come un rettangolo che arriva sotto il sottosistema.

Progettazione della realizzazione dei casi d'uso – Modellazione della concorrenza

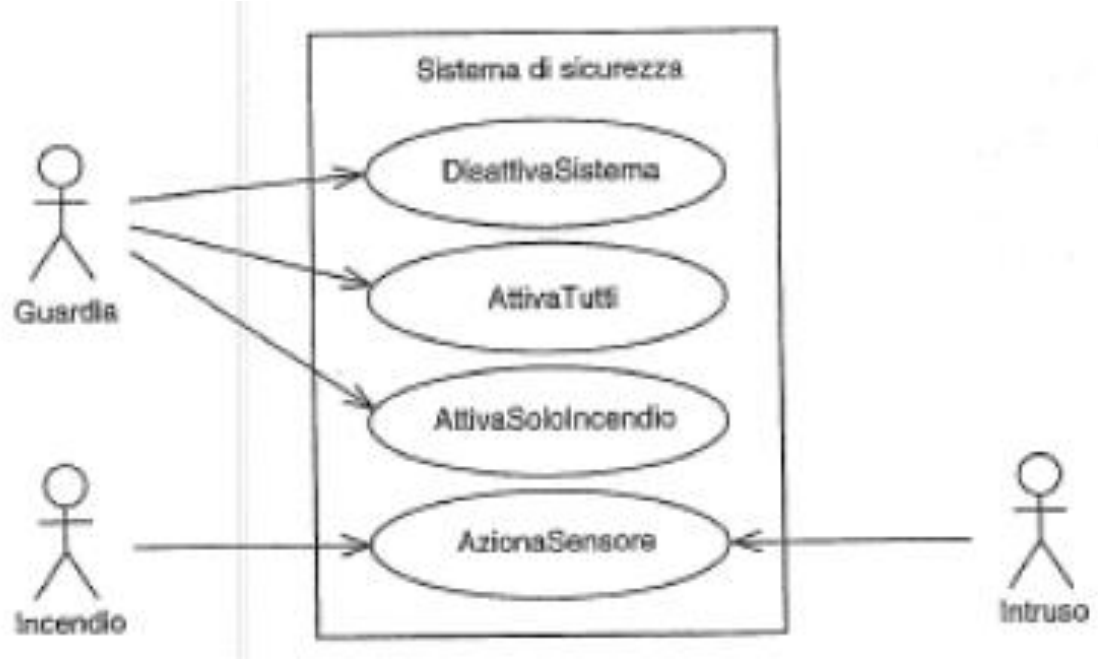
- "Concorrenza" significa che parti del sistema vengono eseguite in parallelo; la concorrenza è uno degli aspetti fondamentali della progettazione.
- UML 2 ha un buon supporto alla concorrenza; in particolare, esso fornisce:
  - Classi attive.
  - Biforcazioni e ricongiunzioni nei diagrammi di attività.
  - L'operatore `par` nei diagrammi di sequenza.
  - La possibilità di inserire un numero di sequenza come prefisso nei diagrammi di comunicazione.
  - La possibilità di avere origini multiple sui diagrammi di temporizzazione.
  - La possibilità di avere stati compositi ortogonali nelle macchine a stati.

Progettazione della realizzazione dei casi d'uso – Modellazione della concorrenza – Classi attive

- Il principio base della modellazione della concorrenza è che ogni thread di controllo o processo concorrente viene modellato come un oggetto attivo, un oggetto che incapsula il proprio thread di controllo.
- Gli oggetti attivi sono istanze di classi attive.
- Gli oggetti attivi e le classi attive vengono disegnati come classi e oggetti normali, ma con il bordo sinistro e il bordo destro doppi.
- La concorrenza tende ad essere molto importante per i sistemi embedded.
- Consideriamo un sistema di sicurezza che può monitorare un insieme di sensori che possono rilevare degli intrusi o un incendio; quando un sensore scatta il sistema fa suonare l'allarme.

Progettazione della realizzazione dei casi d'uso – Modellazione della concorrenza – Classi attive

- La seguente figura illustra il modello dei casi d'uso di questo sistema:



Progettazione della realizzazione dei casi d'uso – Modellazione della concorrenza – Classi attive

- La seguente figura riporta alcune specifiche dei casi d'uso del sistema:

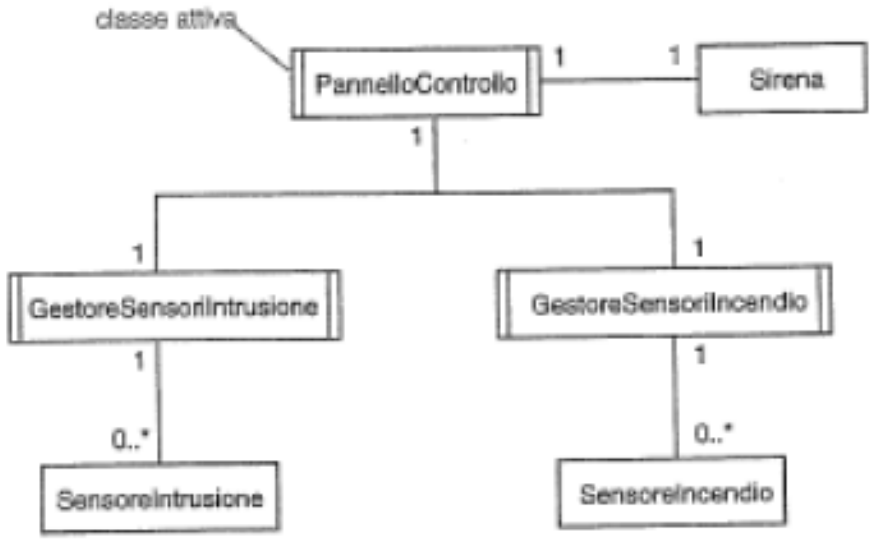
Caso d'uso:DisattivaSistema
ID: 1
Breve descrizione Disattiva il sistema
Attori Guardia
Attori secondari Nessuno
Precondizioni 1. La Guardia dispone della chiave di attivazione.
Sequenza degli eventi principale: 1. La Guardia usa la chiave di attivazione per disattivare il sistema. 2. Il sistema interrompe il monitoraggio dei sensori antifurto e antincendio
Postcondizioni 1. Il sistema è disattivato 2. Il sistema *non* effettua il monitoraggio dei sensori
Sequenza dei flussi alternativa Nessuna

Caso d'uso:AttivaTutti
ID: 2
Breve descrizione Attiva il sistema
Attori Guardia
Attori secondari Nessuno
Precondizioni 1. La Guardia dispone della chiave di attivazione.
Sequenza degli eventi principale: 1. La Guardia usa la chiave di attivazione per attivare il sistema. 2. Il sistema inizia il monitoraggio dei sensori antifurto e antincendio 3. Il sistema emette un segnale di conferma per indicare che è attivo
Postcondizioni 1. Il sistema è attivato 2. Il sistema effettua il monitoraggio dei sensori
Sequenza dei flussi alternativa Nessuna

Caso d'uso:AzionaSensore
ID: 3
Breve descrizione Un sensore è attivato
Attori Incendio Intruso
Attori secondari Nessuno
Precondizioni 1. Il sistema è attivato
Sequenza degli eventi principale: 1. Se l'attore Incendio aziona un SensoreIncendio 1.2 La Sirena suona l'allarme antincendio 2. Se l'attore Sistema allarme aziona un SensoreIntrusione 2.1 La Sirena suona l'allarme
Postcondizioni La Sirena suona
Sequenza dei flussi alternativa Nessuna

Progettazione della realizzazione dei casi d'uso – Modellazione della concorrenza – Classi attive

- Per prima cosa è necessario individuare le classi.
- Nei sistemi embedded l'hardware su cui viene eseguito il sistema è un'ottima fonte di potenziali classi.
- Capita spesso che l'architettura software migliore corrisponda strettamente all'architettura hardware.
- Nel nostro caso l'hardware del sistema di sicurezza è costituito da quattro componenti: il pannello dei controlli, la sirena, l'insieme dei sensori di incendio e l'insieme dei sensori di intrusione.
- Disponendo di queste informazioni è possibile disegnare il seguente diagramma delle classi:



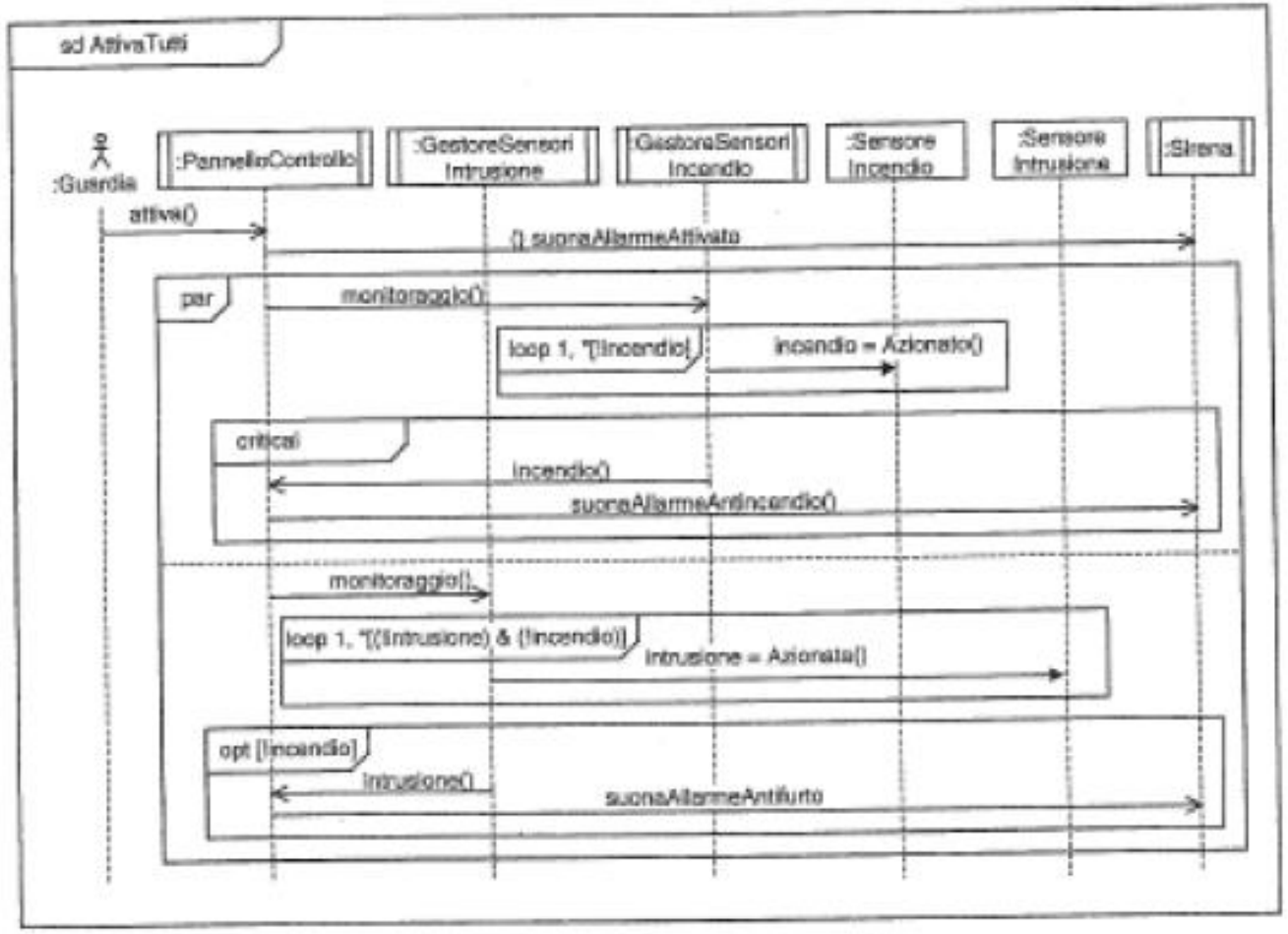
Progettazione della realizzazione dei casi d'uso – Modellazione della concorrenza – Classi attive

- In questo diagramma è necessario usare il multithreading in quanto il sistema di sicurezza deve continuamente monitorare i sensori di incendio e di intrusione.

- Per tale ragione le classi `PannelloControlli`, `GestoreSensoriIntrusione` e `GestoreSensoriIncendio` sono classi attive.

Progettazione della realizzazione dei casi d'uso – Modellazione della concorrenza – Concorrenza nei diagrammi di sequenza

- La seguente figura mostra il diagramma di sequenza per il caso d'uso *Attiva Tutti*.
- Questo diagramma descrive l'uso degli operatori `par`, `loop` e `critical`.



Progettazione della realizzazione dei casi d'uso – Modellazione della concorrenza – Concorrenza nei diagrammi di sequenza

- Il diagramma precedente esprime le seguenti attività:

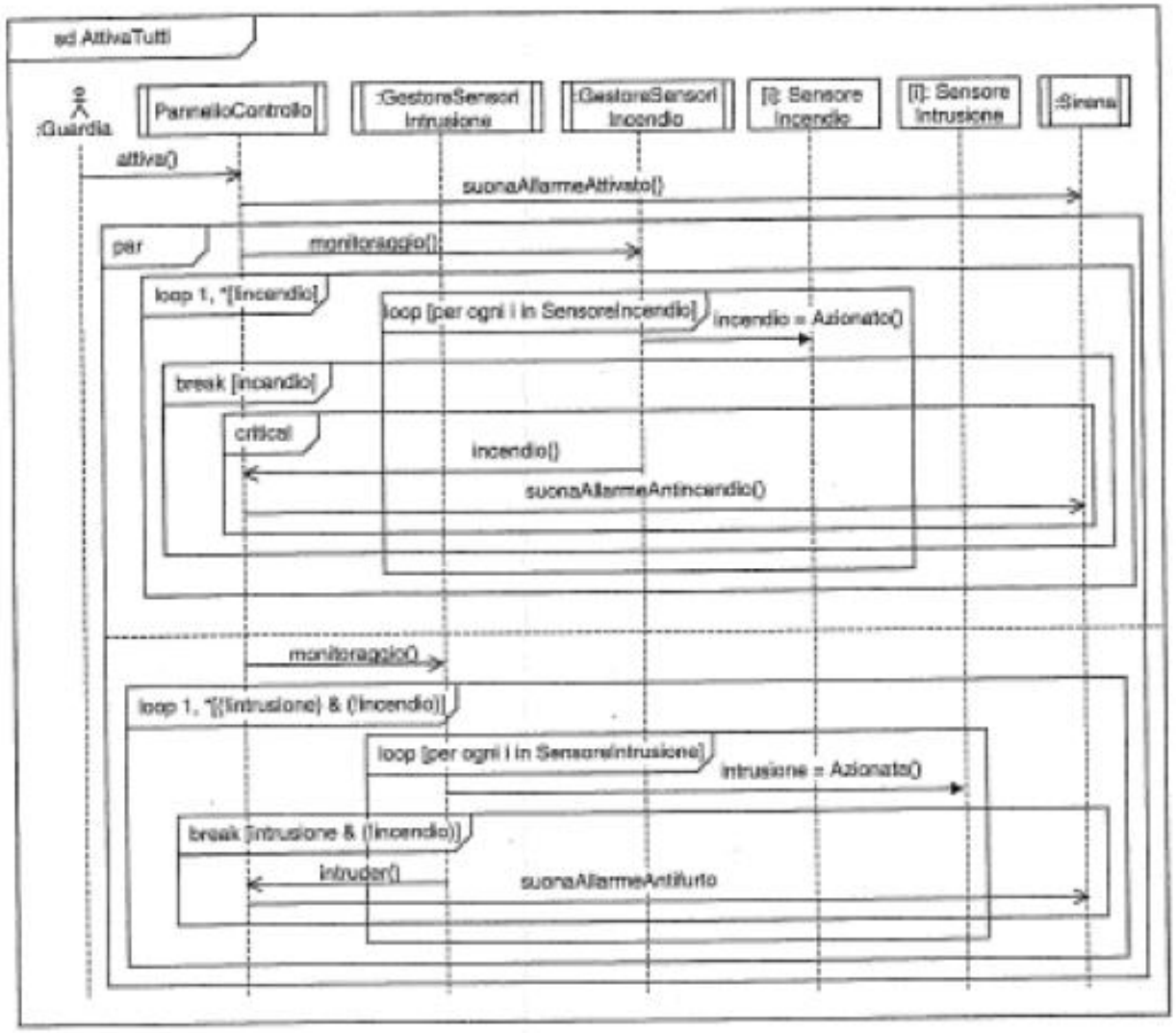
1. `:Guardia` invia il messaggio `attiva()` all'oggetto `:PannelloControllo`.
2. `:PannelloControllo` invia il messaggio `suonaAllarmeAttivato()` a `:Sirena`.
3. `:PannelloControllo` genera due thread di controllo, rappresentati dagli operandi dell'operatore `par`. Spostandosi verso il basso del diagramma, il primo operando di `par` verrà chiamato operando 1 e il secondo operando 2.
4. L'operando 1 di `par` prevede le seguenti azioni:
   - 4.1 `:PannelloControllo` invia il messaggio `monitoraggio()` a `:GestoreSensoriIncendio`.
   - 4.2 `:GestoreSensoriIncendio` entra in un ciclo che esamina ripetutamente `:SensoreIncendio`. Questo ciclo viene eseguito una sola volta (per impostare il valore iniziale della variabile `incendio`) e poi continua a iterare mentre `incendio` è `falso`.
   - 4.3 Quando `incendio` diventa `vero`:
     - 4.3.1 `:GestoreSensoriIncendio` entra in una sezione `critical` in cui:
       - 4.3.1.1 Invia il messaggio `incendio()` a `:PannelloControllo`.
       - 4.3.1.2 `:PannelloControllo` invia il messaggio `suonaAllarmeAttivato()` a `:Sirena`.
5. L'operando 1 di `par` termina.
6. L'operando 2 di `par` prevede le seguenti azioni:
   - 6.1 `:PannelloControllo` invia il messaggio `monitoraggio()` a `:GestoreSensoriIntrusione`.
   - 6.2 `:GestoreSensoriIntrusione` entra in un ciclo che esamina ripetutamente `:SensoreIntrusione`. Questo ciclo viene eseguito una sola volta (per impostare il valore iniziale della variabile `intrusione`) e poi continua a iterare mentre `intrusione AND incendio` sono `falsi`.
   - 6.3 Quando `intrusione` diventa `vero`:
     - 6.3.1 Se `incendio` è `falso`:
       - 6.3.1.1 `:GestoreSensoriIntrusione` invia il messaggio `intrusione()` a `:PannelloControllo`;
       - 6.3.1.2 `:PannelloControllo` invia il messaggio `suonaAllarmeAttivato()` a `:Sirena`.
7. L'operando 2 di `par` termina.
8. L'interazione termina.

Progettazione della realizzazione dei casi d'uso – Modellazione della concorrenza – Concorrenza nei diagrammi di sequenza

- In questa interazione è necessario evidenziare alcuni punti interessanti:
  - Entrambi gli operandi di `par` vengono eseguiti in parallelo.
  - Una sezione `critical` rappresenta un comportamento atomico che non può essere interrotto.
  - Entrambi i `loop` hanno una semantica `Ripetere` … `Fino`.
  - L'allarme che segnala l'incendio deve sempre avere la precedenza sull'allarme che segnala la presenza di intrusi; per ottenere ciò il `loop` dell'operando 2 di `par` viene terminato da un evento `incendio`() o da un evento `intruso`().
- In questo diagramma di sequenza si è mostrato soltanto un unico `SensoreIncendio` e un unico `SensoreIntrusione`.

Progettazione della realizzazione dei casi d'uso – Modellazione della concorrenza – Concorrenza nei diagrammi di sequenza

- Se si vuole visualizzare l'interazione del sistema su molti `SensoriIntrusione` e `SensoriIncendio` si dovrebbe modificare il diagramma aggiungendo due cicli interni:



Progettazione della realizzazione dei casi d'uso – Modellazione della concorrenza – Concorrenza nei diagrammi di sequenza

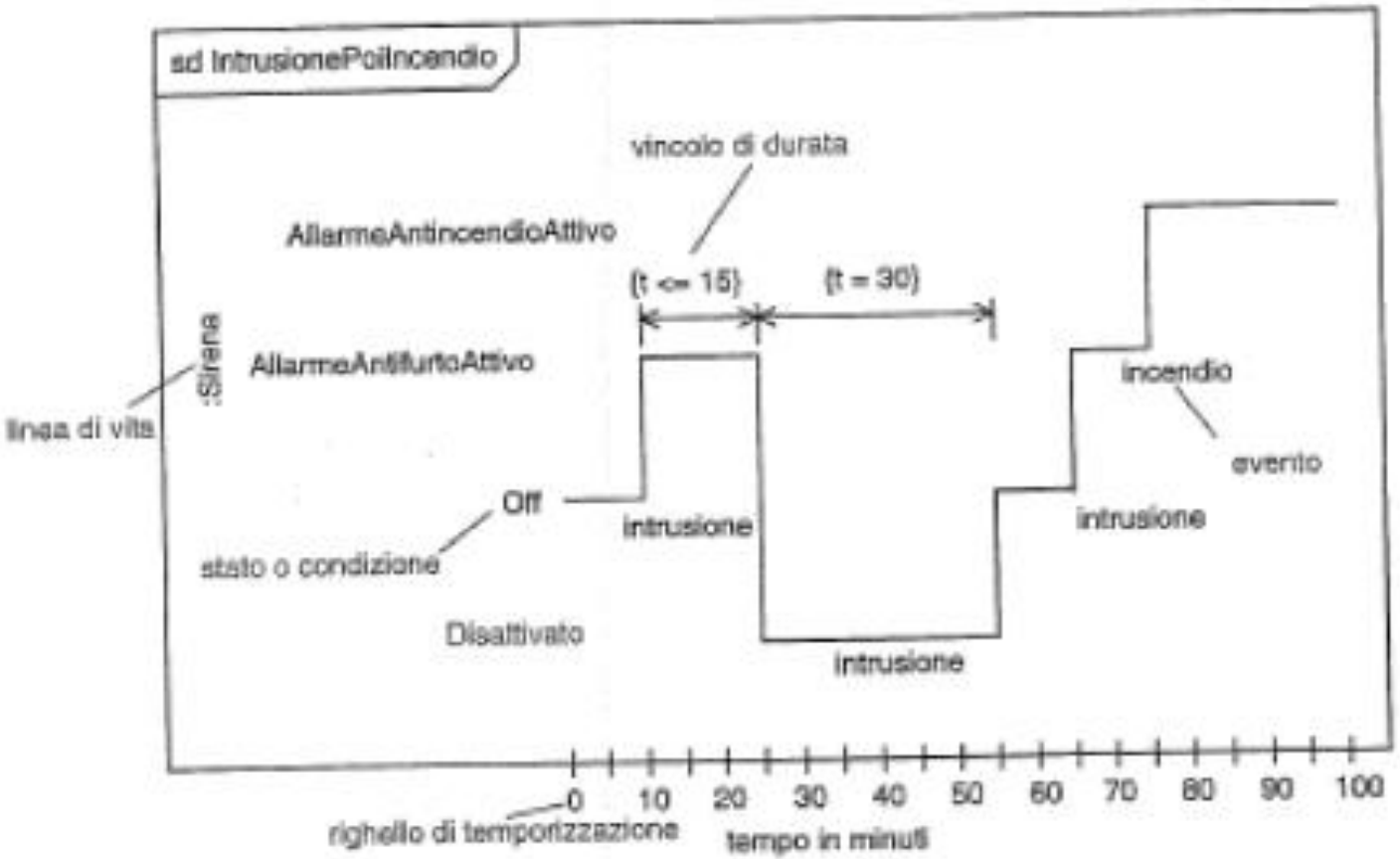
- Il nuovo ciclo esterno rimane inalterato; il nuovo ciclo interno esamina uno per uno tutti i `SensoreIncendio`.
- Si può usare il selettore `[i]` per mostrare il `SensoreIncendio` che il ciclo seleziona di volta in volta come una linea di vita del diagramma di sequenza.
- Questo ciclo contiene un'istruzione di interruzione che termina il ciclo interno ed esegue il suo operando quando `incendio è vero`; in questo caso si può entrare nella sezione `critical` come prima.
- È importante mantenere i diagrammi di sequenza quanto più semplici possibile.
- Il secondo diagramma di sequenza sul sistema visto in precedenza potrebbe essere troppo dettagliato; probabilmente era meglio fermarsi al primo.

Progettazione della realizzazione dei casi d'uso – Diagrammi di temporizzazione

- Un'area in cui UML era debole era la modellazione dei sistemi in tempo reale.
- In questi sistemi la temporizzazione è fondamentale e un evento deve seguirne un altro all'interno di una finestra di temporizzazione specificata.
- Si dice "finestra di temporizzazione" perché il tempo assoluto non è accessibile agli sviluppatori.
- In UML 1 i vincoli di temporizzazione si potevano esprimere su vari diagrammi ma nessun tipo di diagramma era stato attribuito esplicitamente alla modellazione della temporizzazione.
- UML 2 fornisce un apposito diagramma di temporizzazione per modellare sistemi in tempo reale.
- I diagrammi di temporizzazione per molti anni sono stati usati con successo nell'industria elettronica per modellare i vincoli di temporizzazione dei circuiti elettronici.

Progettazione della realizzazione dei casi d'uso – Diagrammi di temporizzazione

- La seguente figura mostra un semplice diagramma di temporizzazione per la classe `Sirena`:



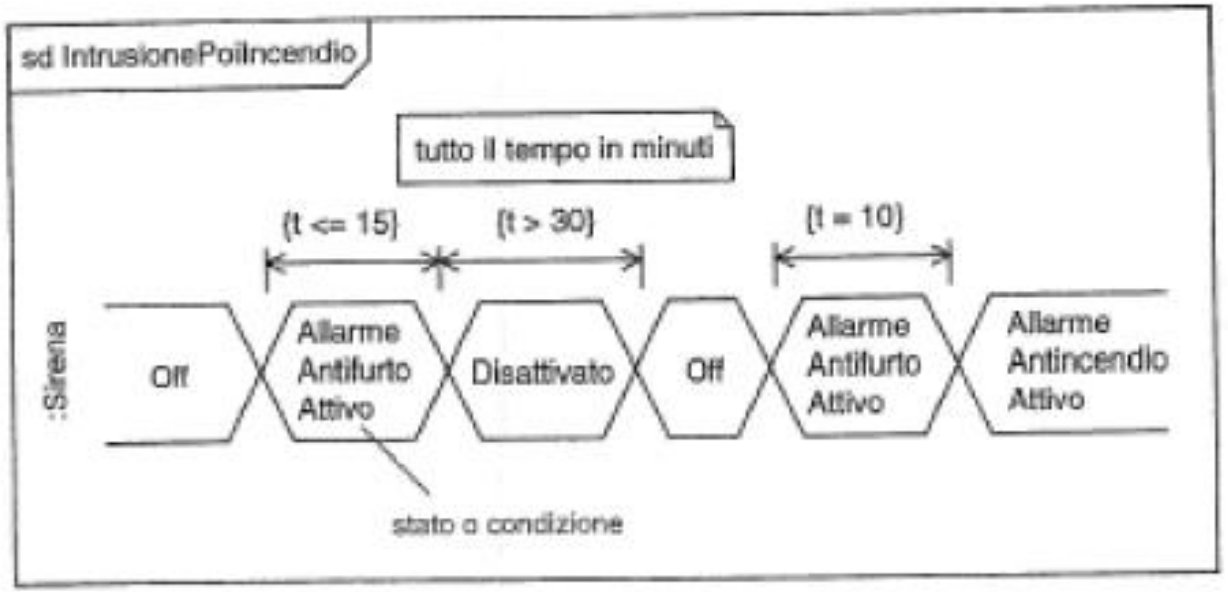
Progettazione della realizzazione dei casi d'uso – Diagrammi di temporizzazione

- Nella seguente figura viene riportata una descrizione del precedente diagramma di temporizzazione:

  - $t = 0$: `:Sirena` è nello stato `Off`.
  - $t = 10$: C'è un evento `intrusione` e `:Sirena` transita nello stato `AllarmeAntifurtoAttivo`.
  - $t = 25$: L'allarme antifurto può suonare al massimo per 15 minuti, a seconda di come è stato impostato. `:Sirena` transita nello stato `Disattivato`. Deve rimanere in questo stato per 30 minuti (a causa della leggi locali).
  - $t = 35$: C'è un evento `intruzione` ma `:Sirena` è `Disattivata` e quindi non può suonare.
  - $t = 55$: `:Sirena` ritorna allo stato `Off`.
  - $t = 65$: Vi è un altro evento `intrusione`. `:Sirena` transita dallo stato `Off` allo stato `AllarmeAntifurtoAttivo`.
  - $t = 75$: Vi è un evento incendio. `:Sirena` transita dallo stato `AllarmeAntifurtoAttivo` allo stato `AllarmeAnticendioAttivo`.
  - $t = 100$: L'interazione finisce lasciando `:Sirena` nello stato `AllarmeAnticendioAttivo`.

Progettazione della realizzazione dei casi d'uso – Diagrammi di temporizzazione

- I diagrammi di temporizzazione possono essere disegnati in una forma più compatta in cui gli stati sono posti orizzontalmente:

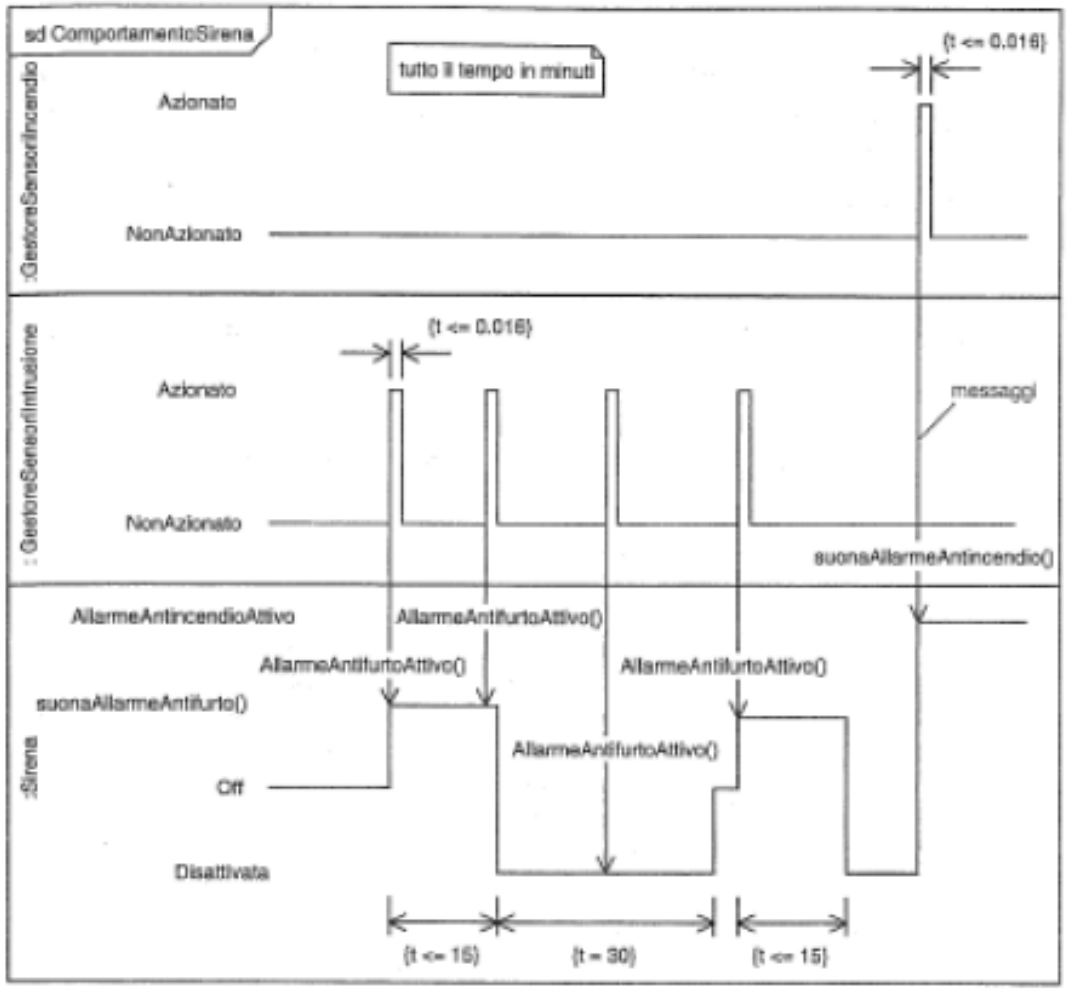


- Con questa forma compatta normalmente viene data più enfasi agli stati e al tempo relativo piuttosto che a una nozione di tempo assoluto modellato da un righello di temporizzazione.

- Si possono usare i diagrammi di temporizzazione anche per illustrare i vincoli di temporizzazione nelle interazioni tra due o più linee di vita.

Progettazione della realizzazione dei casi d'uso – Diagrammi di temporizzazione

- La seguente figura mostra l'interazione tra le linee di vita `:GestoreSensoriIncendio`, `:GestoreSensoriIntrusione` e `:Sirena`:



Progettazione della realizzazione dei casi d'uso – Diagrammi di temporizzazione

- Vanno considerati alcuni punti interessanti di questo diagramma di temporizzazione:
  - Il diagramma di temporizzazione ha tre sottosezioni, una per ogni linea di vita.
  - Nei diagrammi di temporizzazione si possono mostrare i messaggi tra le linee di vita.
  - Una volta avviati, entrambi i tipi di sensore transitano nello stato `Azionato` e poi ritornano nello stato `NonAzionato` in un secondo.
  - `:Sirena` risponde a un evento intrusione quando si trova nello stato `Off`; quando è `Disattivata` ignora gli eventi intrusione.
  - `:Sirena` risponde sempre agli eventi incendio anche quando si trova nello stato `Disattivata`.
- I diagrammi di temporizzazione forniscono un metodo utile per modellare i vincoli di temporizzazione sulle interazioni.

Macchine a stati - Introduzione

- Sia i diagrammi di attività che i diagrammi delle macchine a stati modellano aspetti del comportamento dinamico di un sistema ma con semantiche e scopi diversi.
- I diagrammi di attività si basano sulle reti di Petri e solitamente vengono utilizzati per modellare processi di business a cui partecipano molti oggetti.
- Le macchine a stati UML vengono generalmente usate per modellare il ciclo di vita di un'unica entità reattiva, quale una macchina a stati finiti.
- I tre elementi fondamentali delle macchine a stati sono gli stati, gli eventi e le transizioni:
  - Uno stato è una condizione o una situazione della vita di un oggetto.
  - Un evento è la specifica di un'occorrenza di interesse che ha una collocazione nel tempo e nello spazio.
  - Una transizione è il passaggio da uno stato a un altro in risposta ad un evento.
- Un'entità reattiva è un oggetto (in senso lato) che fornisce il contesto a una macchina a stati.

Macchine a stati - Introduzione

- Un'entità reattiva:
  - Risponde a eventi esterni.
  - Può generare e rispondere a eventi esterni.
  - Ha un ciclo di vita definito che può essere modellato come una successione di stati, transizioni ed eventi.
  - Può avere un comportamento corrente che dipende dai comportamenti precedenti.
- Nella modellazione UML le macchine a stati generalmente vengono definite nel contesto di un particolare classificatore (una classe, un caso d'uso, un sottosistema, un intero sistema).

Macchine a stati – Macchine a stati del comportamento e macchine a stati del protocollo

- In UML ci sono due tipi di macchine a stati:
  - Macchine a stati del comportamento.
  - Macchine a stati del protocollo.
- Le macchine a stati del comportamento utilizzano stati, transizioni ed eventi per definire il comportamento del classificatore di contesto.
- Non possono essere usate per classificatori, quali interfacce e porte, che non hanno un comportamento.
- Gli stati nelle macchine a stati del comportamento possono specificare una o più azioni che vengono eseguite quando si entra nello stato, vi si risiede o si esce.
- Le macchine a stati del protocollo utilizzano stati, transizioni ed eventi per definire il protocollo del classificatore di contesto.

Macchine a stati – Macchine a stati del comportamento e macchine a stati del protocollo

- Il protocollo include:
  - Le condizioni nelle quali le operazioni possono essere chiamate sul classificatore e le sue istanze.
  - I risultati delle chiamate di operazione.
  - L'ordine delle chiamate di operazione.
- Le macchine a stati del protocollo non suggeriscono nulla sull'implementazione di questo comportamento.
- Le macchine a stati del protocollo possono essere usate per definire il protocollo per tutti i classificatori, compresi quelli che non hanno implementazione.
- Gli stati nelle macchine a stati del protocollo non possono specificare delle azioni; questo è il compito delle macchine a stati del comportamento.
- I modellatori raramente fanno una distinzione tra macchine a stati del protocollo e macchine a stati del comportamento.
- Se si desidera distinguere le une dalle altre è possibile aggiungere la parola chiave {`protocollo`} dopo il nome della macchina a stati se questa è una macchina a stati del protocollo.

Macchine a stati – Macchine a stati e classi

- Le macchine a stati di solito vengono utilizzate per modellare il comportamento dinamico di classi.
- Ogni classe può avere un'unica macchina a stati del comportamento che modella tutti i possibili stati, eventi e transizioni per tutte le istanze di quella classe.
- Ogni classe può avere anche uno o più macchine a stati del protocollo, sebbene queste ultime vengano usate più raramente con le classi.
- Una classe eredita le macchine a stati del protocollo dei suoi genitori.
- Se una classe ha più di una macchina a stati queste devono essere coerenti l'una con l'altra.
- Le macchine a stati dovrebbero specificare il comportamento e il protocollo richiesti da tutti i casi d'uso a cui partecipano le istanze della classe.

Macchine a stati – Macchine a stati e UP

- Come i diagrammi di attività le macchine a stati non hanno un posto specifico in UP.
- Le si può utilizzare nel flusso di lavoro dell'analisi per modellare il ciclo di vita di classi dotate di stati interessanti come `Ordine` e `ContoBancario`.
- Le si può utilizzare nel flusso di lavoro della progettazione per modellare cose come la concorrenza.
- Le si può utilizzare nel flusso di lavoro dei requisiti quando si cerca di comprendere un caso d'uso complesso.
- La domanda da porsi è "la creazione di una macchina a stati per qualcosa aggiungerà valore al proprio modello?".
- Le macchine a stati vengono utilizzate nell'ultima parte della fase di Elaborazione e nella prima parte della fase di Costruzione.
- Il problema principale per l'utente è come eseguire il test delle macchine a stati.
- Con la maggior parte degli strumenti di modellazione di UML non si ha a disposizione alcuna opzione se non eseguire una simulazione importante manuale.

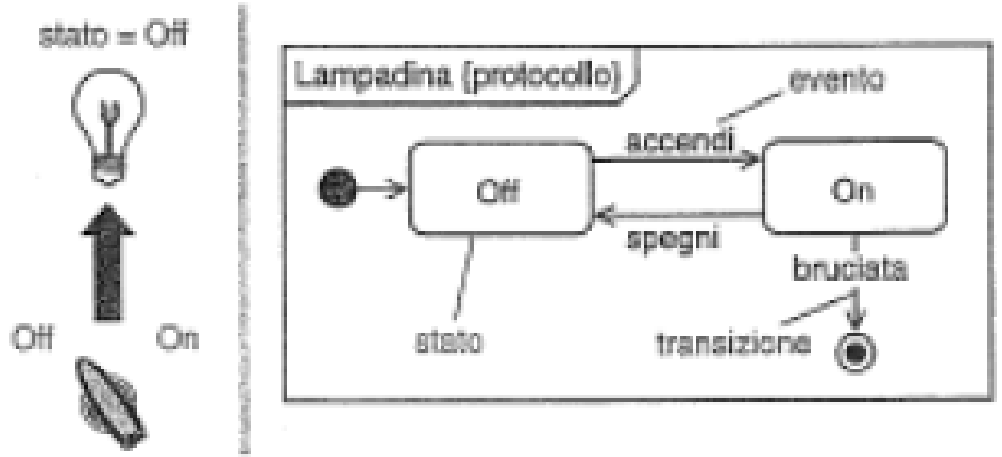
Macchine a stati – Macchine a stati e UP

- Tuttavia, il modo migliore per creare ed eseguire il test delle macchine a stati è simularle; molti strumenti consentono di farlo tra cui, per esempio, Real Time Studio di Artisan Software.

- Alcuni strumenti consentono anche di generare il codice che realizza il comportamento delle macchine a stati.

Macchine a stati – Diagrammi delle macchine a stati

- Per spiegare le macchine a stati si esamina un semplice esempio, preso dal mondo reale.
- La figura seguente illustra come sia possibile inviare eventi a una lampadina utilizzando un interruttore:



- I due eventi che possono essere inviati sono `accendi` e `spegni`.
- Un diagramma delle macchine a stati contiene esattamente una macchina a stati per ogni singola entità reattiva.

Macchine a stati – Diagrammi delle macchine a stati

- Il diagramma delle macchine a stati può essere disegnato in una cornice esplicita oppure può esistere all'interno di cornici implicite fornite da uno strumento di modellazione.
- Se si vuole si può premettere `Macchina a stati` al nome della macchina a stati, ma raramente è necessario.
- Le macchine a stati hanno una sintassi ben identificabile:
  - Gli stati sono rappresentati con rettangoli arrotondati, tranne lo stato iniziale (disco nero) e lo stato finale (disco nero bordato).
  - Le transizioni indicano un possibile percorso tra due stati e sono modellate con delle frecce.
  - Gli eventi sono scritti sopra la transizione che attivano.
- La semantica di base è altrettanto semplice: quando un'entità reattiva che si trova nello stato `A` riceve l'evento `unEvento` può effettuare una transizione allo stato `B`.
- Ogni macchina a stati dovrebbe avere uno stato iniziale (disco nero) che indica il primo stato della sequenza e uno stato finale (disco nero bordato) che termina la sequenza delle transizioni.
- Nelle macchine a stati gli eventi sono da considerarsi istantanei.

Macchine a stati – Diagrammi delle macchine a stati

- Gli eventi istantanei consentono di operare una notevole semplificazione della teoria delle macchine a stati.
- Gli oggetti possono cambiare stato quando ricevono un evento; quando la lampadina riceve l'evento `spegni` torna allo stato `Off`.
- In qualunque momento potrebbe verificarsi l'evento `bruciata` e questo termina la macchina a stati.

Macchine a stati – Diagrammi delle macchine a stati – Stati

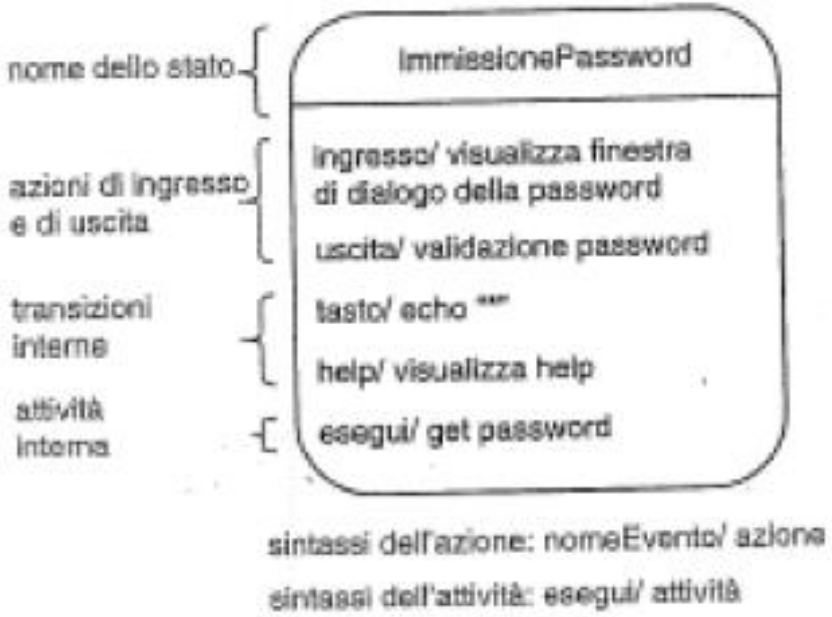
- Uno stato può essere definito come:

  Una condizione o una situazione della vita di un oggetto durante la quale tale oggetto soddisfa una condizione, esegue un'attività o aspetta un evento

- Lo stato di un oggetto varia nel corso del tempo, ma in qualsiasi momento è determinato da:
  - I valori dei suoi attributi.
  - Le relazioni che ha con altri oggetti.
  - Le attività che sta eseguendo.
- I messaggi scambiati sono eventi che possono provocare una transizione di stato degli oggetti.
- È importante decidere quale debba essere esattamente il significato di "stato": è necessario individuare gli stati che rivestono interesse per il sistema e per i suoi utenti.
- Ad esempio, nel caso di una lampadina interessano solo gli stati `on`, `off` e `bruciata` e non tutti i diversi valori della tensione che consentono di tenere accesa la lampadina.

Macchine a stati – Diagrammi delle macchine a stati – Stati – Sintassi per gli stati

- Un esempio della sintassi UML per gli stati viene riportato nella seguente figura:



- Ogni stato in una macchina a stati del comportamento può contenere zero o più azioni e attività.
- Gli stati nelle macchine a stati del protocollo non hanno azioni o attività.
- Le azioni sono considerate istantanee mentre le attività richiedono una quantità finita di tempo e possono essere interrotte.

Macchine a stati – Diagrammi delle macchine a stati – Stati – Sintassi per gli stati

- In uno stato ogni azione è associata ad una transizione interna che viene attivata da un evento.
- Uno stato può contenere un qualunque numero di azioni e transizioni interne.
- Una transizione interna consente di evidenziare che è successo qualcosa che vale la pena di modellare ma che non provoca la transizione ad un nuovo stato.
- Esistono due azioni speciali, ovvero l'azione di ingresso e l'azione di uscita che sono associate agli eventi speciali `ingresso` e `uscita`.
- L'evento `ingresso` è la prima cosa che succede quando si entra in questo stato e determina l'azione di ingresso associata.
- L'evento `uscita` è l'ultimissima cosa che accade istantaneamente e automaticamente all'uscita dallo stato e provoca l'esecuzione dell'azione di uscita associata.
- L'esecuzione di un'attività dura un intervallo di tempo finito e può essere interrotta dalla ricezione di un evento.

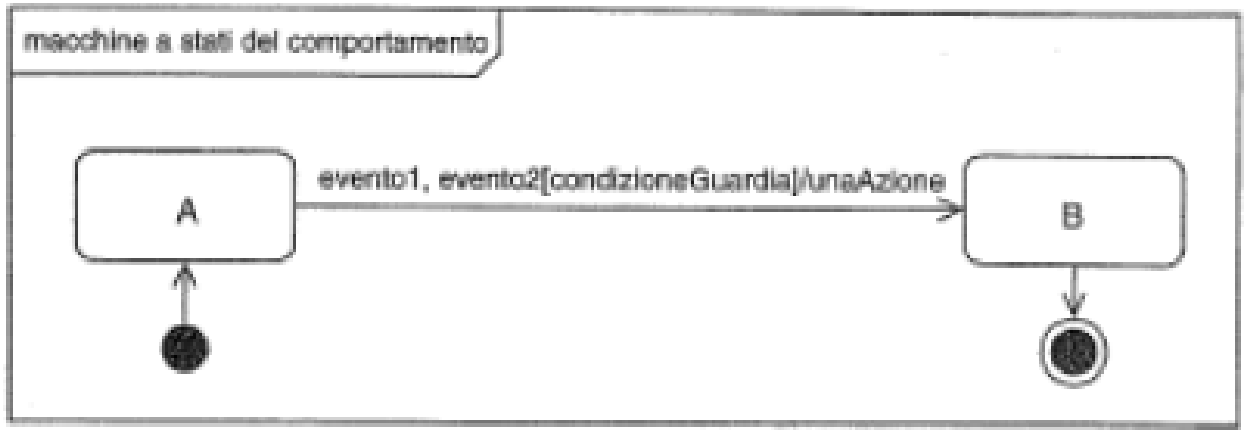
Macchine a stati – Diagrammi delle macchine a stati – Stati – Sintassi per gli stati

- La parola chiave `esegui` indica un'attività.

- Mentre le azioni, in quanto atomiche, terminano sempre la loro esecuzione, è possibile invece interrompere un'attività prima che questa abbia finito l'esecuzione.

Macchine a stati – Diagrammi delle macchine a stati – Transizioni

- La seguente figura riassume la sintassi UML per le transizioni per le macchine a stati del comportamento:



- Le transizioni in una macchina a stati del comportamento hanno una sintassi semplice che può essere utilizzata per le transizioni esterne (indicate con una freccia) e per le transizioni interne (annidate all'interno di uno stato).

- Ogni transizione ha tre elementi opzionali, ovvero:

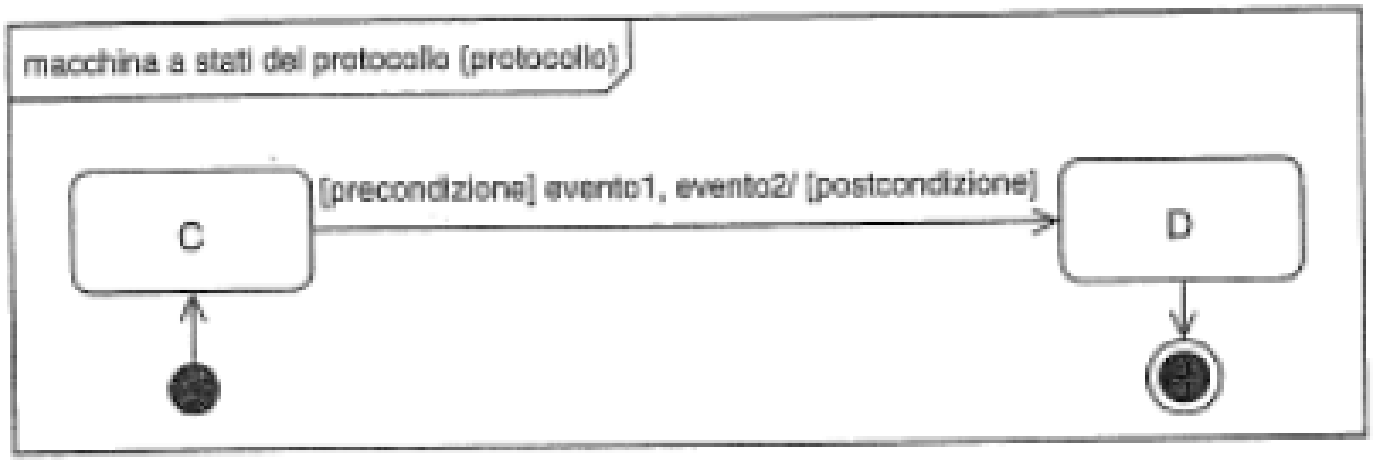
  - Zero o più eventi.

  - Zero o una condizione di guardia.

  - Zero o più azioni.

Macchine a stati – Diagrammi delle macchine a stati – Transizioni

- Nella figura precedente quando si verifica `evento1` o `evento2`, se la `condizioneGuardia` risulta vera, viene eseguita `unaAzione` e si entra immediatamente nello stato `B`.
- L'azione può coinvolgere delle variabili nell'ambito della macchina a stati.
- Esempio:
  - `azioneEseguita (azioneEvento) / comando = azioneEvento.getAzioneComando()`
- Le transizioni nelle macchine a stati del protocollo hanno una sintassi leggermente diversa, come illustrato nella seguente figura:



Macchine a stati – Diagrammi delle macchine a stati – Transizioni

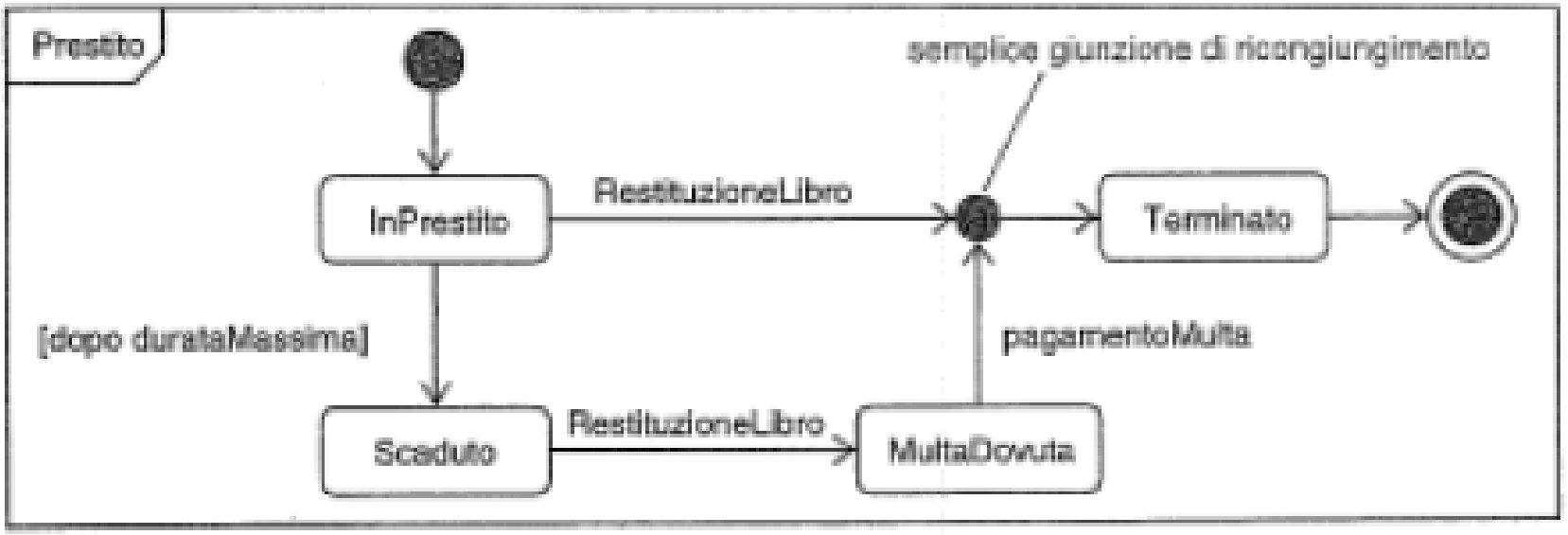
- In particolare:
  - Non c'è alcuna azione.
  - La condizione di guardia viene sostituita da precondizioni e postcondizioni.
- Nelle macchine a stati sia del protocollo che del comportamento se una transizione non ha alcun evento associato viene detta transizione automatica; essa scatta quando la condizione di guardia o la sua precondizione è vera.

Macchine a stati – Diagrammi delle macchine a stati – Transizioni – Connessione delle transizioni: lo pseudo-stato di giunzione

- Le transizioni possono essere connesse tramite pseudo-stati di giunzione che rappresentano punti in cui le transizioni si ricongiungono o si ramificano.

- Essi vengono rappresentati come dischi neri che hanno una o più transizioni di input e una o più transizioni di output.

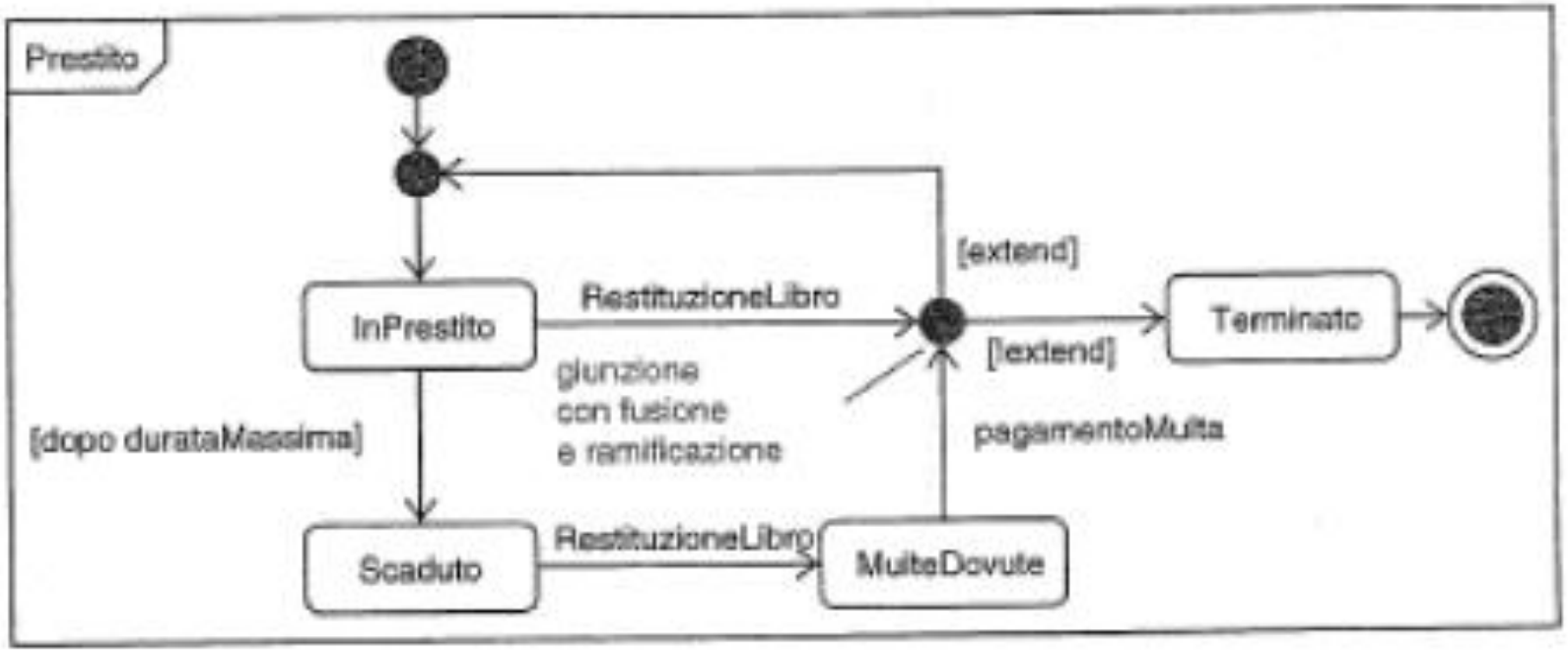
- Un esempio di pseudo-stato di giunzione viene riportato nella seguente figura:



Macchine a stati – Diagrammi delle macchine a stati – Transizioni – Connessione delle transizioni: lo pseudo-stato di giunzione

- Uno pseudo-stato di giunzione può avere più di una transizione di output e in questo caso ogni transizione deve essere protetta da una condizione di guardia mutuamente esclusiva.

- Un esempio di pseudo-stato di giunzione con più transizioni in uscita viene riportato nella seguente figura:

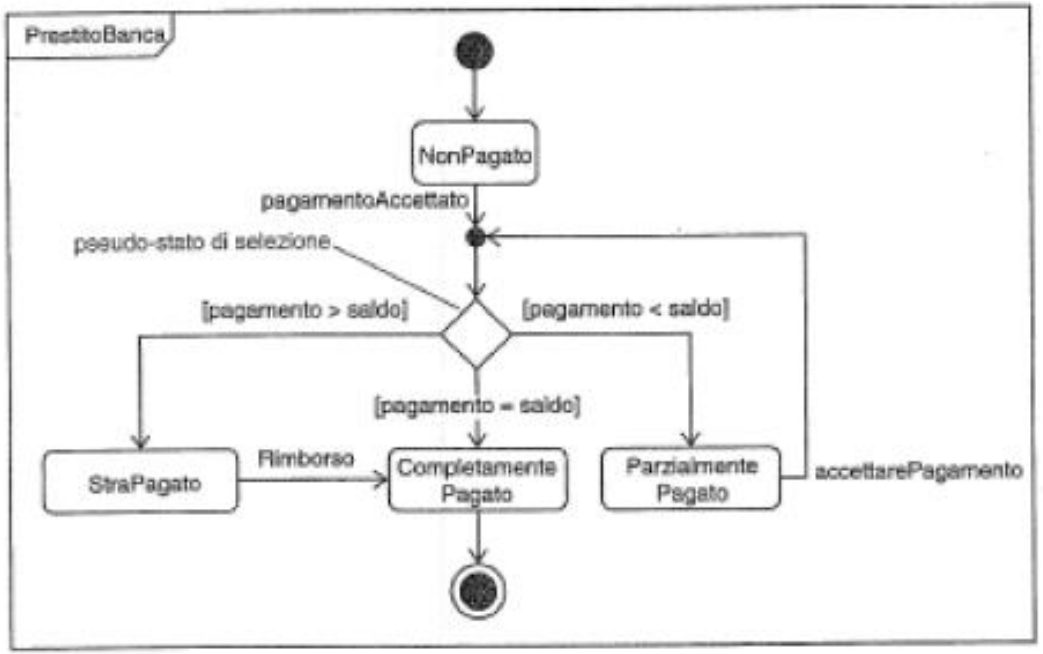


Macchine a stati – Diagrammi delle macchine a stati – Transizioni – Ramificazione delle transizioni: lo pseudo-stato di selezione

- Se si desidera mostrare una semplice ramificazione senza un ricongiungimento si dovrebbe usare uno pseudo-stato di selezione.

- Questo consente di dirigere il flusso attraverso la macchina a stati secondo le condizioni specificate sulle sue transizioni di output.

- Per esempio, la seguente figura mostra una macchina a stati del comportamento per una semplice classe `PrestitoBanca`:



Macchine a stati – Diagrammi delle macchine a stati – Transizioni – Ramificazione delle transizioni: lo pseudo-stato di selezione

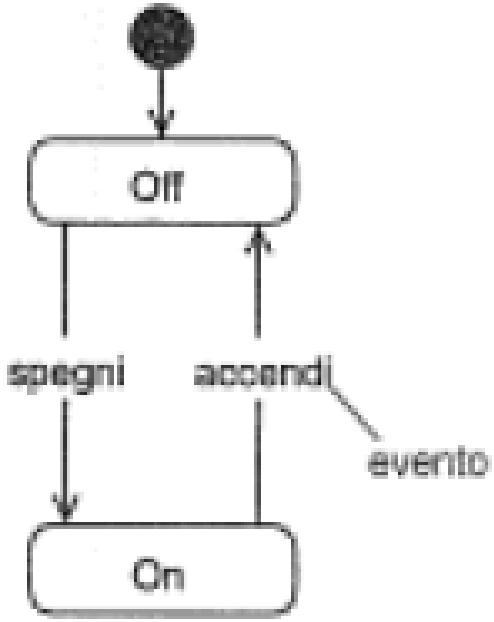
- Le condizioni delle transizioni in uscita dello pseudo-stato di selezione devono essere mutuamente esclusive per assicurare che in qualsiasi momento può scattare solo una di esse.

Macchine a stati – Diagrammi delle macchine a stati – Eventi

- UML definisce un evento come:

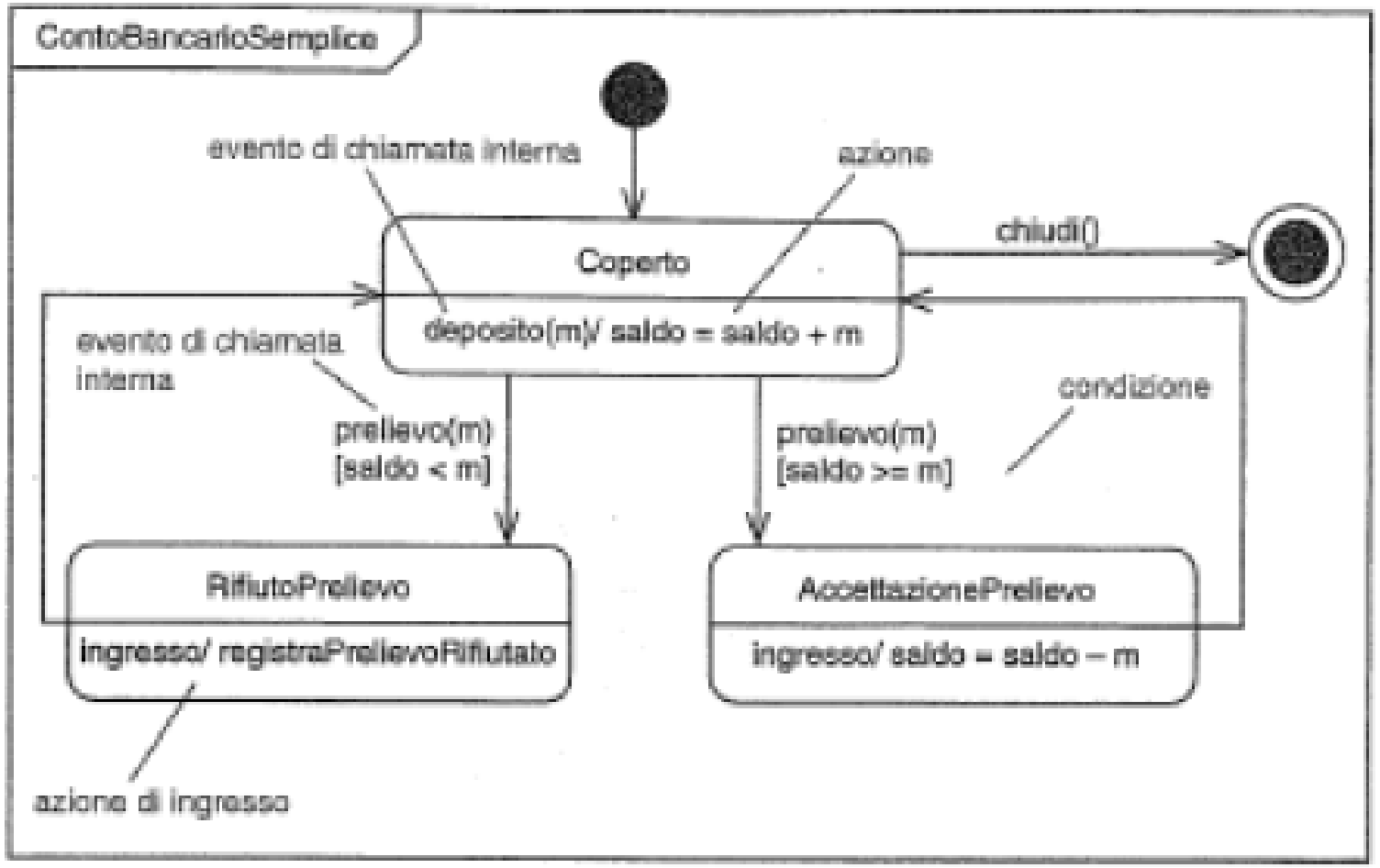
> La specifica di un'occorrenza di interesse che ha una collocazione nel tempo e nello spazio

- Gli eventi attivano le transizioni nelle macchine a stati.
- Gli eventi possono essere riportati esternamente sulle transizioni, come illustrato nella seguente figura:



Macchine a stati – Diagrammi delle macchine a stati – Eventi

- Gli eventi possono essere riportati internamente agli stati, come illustrato nella seguente figura:



Macchine a stati – Diagrammi delle macchine a stati – Eventi

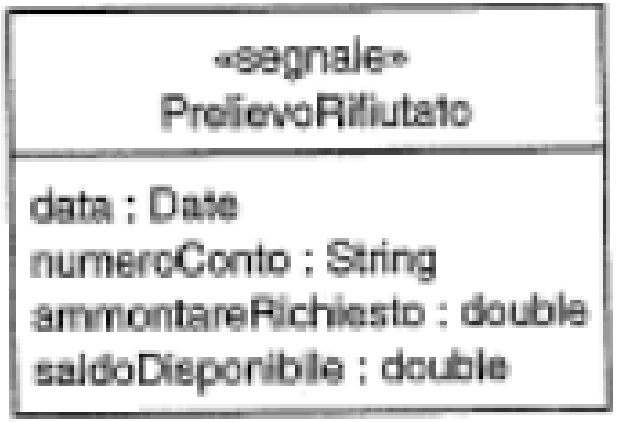
- Esistono quattro tipi di evento, ognuno dei quali ha una diversa semantica:
  - Evento di chiamata.
  - Evento di segnale.
  - Evento di variazione.
  - Evento temporale.

Macchine a stati – Diagrammi delle macchine a stati – Eventi – Eventi di chiamata

- L'evento di chiamata è la richiesta che una specifica operazione venga invocata all'istanza della classe di contesto.
- L'evento di chiamata dovrebbe avere la stessa segnatura di un'operazione della classe di contesto della macchina a stati.
- La ricezione dell'evento di chiamata è l'attivazione dell'operazione da eseguire.
- L'esempio della figura precedente riporta un frammento della macchina a stati relativa a una classe `ContoBancarioSemplice`. Questa classe è soggetta ai seguenti limiti di business:
  - I conti devono avere sempre un saldo superiore o uguale a zero.
  - Se il saldo risultasse inferiore a zero un prelievo verrebbe rifiutato.
- La figura mostra eventi di chiamata interni ed esterni che corrispondono a operazioni della classe `ContoBancarioSemplice`.
- È possibile associare a un evento di chiamata una sequenza di azioni, separate da punto e virgola.
- Se l'operazione ha un tipo restituito l'evento di chiamata ha un valore restituito di quel tipo.

Macchine a stati – Diagrammi delle macchine a stati – Eventi – Eventi di segnale

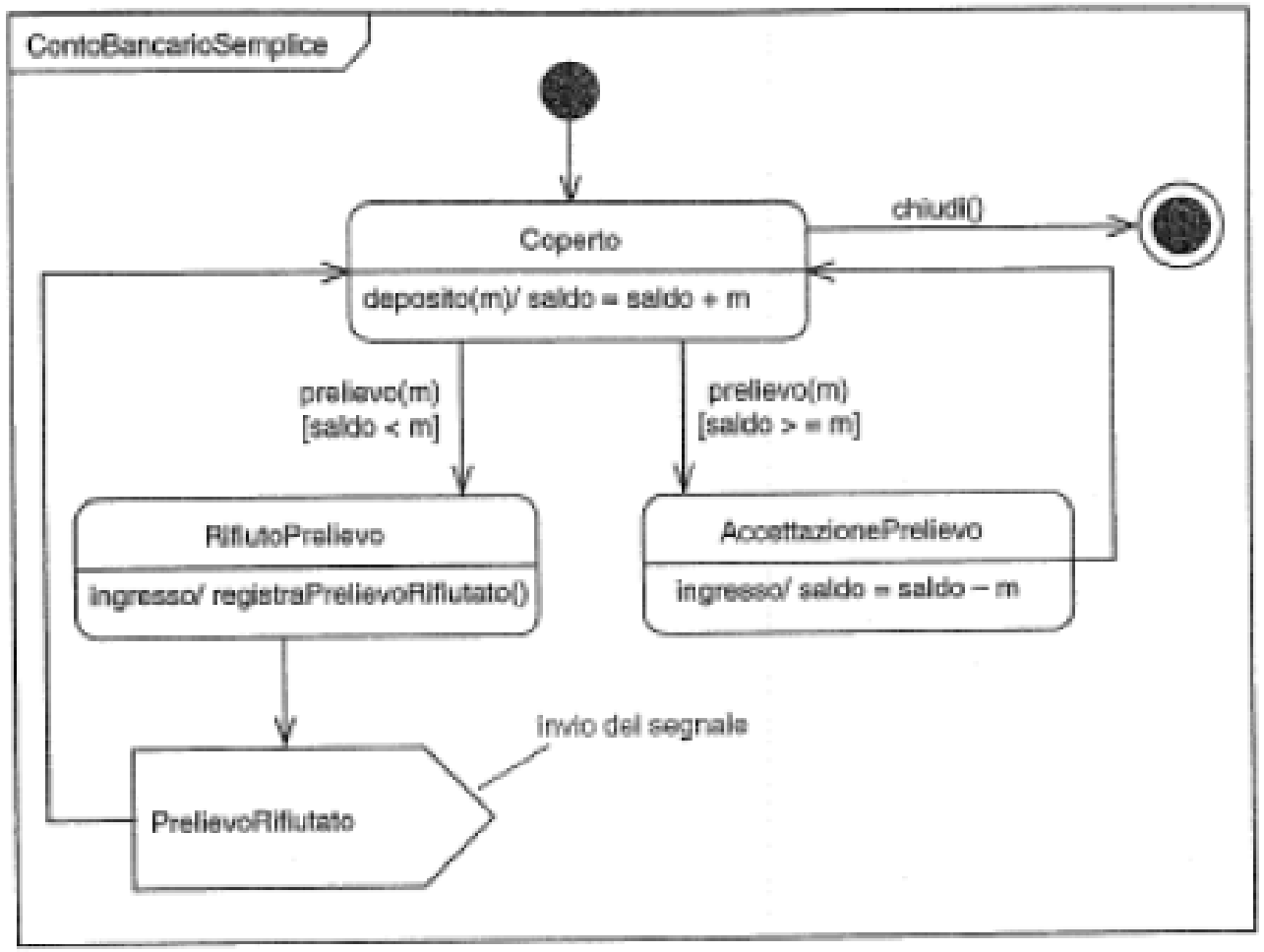
- Un segnale è un pacchetto di informazioni inviato in modo asincrono da un oggetto a un altro.
- Il segnale viene modellato come una classe con stereotipo <<segnale>> che contiene le informazioni che devono essere comunicate nei suoi attributi, come illustra la seguente figura:



- Un segnale di solito non ha operazioni perché il suo scopo è quello di trasportare informazioni.

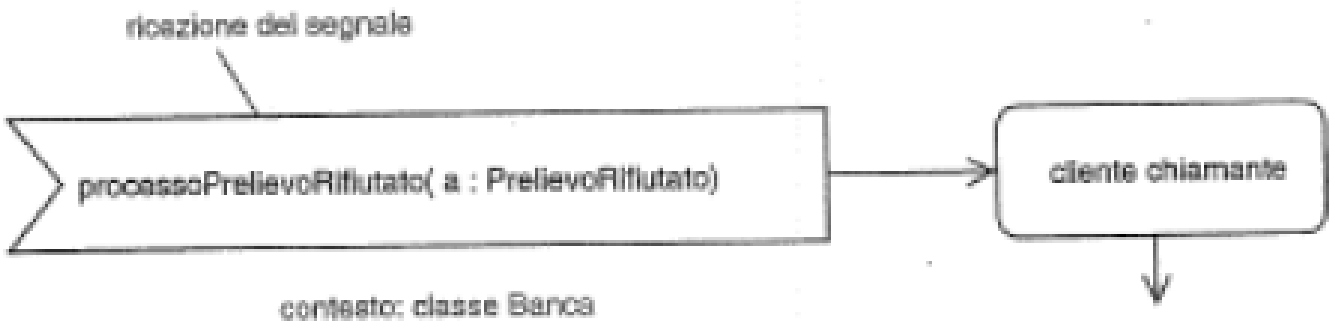
Macchine a stati – Diagrammi delle macchine a stati – Eventi – Eventi di segnale

- Nella figura seguente la macchina a stati per `ContoBancarioSemplice` è stata aggiornata così che invia un segnale quando viene rifiutato un prelievo.



Macchine a stati – Diagrammi delle macchine a stati – Eventi – Eventi di segnale

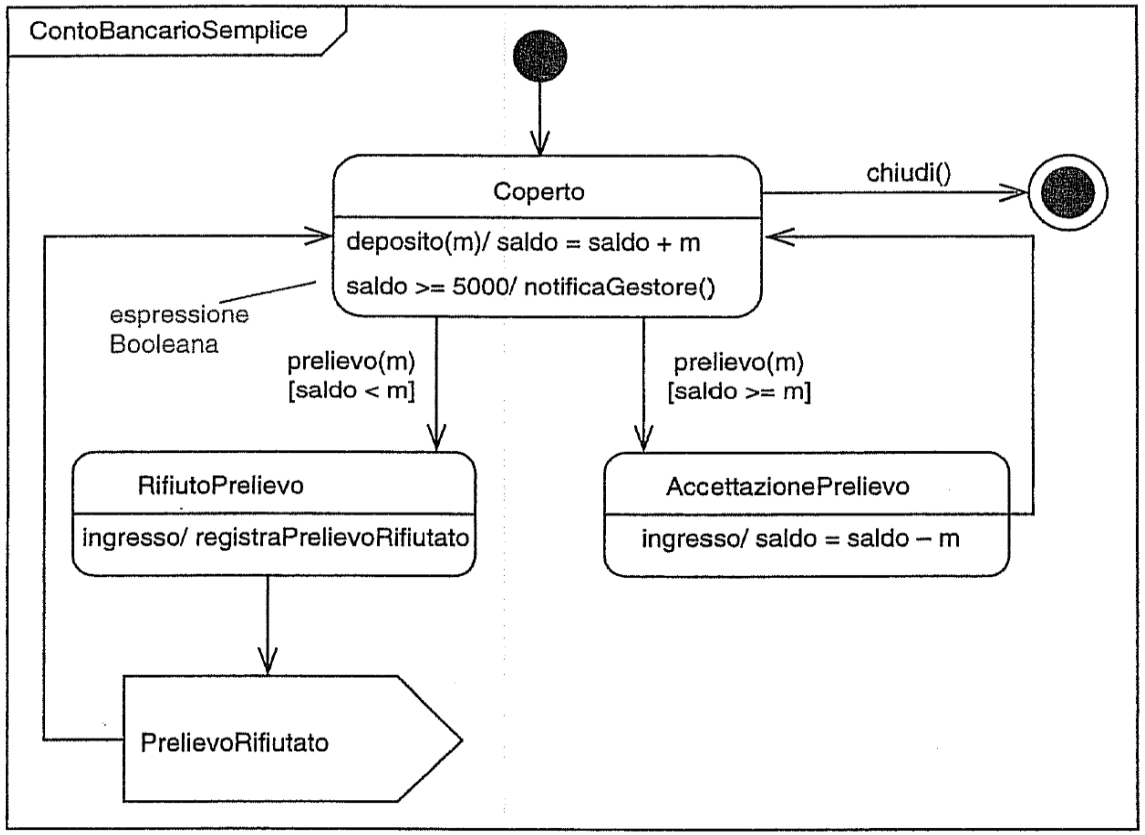
- L'invio di un segnale è indicato da un pentagono convesso contenente il nome del segnale.
- La ricezione di un segnale è indicata da un pentagono concavo, come mostrato nel frammento della macchina a stati della figura seguente:



- È possibile anche mostrare la ricezione dei segnali su transizioni interne ed esterne utilizzando la notazione standard nomeEvento/azione vista in precedenza.

Macchine a stati – Diagrammi delle macchine a stati – Eventi – Eventi di variazione

- L'evento di variazione è specificato con un'espressione booleana.
- Nella seguente figura viene riportato un evento di variazione:

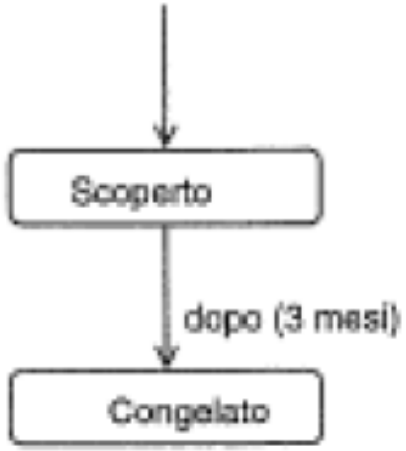


Macchine a stati – Diagrammi delle macchine a stati – Eventi – Eventi di variazione

- L'azione associata a questo tipo di evento viene eseguita quando il valore dell'espressione booleana passa da `falso` a `vero`.
- Dal punto di vista implementativo un evento di variazione implica un ciclo di test continuo della condizione booleana per tutta la durata dello stato.
- Gli eventi di variazione sono attivati al limite positivo; ciò significa che vengono attivati ogni volta che il valore dell'espressione booleana cambia da `falso` a `vero`.
- L'espressione booleana deve ritornare a `falso` e poi ripassare a `vero` perché l'evento di variazione venga riattivato.
- L'attivazione al limite positivo è esattamente il comportamento che si desidera per `ContoBancarioSemplice` mostrato nella figura precedente.

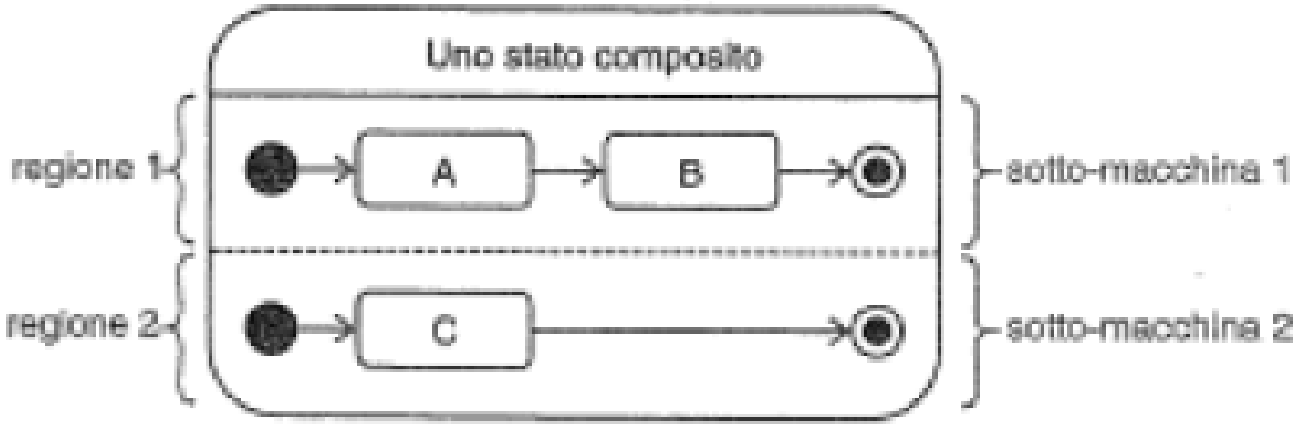
Macchine a stati – Diagrammi delle macchine a stati – Eventi – Eventi temporali

- Gli eventi temporali di solito sono indicati con le parole chiave `quando` e `dopo`.
- La parola chiave `quando` specifica un determinato momento in cui viene attivato l'evento.
- La parola chiave `dopo` specifica un intervallo di tempo al cui scadere viene attivato l'evento.
- Bisognerebbe assicurarsi che le unità temporali (ore, giorni, mesi, etc.) siano registrate sul diagramma per ciascun evento temporale.
- La figura seguente mostra una rappresentazione di un evento temporale:



Macchine a stati – Stati compositi

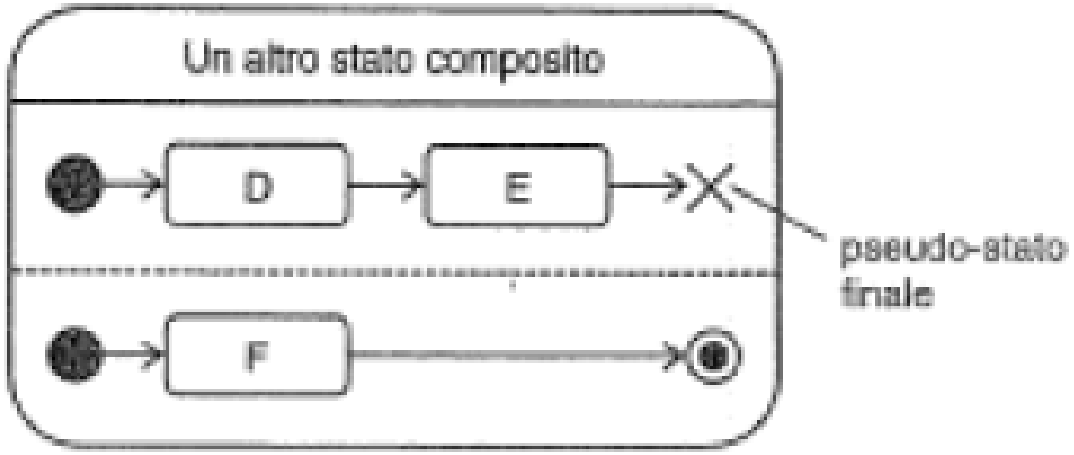
- Uno stato composito è uno stato che contiene altri stati annidati; questi sono organizzati in una o più macchine a stati chiamate sotto-macchine.
- Ciascuna sotto-macchina esiste nella propria regione all'interno dell'icona dello stato composito.
- Un esempio di stato composito viene mostrato nella seguente figura:



- Gli stati annidati ereditano tutte le transazioni degli stati che li contengono.
- Lo pseudo-stato finale della sotto-macchina viene applicato solo all'interno di quella regione.

Macchine a stati – Stati compositi

- Se si desidera fermare l'esecuzione dell'intero stato composito si può utilizzare lo pseudo-stato finale, come mostrato nella seguente figura:



- Anche gli stati annidati possono essere stati compositi; tuttavia, l'annidamento degli stati compositi dovrebbe essere costituito al massimo da due o tre livelli.

- È possibile indicare che uno stato è composito senza mostrare esplicitamente la sua scomposizione aggiungendo l'icona di composizione allo stato.

Macchine a stati – Stati compositi

- Nella seguente figura viene mostrato l'utilizzo dell'icona di composizione per rappresentare uno stato composito.

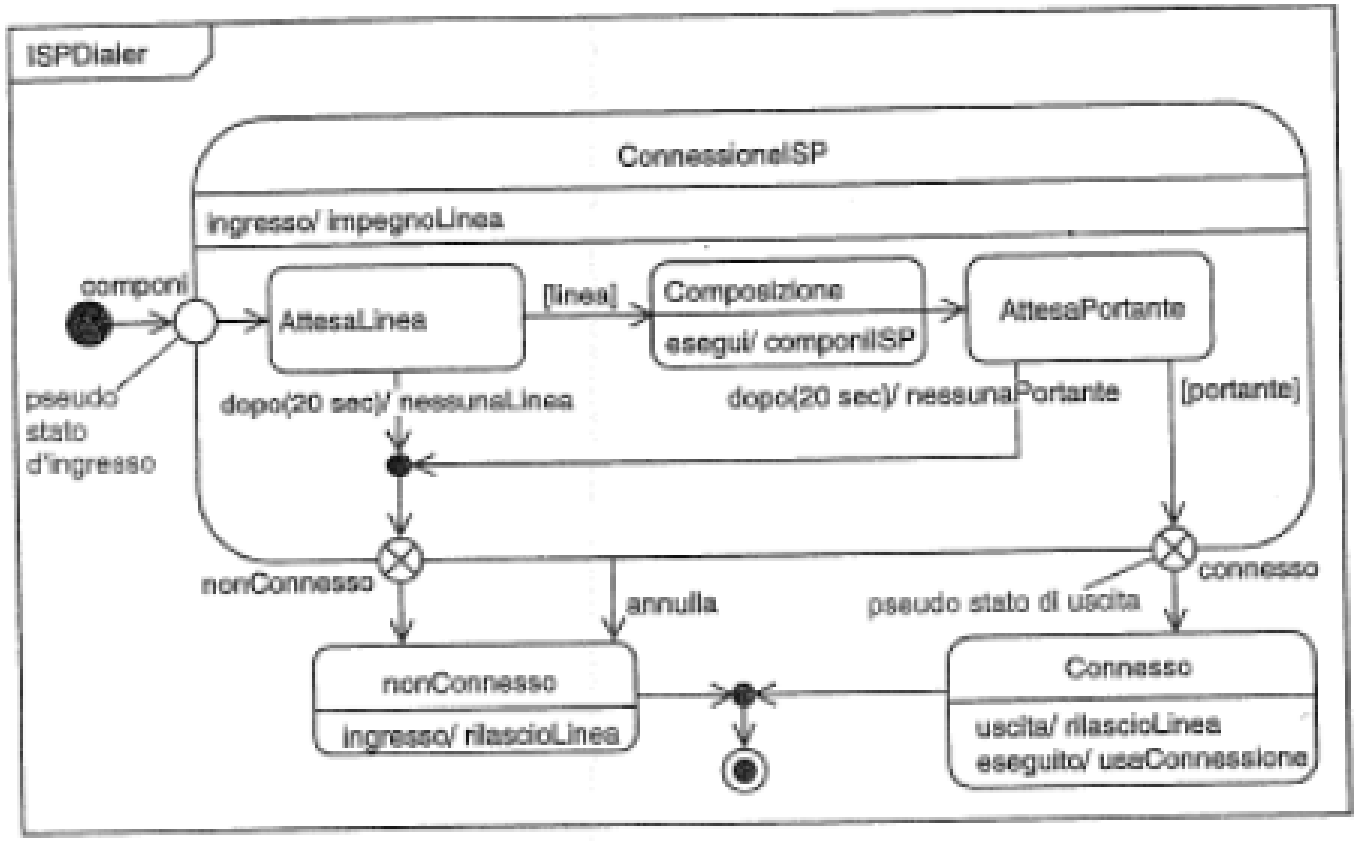


- Esistono due tipi di stato composito che dipendono dal numero delle regioni:
  - Stato composito semplice, quando è presente una sola regione.
  - Stati compositi ortogonali, quando sono presenti due o più regioni.

Macchine a stati – Stati compositi – Stati compositi semplici

- Uno stato composito semplice contiene una singola regione.

- Per esempio la seguente figura mostra la macchina a stati con uno stato composito semplice per una classe chiamata `ISPDialer`.



Macchine a stati – Stati compositi – Stati compositi semplici

- Ci sono due punti interessanti riguardanti questa macchina a stati:
  - La transizione in uscita da `ConnessioneISP` e attivata dall'evento `annulla` viene ereditata da ogni sottostato nella sotto-macchina `ConnessioneISP`.
  - `ConnessioneISP` ha uno pseudo-stato di ingresso chiamato `componi` e due pseudo-stati di uscita chiamati `nonConnesso` e `connesso`.
    - Uno pseudo-stato di ingresso è rappresentato come un disco nero di solito posto sul bordo dello stato composito (sebbene possa essere messo anche all'interno del bordo).
    - Lo pseudo-stato di uscita, rappresentato come un cerchio con una croce, indica un'uscita dalla sotto-macchina.
- Gli pseudo-stati di ingresso e di uscita sono molto utili perché consentono di definire modi diversi di ingresso e uscita dagli stati compositi.
- Essi forniscono, inoltre, punti di connessione a cui è possibile collegare transizioni a/da altri stati.

Macchine a stati – Stati compositi – Stati compositi semplici

- Una spiegazione dettagliata della macchina a stati della classe `ISPDialer` viene riportata nella seguente figura:

1. Si entra nel superstato `ConnessioneISP` attraverso lo pseudo-stato di ingresso `componi` e immediatamente viene eseguita l'azione di ingresso `impegnoLinea`.
2. Si entra nella regione singola e nello stato `AttesaLinea`.
3. Si resta nello stato `AttesaLinea` per un massimo di 20 secondi.
4. Se non si ottiene la linea entro 20 secondi:
   - 4.1 Si esegue l'azione `NessunaLinea` e si effettua una transizione nello stato `NonConnesso`, tramite lo pseudo-stato di uscita `nonConnesso`.
   - 4.2 All'ingresso dello stato `NonConnesso` si esegue il rilascio della linea (azione `rilascioLinea`).
   - 4.3 Si effettua una transizione allo stato `fine`.
5. Se si ottiene la linea (cioè se la condizione di guardia `[linea]` risulta `vera`) entro 20 secondi:
   - 5.1 Si effettua una transizione nello stato `Composizione` in cui si esegue l'attività `componiISP`.
   - 5.2 Appena termina l'attività `componiISP` si effettua automaticamente una transizione nello stato `AttesaPortante`.
   - 5.3 Si resta nello stato `AttesaPortante` per un massimo di 20 secondi.
   - 5.4 Se non si ottiene una portante entro 20 secondi:
     - 5.4.1 Si esegue l'azione `NessunaPortante` mentre si effettua una transizione nello stato `NonConnesso` tramite lo pseudo-stato di uscita `nonConnesso`.
     - 5.4.2 All'entrata di `NonConnesso` si esegue il rilascio della linea.

Macchine a stati – Stati compositi – Stati compositi semplici

5.4.3 Si effettua una transizione nello stato `fine`.

5.5 Se si ottiene una portante entro 20 secondi:

5.5.1 Avviene automaticamente una transizione dal superstato `ConnessioneISP` allo stato `Connesso`, tramite lo pseudo-stato di uscita `connesso`.

5.5.2 Si esegue l'azione `usaConnessione` fino a quando non è terminata.

5.5.3 All'uscita da `Connesso` si esegue il rilascio della linea.

5.4.4 Si effettua una transizione allo stato `fine`.

6. Se in qualunque momento, mentre si è nel superstato `ConnessioneISP`, si riceve l'evento `annulla`:

6.1 Si effettua immediatamente una transizione nello stato `NonConnesso`.

6.2 All'ingresso di `NonConnesso` si esegue il rilascio della linea.

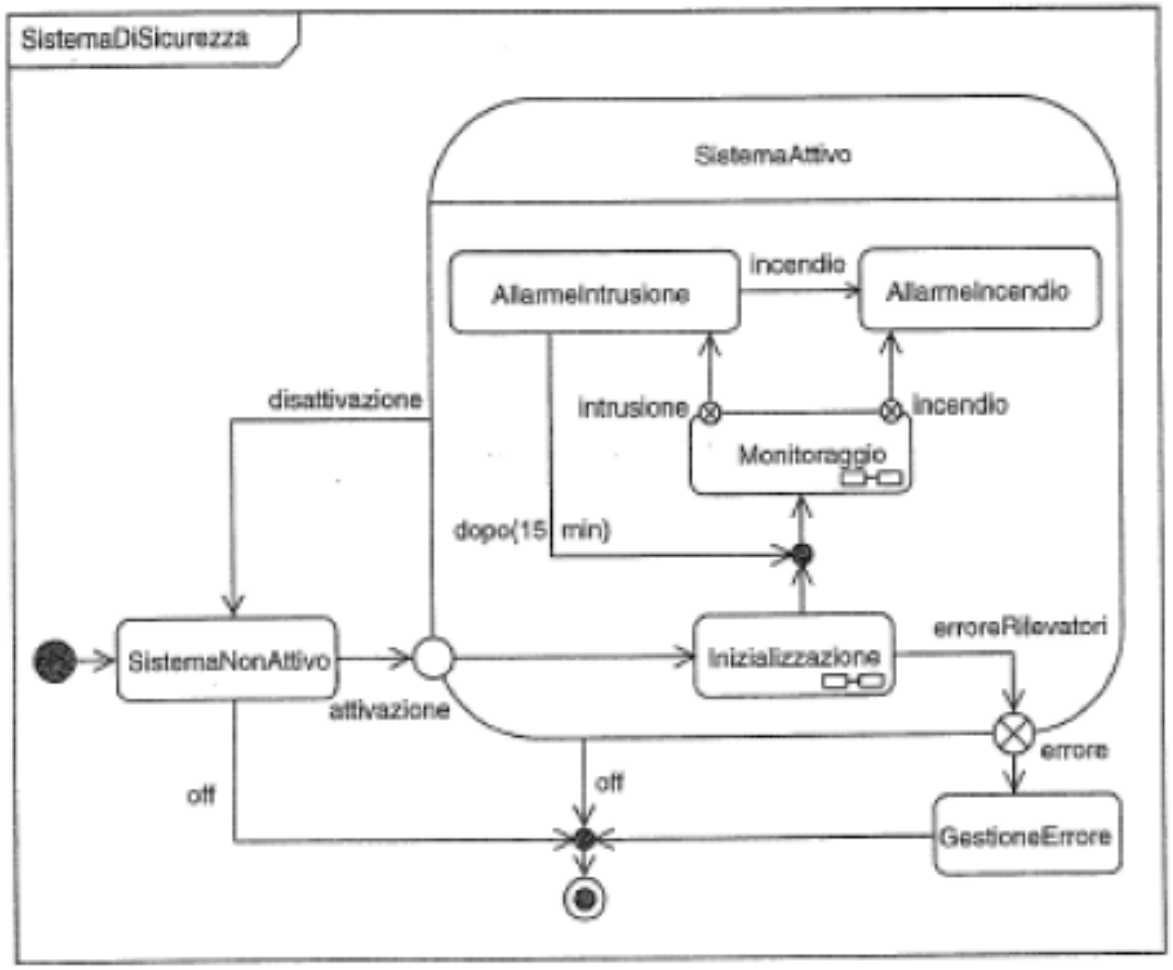
6.3 Si effettua una transizione nello stato `fine`.

Macchine a stati – Stati compositi – Stati compositi ortogonali

- Gli stati compositi ortogonali contengono due o più sotto-macchine che effettuano l'esecuzione parallelamente.
- Quando si entra nello stato composito tutte le sotto-macchine iniziano immediatamente l'esecuzione: si tratta di una biforcazione implicita.
- Esistono due modi per uscire dallo stato composito:
  - Tutte le sotto-macchine terminano l'esecuzione: questa è una ricongiunzione implicita.
  - Una delle sotto-macchine esegue una transizione a uno stato esterno al superstato, di solito tramite uno pseudo-stato di uscita.
    - Ciò non causa una ricongiunzione: non avviene alcuna sincronizzazione delle sotto-macchine e le altre sotto-macchine vengono semplicemente interrotte.

Macchine a stati – Stati compositi – Stati compositi ortogonali

- Consideriamo il seguente sistema di sicurezza costituito da un apparato di controllo, alcuni rilevatori di intrusione e di incendio e un allarme:

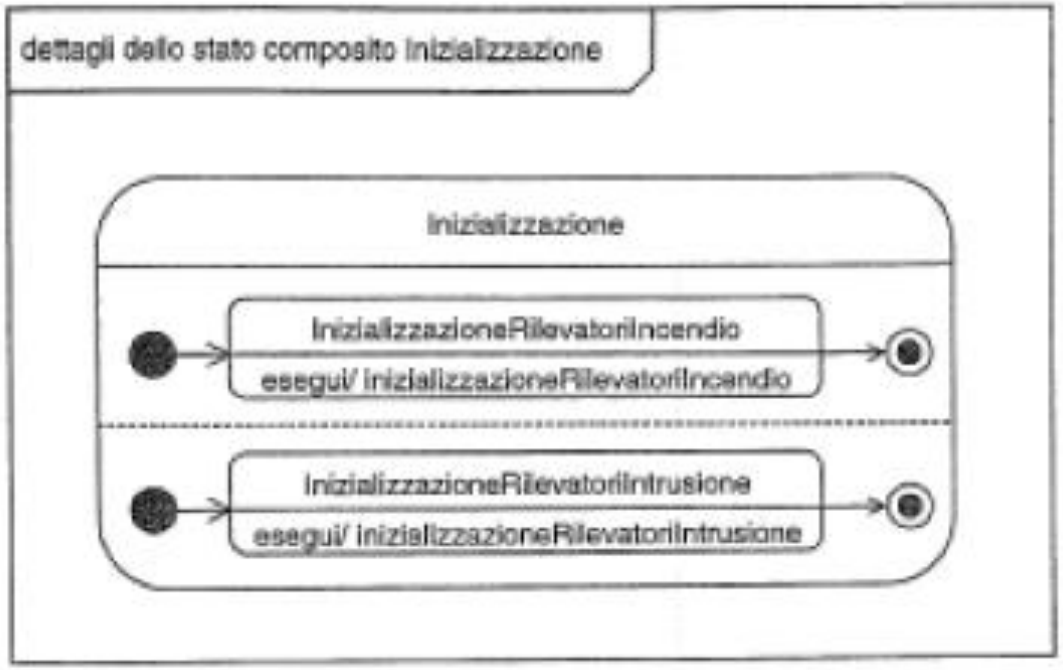


Macchine a stati – Stati compositi – Stati compositi ortogonali

- Questa macchina a stati esprime due caratteristiche fondamentali del sistema di allarme.
- Quando scatta il rilevatore di intrusione viene attivato l'allarme antifurto che suona per 15 minuti, dopo di che il sistema ritorna allo stato `Monitoraggio`.
- Se avviene un incendio mentre l'allarme antifurto sta suonando viene effettuata una transizione immediata dallo stato `AllarmeIntrusione` allo stato `AllarmeIncendio` e l'allarme antincendio suona.
- `Inizializzazione` e `Monitoraggio` sono stati compositi.

Macchine a stati – Stati compositi – Stati compositi ortogonali

- La figura seguente mostra l'espansione dello stato composito `Inizializzazione`:



- Quando si entra in questo stato avviene una biforcazione e due sotto-macchine iniziano l'esecuzione parallelamente.

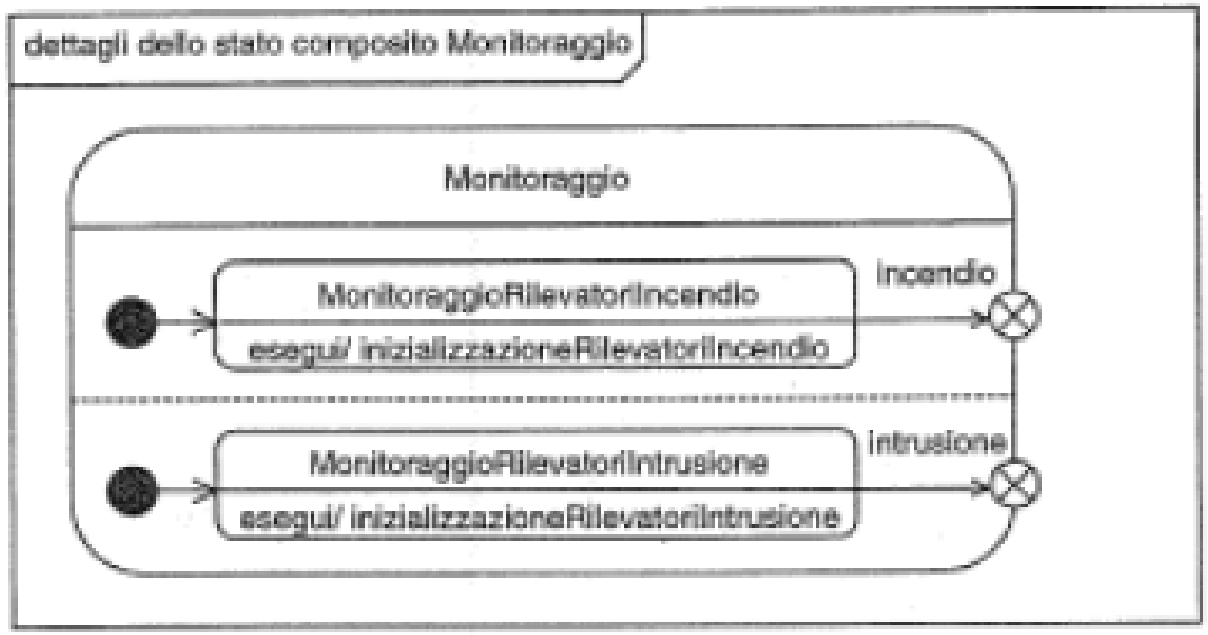
- In condizioni normali viene effettuata automaticamente la transizione dal superstato `Inizializzazione` quando entrambe le sotto-macchine hanno terminato l'esecuzione.

Macchine a stati – Stati compositi – Stati compositi ortogonali

- Esiste anche una transizione `erroreRilevatori` dello stato `Inizializzazione` che viene ereditata da entrambi i suoi sottostati.

- Talvolta si desidera far partire thread di controllo concorrenti senza doverli sincronizzare con una ricongiunzione quando terminano.

- Questo è il caso dello stato composito `Monitoraggio` mostrato nel diagramma di macchine a stati delle figura seguente:



Macchine a stati – Stati compositi – Stati compositi ortogonali

- Questo stato ha alcune caratteristiche interessanti:
  - Non esiste una sincronizzazione tra le due sotto-macchine:
    - Quando si presenta l'evento `incendio` viene eseguita una transizione esplicita da `MonitoraggioRilevatoriIncendio` allo pseudo-stato di uscita `Incendio`, che comporta l'uscita da `Monitoraggio`.
    - Quando si presenta l'evento `intrusione` viene effettuata una transizione esplicita da `MonitoraggioRilevatoriIntrusione` allo pseudo-stato di uscita `Intrusione` che comporta l'uscita da `Monitoraggio`.
  - Lo stato composito `Monitoraggio` e tutti i suoi sottostati ereditano la transizione `off` dal superstato `SistemaAttivo`.
- Questo esempio mostra come l'utilizzo di stati compositi concorrenti, con o senza sincronizzazione, consenta di modellare il parallelismo in modo molto efficace.

Macchine a stati – Stati della sotto-macchina

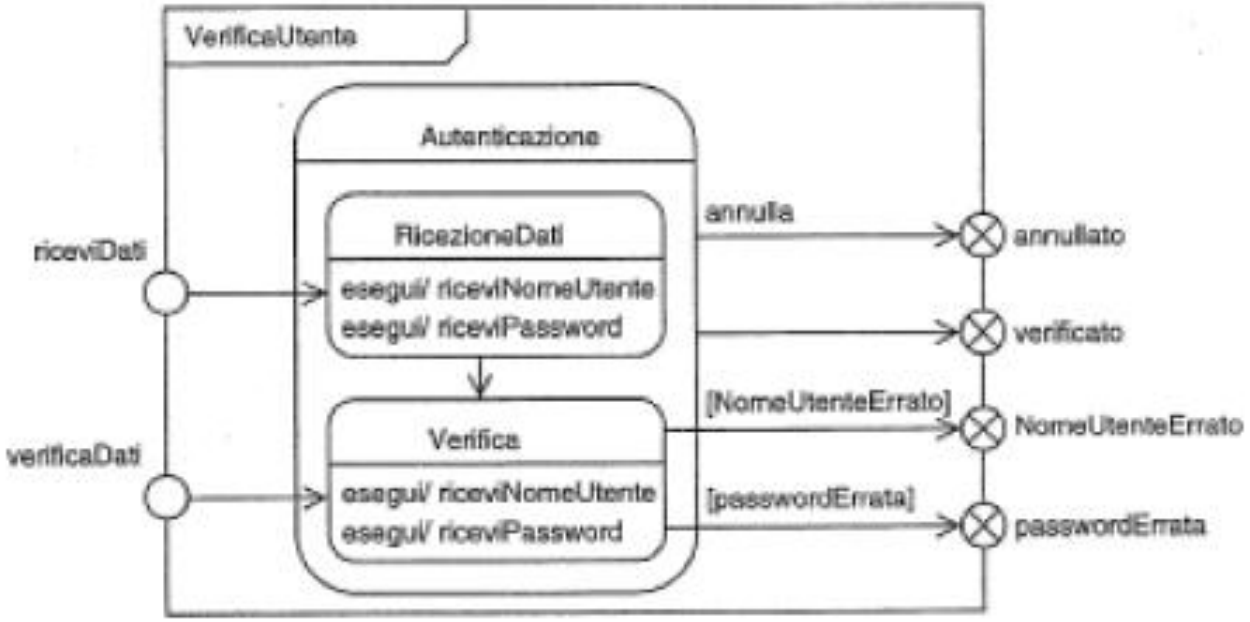
- Uno stato della sotto-macchina è uno stato speciale che fa riferimento a una macchina a stati riportata in un diagramma separato.
- Lo si può considerare simile a una chiamata di sub-routine da una macchina a stati all'altra.
- Gli stati delle sotto-macchine equivalgono semanticamente agli stati compositi.
- Gli stati delle sotto-macchine possono essere riutilizzati per semplificare macchine a stati complesse.
- Gli stati delle sotto-macchine offrono un modo per riutilizzare il comportamento.
- Per esempio si potrebbero avere due sistemi di sicurezza molto simili che hanno in comune lo stesso comportamento.

Macchine a stati – Stati della sotto-macchina

- Gli stati della sotto-macchina vengono denominati nel modo seguente:

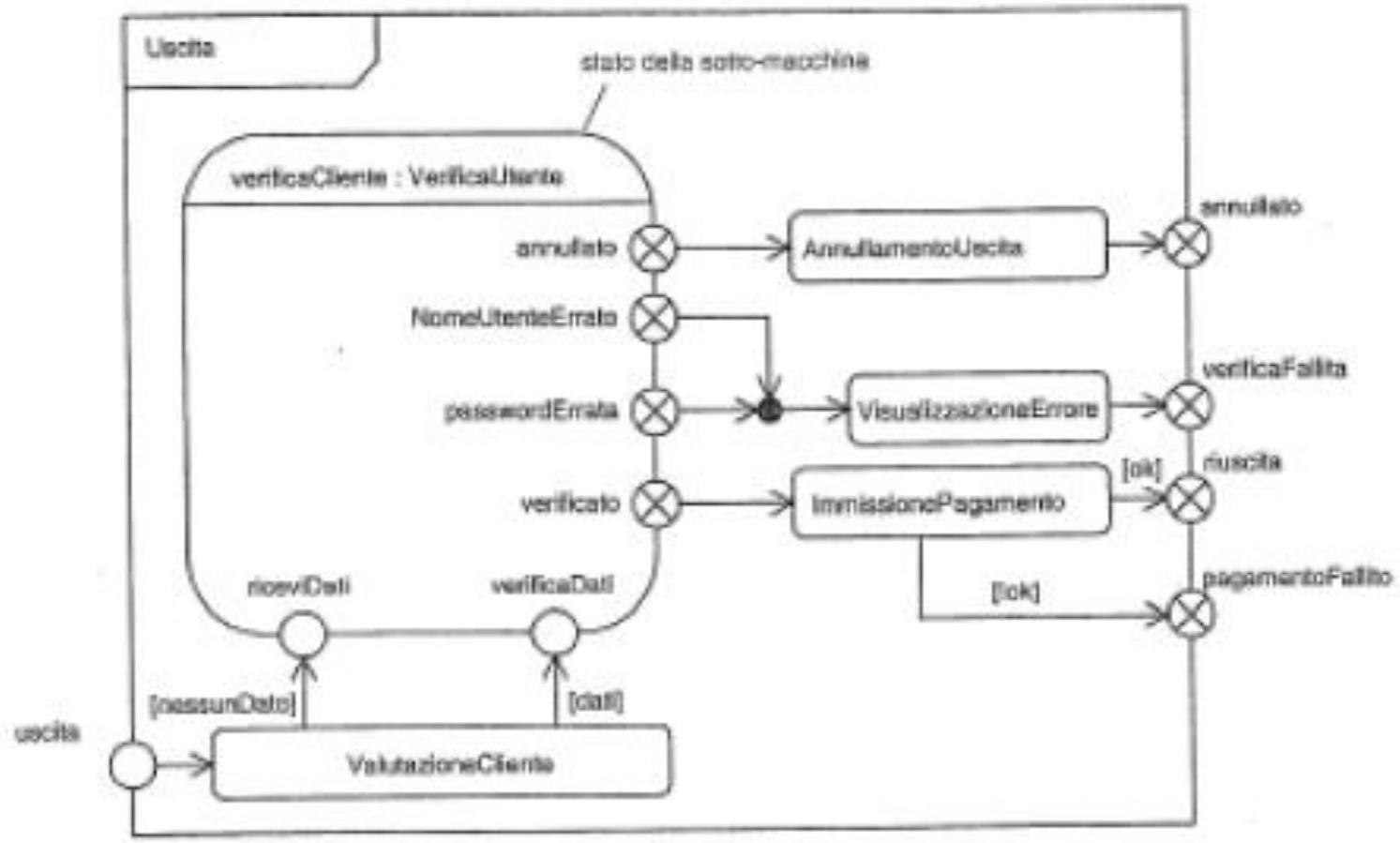
```
Nome dello stato: nome del diagramma della macchina a stati
                  a cui fa riferimento
```

- La figura seguente mostra un diagramma della macchina a stati che descrive il comportamento che si vorrebbe riutilizzare in un altro diagramma.



Macchine a stati – Stati della sotto-macchina

- Si noti che si possono visualizzare gli pseudo-stati di ingresso e di uscita sul bordo della cornice.
- Si può far riferimento alla macchina a stati precedente usando uno stato della sotto-macchina:

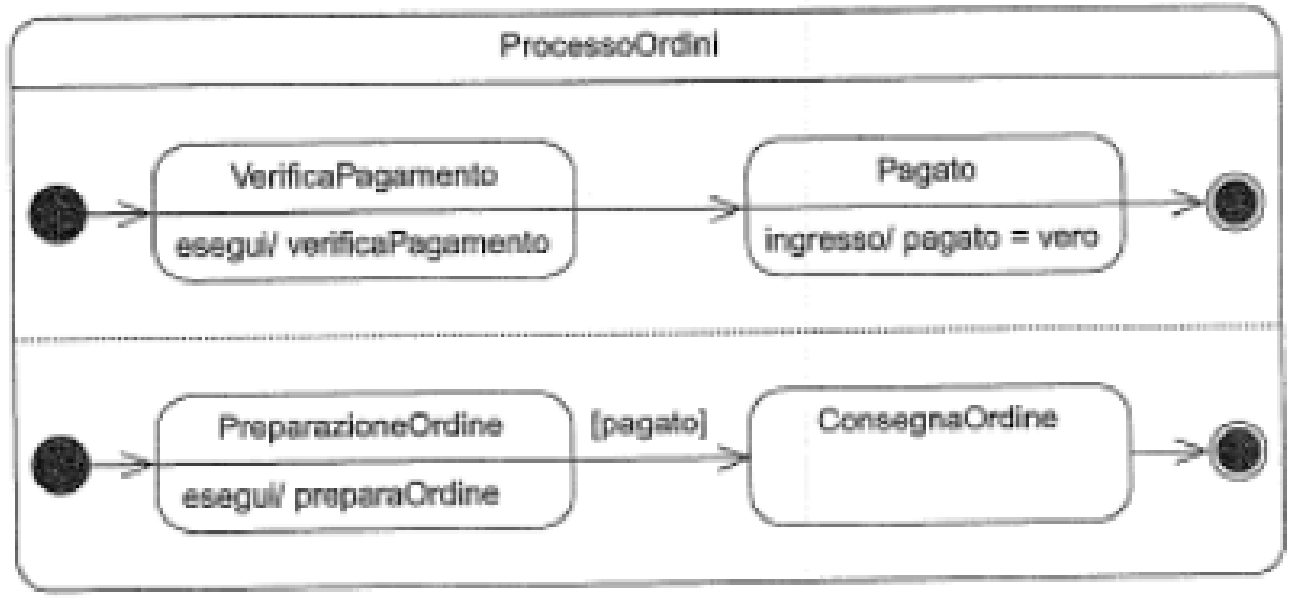


Macchine a stati – Comunicazione tra sotto-macchine

- In precedenza abbiamo visto l'utilizzo delle biforcazioni e delle ricongiunzioni per creare sotto-macchine concorrenti e per risincronizzarle.
- Si tratta di una forma di comunicazione sincrona tra sotto-macchine.
- Capita spesso di avere l'esigenza di far comunicare due sotto-macchine senza però volerle sincronizzare; si parla di comunicazione asincrona.
- UML prevede la comunicazione asincrona consentendo ad una sotto-macchina di lasciarsi dietro dei "messaggi" e degli "indicatori" pur continuando l'esecuzione; le altre sotto-macchine possono leggere questi indicatori quando hanno tempo per farlo.
- È possibile creare questi indicatori gestendo i valori degli attributi dell'oggetto di contesto della macchina a stati.

Macchine a stati – Comunicazione tra sotto-macchine

- Consideriamo la seguente figura:



- Nella sotto-macchina in alto, in uscita dallo stato `VerificaPagamento` avviene una transazione nello stato `Pagato`, in cui il valore dell'attributo `Pagato` viene impostato a `vero`.

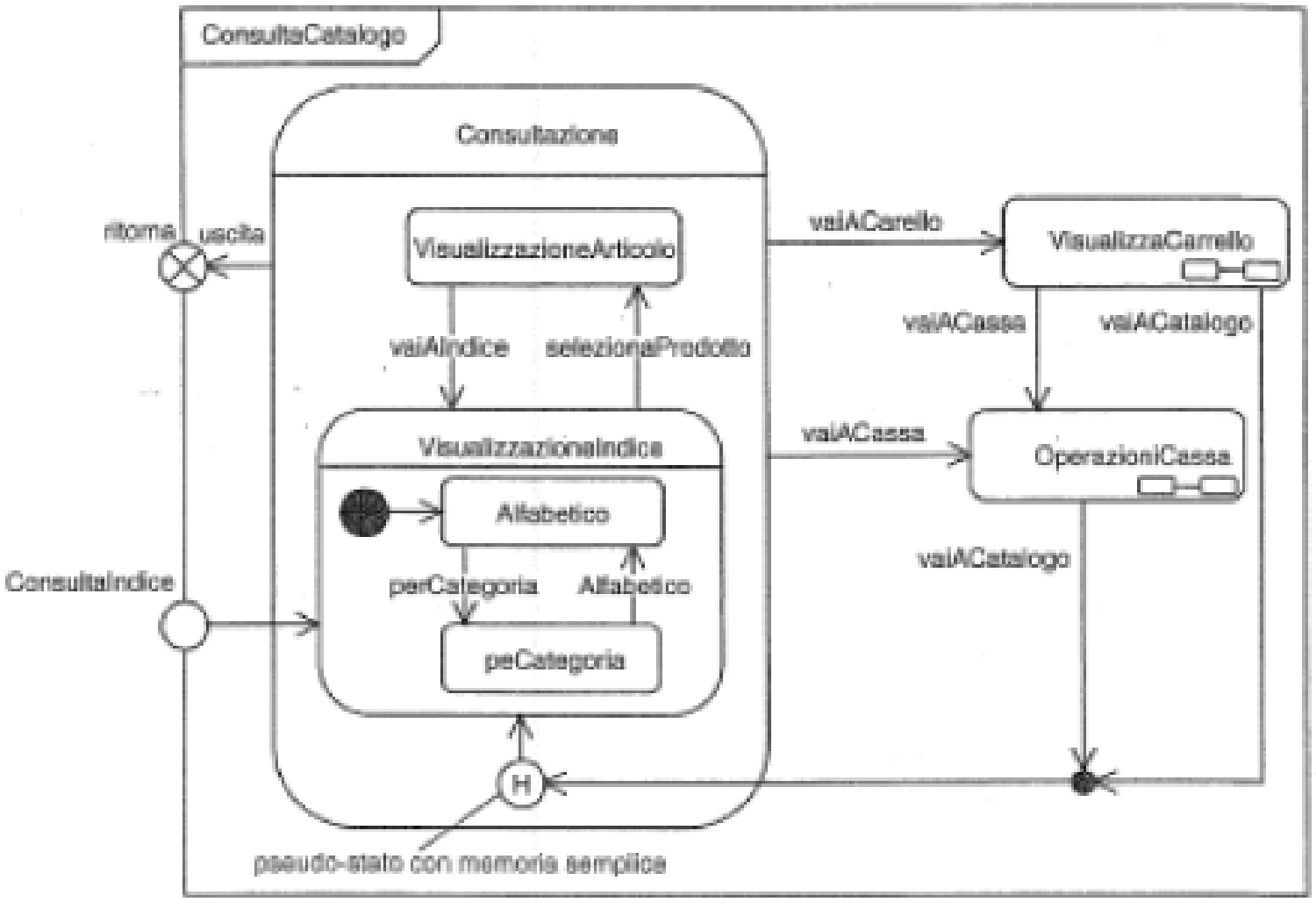
- Nella sotto-macchina in basso, quando si completa lo stato `PreparazioneOrdine` può avvenire una transizione nel nuovo stato `ConsegnaOrdine`, ma solo se l'attributo `Pagato` risulta `vero`.

Macchine a stati – Stati con memoria

- Modellando le macchine a stati capita spesso la seguente situazione:
  - La macchina si trova nel sottostato `A` di uno stato composito.
  - Avviene una transizione in uno stato esterno allo stato composito (e quindi anche al suo sottostato A).
  - Si passa attraverso uno o più altri stati esterni.
  - Avviene una transizione che riporta nello stato composito, ma si vorrebbe poter continuare l'esecuzione dal suo sottostato `A` dove è stata interrotta.
- Per gestire questa situazione lo stato composito deve in qualche modo ricordarsi del sottostato in cui si trovava quando è avvenuta la transizione verso lo stato esterno.
- UML prevede appositi pseudo-stati con memoria.
- Esistono due tipi di pseudo-stati con memoria:
  - Pseudo-stato con memoria semplice.
  - Pseudo-stato con memoria multilivello.

Macchine a stati – Stati con memoria – Stati con memoria semplice

- La figura seguente mostra una macchina a stati del caso d'uso `ConsultaCatalogo` di un sistema di e-commerce:



Macchine a stati – Stati con memoria – Stati con memoria semplice

- In questo caso è possibile uscire dal superstato `Consultazione` con tre diversi eventi:
  - `uscita`.
  - `vaiACarrello`.
  - `vaiACassa`.
- Tornando allo stato `Consultazione` da `VisualizzazioneCarrello` o da `OperazioniCassa` sarebbe preferibile per l'utente poter tornare sulla stessa pagina del catalogo da cui si è partiti.
- Lo pseudo-stato con memoria semplice ricorda in quale sottostato si era quando si è lasciato il superstato.
- Quando si torna da uno stato esterno allo stato con memoria, l'indicatore solitamente redireziona la transizione sull'ultimo sottostato memorizzato.
- Può avere molte transizioni in ingresso ma una sola in uscita.

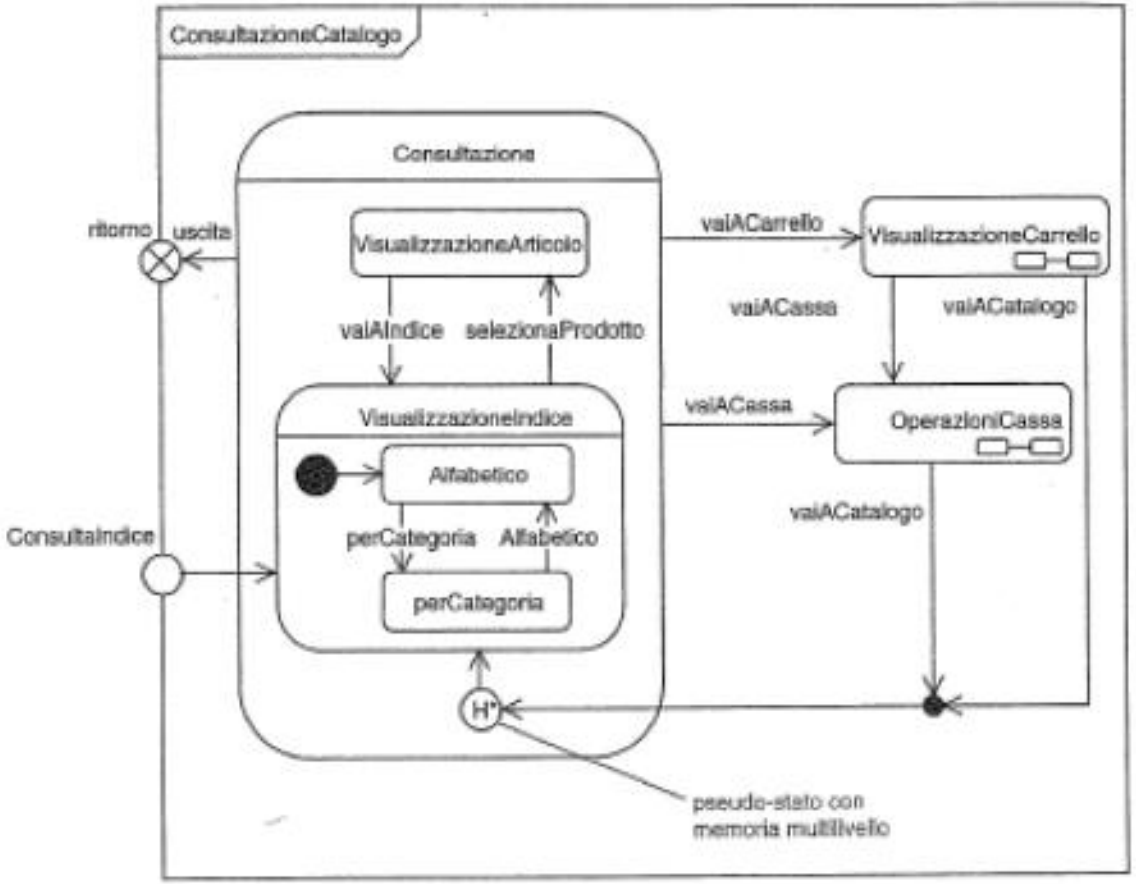
Macchine a stati – Stati con memoria – Stati con memoria semplice

- Se si tratta della prima volta che si entra nel superstato allora non esiste memoria dell'ultimo sottostato attivo (in questo caso `VisualizzazioneIndice` o `VisualizzazioneArticolo`).
- In questo caso viene attivata l'unica transizione in uscita dall'indicatore dello stato con memoria; nell'esempio illustrato si procederebbe al sottostato `VisualizzazioneIndice`.
- Gli stati con memoria consentono a un superstato di ricordarsi il sottostato che era attivo quando è avvenuta l'ultima transizione in uscita da esso.
- Tuttavia, nella figura precedente, anche il sottostato `VisualizzazioneIndice` è composto.
- Uno stato con memoria semplice non può ricordarsi dei sottostati di un suo sottostato; per questo ci vuole una memoria multilivello.

Macchine a stati – Stati con memoria – Stati con memoria semplice

- Uno stato con memoria multilivello può ricordarsi anche l'eventuale ultimo sottostato attivo di questo sottostato attivo, e così via, per un qualunque numero di stati annidati.

- Nella figura seguente la macchina a stati precedente è stata modificata per usare gli stati con memoria multilivello:



Macchine a stati – Stati con memoria – Stati con memoria semplice

- In questo caso non si ritornerà solo a `VisualizzazioneIndice` o `VisualizzazioneArticolo` ma si ritornerà al tipo corretto di indice (`Alfabetico` o `PerCategoria`).
- In linea di principio si potrebbe modellare lo stesso comportamento senza l'uso di stati con memoria multilivello, ma tale attività sarebbe molto più complessa.
- Come gli stati con memoria semplice, gli stati con memoria multilivello possono avere molte transizioni in ingresso ma una sola transizione in uscita.
- La transizione in uscita viene attivata se è la prima volta che si entra nel superstato e se non esiste memoria dell'ultimo sottostato attivo.